

Policy Compliant Secure Messaging

Joël Alwen¹, Xiaohui Ding² *, Sanjam Garg^{3,4}, and Yiannis Tselekounis²

¹ AWS Wickr

² Royal Holloway, University of London

³ University of California, Berkeley

⁴ Exponential Science Foundation

Abstract. We initiate the holistic study of Policy Compliant Secure Messaging (PCSM). A content policy is a predicate over messages deciding which messages are considered harmful and which not. A PCSM protocol is a type of end-to-end encrypted (E2EE) messaging system that guarantees E2EE privacy and authenticity for all policy compliant messages but detects and verifiably reports harmful content prior to its delivery. This stands in contrast to prior content moderation systems for E2EE messaging where detection relies on receivers reporting the harmful content themselves which makes them unsuited for most PCSM applications (e.g., for preventing the *wilful* distribution of harmful content). Our holistic PCSM notion explicitly captures several new roles such as policy creator, auditor and judge, to more accurately separate and model the different goals and security concerns of stakeholders when deploying PCSM.

We present efficient PCSM constructions for arbitrary policy classes, as well as for hash-based ones, achieving various levels of security, while maintaining the core security properties of the underlying E2EE layer. For hash-based PCSM, we encapsulate Apple’s recent PSI protocol used in their content moderation system, and we properly adapt it to realize the desired PCSM functionality, and analyze the resulting protocol’s security. To our knowledge, our work is the first that rigorously study Apple’s PSI for server-side content moderation within the broader context of secure messaging, addressing the diverse goals and security considerations of stakeholders when deploying larger systems.

Keywords: Secure messaging · Policies · Provable security

* Xiaohui Ding was supported by the EPSRC and the UK Government as part of the Centre for Doctoral Training in Cyber Security for the Everyday at Royal Holloway, University of London (EP/S021817/1).

Table of Contents

1	Introduction	2
1.1	Our Contributions	5
1.2	Technical Approach	8
1.3	Comparison with Prior Work	12
1.4	Further Discussion	13
1.5	Future Work	16
1.6	PCSM vs. Deniability	16
2	Preliminaries	17
2.1	Non-Interactive Secure Computation	17
2.2	Obfuscation	18
3	Policy Compliant Secure Messaging	19
3.1	PCSM Protocols	19
3.2	PCSM Security Definition	21
4	The POCO Protocol	24
4.1	Protocol Functionalities	24
4.2	Protocol Operations	25
4.3	Security (High Level)	28
5	Black-box Hash-Based PCSM	30
6	Strong Policy Privacy	33
A	Deferred Preliminaries	40
A.1	Committing Symmetric Encryption	40
A.2	Non-Interactive Zero-Knowledge	41
A.3	Digital Signatures	41
B	Deferred Proofs	42
B.1	Security Proof of POCO	42
B.2	Security Proof of H-POCO	52

1 Introduction

End-to-end encrypted (E2EE) secure messaging is a widely used class of cryptographic protocols, enabling clients to communicate asynchronously and securely over untrusted network and server infrastructure in long lasting sessions. At a technical level, the defining characteristic of E2EE is that the key material used to encrypt and authenticate messages is only known to the sender and receivers. This prevents any third parties, including telecommunication, cloud and internet providers from reading, modifying or injecting messages. E2EE is used by billions of users on a daily basis as it forms the cryptographic core of most popular messaging applications such as Signal, WhatsApp, Apple’s iMessage and Meta’s Messenger.

The strong privacy guarantees offered by E2EE systems have sparked a debate between law enforcement agencies on the one side, and technology firms and

privacy activists, on the other. The former views E2EE as interfering with their ability to maintain public safety and prosecute criminals, hampering investigative capabilities [20,45,53]. Specifically, law enforcement agencies claim that denying them access to messages while in transit leads to harmful and illegal content more rapidly spreading via E2EE communication systems, posing significant concerns about the public safety of highly vulnerable members of our societies, including sexually exploited children.

For very similar reasons, albeit less controversially, E2EE presents a serious obstacle to using some popular network defense mechanisms such as deep packet inspection by firewalls and network intrusion detection systems. In each case, the problem stems from the fact that E2EE prevents third parties (i.e., anyone not a sender or intended receiver) from reading messages, therefore content moderation via inspection while in transit seems to become impossible. Consequently, organisations lack some of the most effective tools for moderating the data traversing their networks and detecting and preventing intrusions and data exfiltration.

In July 2019, the governments of the United Kingdom, United States, Australia, New Zealand, and Canada issued a communique stating the need for lawful access to encrypted communications [40], while in October 2019, the Council of the EU adopted its conclusions [42]. The WePROTECT Global Alliance, NCMEC, and a coalition of more than 100 other child protection organisations and experts from around the world have all called for action to ensure that measures to increase privacy — including end-to-end encryption — should not come at the expense of children’s safety [2]. In May 2021 the Parliament of the United Kingdom published the “Online Safety Bill” [43], which is a legislation that requires platforms, including E2EE systems, to have mechanisms in place to scan for child sexual abuse material (CSAM), while in May 2022, the European Commission published the “Child Sexual Abuse” (CSA) proposal [21], which proposes regulations for increasing child safety online without offering any technical solutions that would not require either removing E2EE entirely or offer a weaker version of encryption. As a response, in September 2022, the United Nations Human Rights Chief presented the report “The right to privacy in the digital age” [44], defending the importance of end-to-end encryption in the protection of democracy and human rights worldwide.

The above discussion gives rise to two contrasting outcomes: a world embracing end-to-end encryption, devoid of any content moderation, or a world that forfeits E2EE but implements content moderation. As neither of these results appear satisfactory, we are motivated to explore the space in between, or, more fundamentally, to ascertain the existence of a middle ground. With this goal in mind, we ask the following (seemingly impossible) question:

Can we enable server-side content moderation (with security against malicious servers) in E2EE messaging without interfering with their E2EE properties?

Existing solutions & limitations. One of the most important lines of research for the results in this work build so-called traceback mechanisms for E2EE

systems [52,38,46,33]. These extend the notion of message franking [31,25,51]. Such schemes aim for the following objectives:

1. Conceal sent user messages, even from the server, and
2. guaranteeing anonymity of the sender of any harmless content.

The core component in such systems is a report mechanism that can be used by receivers to report harmful content *after* delivery. Once the server receives a report from a receiver, the server and receiver cooperate to identify the originator of the harmful message. Those systems do not provide any sort of proactive security since detection occurs only after delivery and requires help from the receiver (or self reporting by the sender as is effectively the case for simple deployment of Apple’s PSI based moderation). They are also insecure against malicious servers. Consequently, malicious receivers can report (and de-anonymize) senders, even if they sent harmless content. Moreover, harmful content that remains unreported by receivers cannot be detected using traceback mechanisms.

The recent work of Bartusek et al. [14] on traceback mechanisms improves over prior works by providing security against malicious servers. For this, they introduce the notion of *pre-constrained group signatures* which supports identifying users who sign pre-defined harmful content. In [14], the sender of any harmless content (according to a pre-defined policy) remains anonymous, even if the server receives a report from a receiver about that content. However, similar to prior work, in [14], detection occurs only after delivery and requires help from the receiver.

The above works make valuable contributions to content moderation, however, their results still suffer from significant limitations when considered in the context of policy-compliant messaging. Namely:

1. **Need client cooperation:** These results target abuse prevention, and detection relies on receivers reporting harmful content. Consequently, their design is unsuitable for applications such as preventing data exfiltration attacks, willful distribution of harmful content, malware spreading and intrusions.
2. **Lack separation of various stakeholders:** These systems do not separate the roles of the *auditor* auditing the incoming ciphertexts and the *judge* authorized to inspect the extracted harmful content in plaintext. In many cases, the former auditor role is more appropriate for the messaging service provider. In contrast, the latter role of the judge may be better suited to, say, law enforcement or the security department of a larger organization using the messaging system. Lack of this auditor-judge separation can be problematic for some classes of content, such as CSAM or confidential and classified material, where ideally, we would not like the service provider to ever receive such content in plaintext form. More generally, this precludes separating the goal of ensuring policies are enforced, from orthogonal goals of obtaining verifiable evidence of violations and of validating that no false positive occurred via human inspection (a common practical concern with most automated policies).

3. **Interfere with E2EE security:** Prior works, typically, do not focus on E2EE secure messaging protocols (such as Signal) and preserving the enhanced security they provide (forward-security, post-compromise security, and security in case of use of bad-randomness). We stress here that works that do consider E2EE secure messaging (cf. [14]) rely on client-side reporting.

Another important line of works builds private set intersection [15] and private membership computation protocols [36], as a mechanism for detecting and reporting CSAM content, while [48] presents techniques for publicly verifiable private hash matching, enabling services to match user content against a hash set of known CSAM without revealing the hash set to users or non-matching content to the service, while additionally enabling users to publicly verify that lawful content is not within the hash set. While these works make valuable contributions to content moderation, they leave open key questions about defining secure messaging in the presence of policies, addressing the diverse goals and security considerations of stakeholders in large-scale deployments, and integrating these building blocks with E2EE messaging. Naïvely composing a secure messaging protocol with a private set intersection (PSI) protocol—such as running them in parallel—can compromise post-compromise security and forward secrecy. Conversely, embedding a PSI protocol within an E2EE messaging framework would prevent server-side tracking.

1.1 Our Contributions

In this work, we introduce the notion of *policy-compliant secure messaging* (PCSM), which ensures end-to-end privacy and authenticity for harmless content (i.e., do not interfere with the security of the underlying E2EE protocol) while guaranteeing detection, interception and verifiable reporting of harmful content prior to its delivery to the receiver. PCSM allows for arbitrary policies and policy classes. Thus, our solution offers a novel tradeoff in the content moderation of E2EE communication debate — users sending benign content get the same E2EE security as they are used to, and harmful content is flagged without cooperation from the receiver.

Our Model. We consider the simplest case of secure messaging involving a single *receiver* R and *sender* S whose goal it is to send messages securely to R . In our system, we define three additional entities — a *policy creator* (PC), an *auditor* A , and a *judge* J . The *policy creator* (PC) is the entity that specifies a concrete policy, denoted by $\mathbf{P} \in \mathcal{P}$, taken from some publicly known set of permissible policies, known as a *policy class* $\mathcal{P}$. A message m sent by S should remain private to anyone besides R as long as $\mathbf{P}(m) = 1$. However, if $\mathbf{P}(m) = 0$, the *auditor* A should learn that m is non-compliant (presumably) halting delivery to R .⁵

⁵ In practice, the auditor could be implemented directly by the messaging system’s service provider as part of the delivery system, or by a third party which then signals to service provider whether or not to continue with delivery.

Furthermore A obtains a report for m which it can forward to the *judge* J which can verify that $\mathbf{P}(m) = 0$ and that S did indeed attempt to send m to R .⁶

We formalize security (and functionality) for PCSM via a UC ideal functionality which formally captures a host of intuitive security guarantees pertaining to different corruption scenarios. That is, we allow a static adversary to corrupt any subset of parties in $\{PC, A, S\}$ at the start of execution and constrain what it can do in each setting.

PCSM security for harmless content. A crucial design requirement for our system is that as long as only harmless messages⁷ are sent, our definition collapses to standard security for uni-directional two-party secure messaging.

- **Message Privacy for Policy Compliant Content:** If the sender S is honest,⁸ messages that satisfy the policy $\mathbf{P}$ chosen by the policy creator remain private against $\{PC, A, J\}$. In this case, our framework allows for preserving the security properties provided by the underlying E2EE protocol.

PCSM security for policy determined harmful content. On the other hand, the parties $\{PC, A, J\}$ are relevant when policy non-compliant content is circulated, and we have additional security requirements based on who among these participants is corrupted.

- **Policy Validity:** The policy is guaranteed to belong to a public policy class. I.e. $\mathbf{P} \in \mathcal{P}$, regardless of who is corrupted.
- **Policy Privacy:** We ask for very strong policy privacy. We require that nothing about the policy $\mathbf{P} \in \mathcal{P}$ is revealed to the adversary (corrupting anyone besides the PC) beyond (i) what is leaked by $\mathcal{P}$ itself and (ii) what can be learned by testing how $\mathbf{P}$ rejects/accepts certain messages. Additionally, we also consider the notion of (*weak*) *privacy policy* that offers the same security as above but does not allow for the corruption of A (in addition to the PC).
- **Message Privacy for Policy Non-compliant Content:** If the honest sender S sends a message not satisfying the policy $\mathbf{P}$, the non-compliant message itself (besides the fact that it is non-compliant) remains hidden from any adversary, assuming J is honest.

⁶ Of course, a malicious auditor could still refuse to deliver messages to parties even when $\mathbf{P}(m) = 1$. However, this amounts to a trivial denial-of-service (DoS) attack inherent to all asynchronous messaging systems that use some type of untrusted intermediary relay. Importantly, our system does not introduce any additional DoS vulnerabilities beyond those already present in a typical (non-policy compliant) E2EE messaging system.

⁷ To avoid confusion, we refer to protocol messages as “(protocol) packets” and reserve the term “message” to exclusively denote actual content sent by S .

⁸ Here and subsequently, “honest” only denotes following the honest protocol but says nothing about what content is being sent. Honest senders may still try to send harmful content, as instructed by the environment in our experiment.

- **Detectability:** Packets sent by (a possibly corrupt) S and audited by an honest A are either rejected by A (e.g. because they are mal-formed) or *detected* as containing harmful content if delivery to R would have resulted in R outputting policy non-compliant content.⁹
- **Verifiable Reporting:** Furthermore, in the case of the detection of a non-compliant message (as above), A gets access to a report that is not leaking anything about the message and it can be shared with the judge J . Given this report, J can recover the sent non-compliant message, along with irrefutable evidence that S indeed sent that message.
- **Unframeability:** If S is honest and never sent a message m , then the judge J never accepts a report (from a possibly corrupt auditor) claiming that S sent m . In particular, this holds for harmful messages for which $\mathbf{P}(m) = 0$.

Since PCSM aims to protect receivers from receiving harmful content, we only consider honest R . We refer to Sec. 1.4 for a more detailed discussion motivating our corruption model, the model for policies, policy classes and the notions of policy privacy and validity.¹⁰

PCSM for arbitrary policy classes. We present the following results:

- A PCSM protocol that achieves *weak policy privacy* with respect to *arbitrary policy classes*, based on standard assumptions (cf. Sec. 4, Theorem 1).
- A PCSM protocol that achieves *strong policy privacy* with respect to *arbitrary policy classes*, based on virtual black box obfuscation (VBB) [13,12] and other standard assumptions (cf. Sec. 6, Theorem 3).
- As a corollary, given that VBB for lists of point functions exists [39], we derive a PCSM protocol that achieves *strong policy privacy* with respect to the class of *lists of point functions*,¹¹ based on standard assumptions (cf. Sec. 6, Corollary 2).

PCSM for hash-based policy classes. For hash-based policies, every policy is described via a list of hash values, and any message with a hash contained in the list of a policy violates that policy. We present the following results:

- We leverage Apple’s private set intersection (PSI) system into the context of secure messaging, and we generically construct in a black-box way PCSM that achieves *weak policy privacy* for *hash-based policies* and *honest-but-curious* senders (rather than malicious), based on Apple’s *private set intersection with associated data* (PSI-AD) [15] primitive, and other standard assumptions (cf. Sec. 5, Theorem 2).

⁹ An honest A would typically block the delivery of such harmful content to R .

¹⁰ As in most UC works, our intuitive security properties serve to build intuition for the formal UC definition, and in our security proofs we prove indistinguishability between the real and ideal worlds, i.e., we are not proving individual properties as in game based definitions.

¹¹ Such a policy defines a list L of points and on input x the policy outputs 0 if $x \in L$.

- As a corollary, we derive a (non-black box) PCSM protocol for *hash-based policies* and *malicious senders*, under standard assumptions (cf. Sec. 4, Corollary 1).

To our knowledge, our work is the first that studies Apple’s PSI for server-side content moderation within the broader context secure messaging, modeling the different goals and security concerns of stakeholders when deploying larger systems.

1.2 Technical Approach

In the current section we present the high level ideas behind our constructions and we discuss the challenges in achieving the security properties of PCSM, listed in Sec. 1.1.

The symmetric layer. We build PCSM generically via a composition with a symmetric layer. Our PCSM protocols make use of a generic two-party asynchronous ratcheting protocol like Signal’s Double Ratchet [50] (analyzed formally by [19,3,18,16,34]) as a black-box building block. Such protocols enable users to initialize a common private state and ratchet it forward each time a message is sent or received. Upon each such ratchet a new symmetric private key is derived and used to encrypt/decrypt the next message sent/received, providing so-called *post-compromise forward security* (PCFS).¹² In our work we show how to generically build PCSM via composition with E2EE protocols, *without hurting* their underlying PCFS privacy and authenticity properties.¹³

Policy compliance via a modular composition. Given the symmetric layer, the starting point behind the policy compliance layer of our first protocol, called POCO, is a non-interactive secure computation scheme $\text{NISC} = (\text{Init}, \text{Enc}, \text{Dec})$ [32,1,10], which enables two parties, say A, B , with private inputs x, y , respectively, to non-interactively compute a two-input function $f(x, y)$, ensuring that B learns only $f(x, y)$, while A learns nothing about y . At a high level, a NISC execution evolves as follows. Initially, B computes $(\text{pub}, \text{priv}) \leftarrow \text{NISC.Init}(y)$ to generate public and private states, pub, priv , respectively, where pub is later used by A to “encrypt” their own inputs x by executing $c \leftarrow \text{NISC.Enc}(\text{pub}, x)$, and priv is used by B to recover $f(x, y)$, by executing the decryption $\text{NISC.Dec}(\text{priv}, c)$. Note that, with a single execution of NISC.Init , the primitive allows polynomially many encryption/decryption computations.

¹² Intuitively, PCFS means that post-compromise security and forward secrecy hold *simultaneously*. This is strictly stronger than the two properties holding each on its own. In other words, PCFS means messages sent in a given epoch remain secure (private and authentic) despite client state’s leaking in *both* past and future epochs.

¹³ Our work focuses on the “policy-compliance” aspect and does not explicitly model PCFS. We leave the task of formally unifying the two notions under a single definition for future work.

Our first construction utilizes a NISC scheme executed between the sender S and PC, A ; PC executes the initialization with a policy of its choice, S encrypts its input and sends it to A , and A evaluates the policy in a privacy preserving way via the decryption operation of NISC. For starters, assume that the sender S is honest but curious, and NISC_f securely computes the following function:

$$f(m, \mathbf{P}) := \text{If } m \text{ satisfies policy } \mathbf{P}, \text{ return "good", otherwise, return } m.$$

Given a NISC scheme, NISC_f , for f , we extend the sender's "Send" operation beyond just ratcheting and symmetric encryption, and we consider a composition between a symmetric encryption scheme SE and NISC_f as follows. Each time PC wishes to set a new policy, executes the initialization process of NISC, i.e., executes $(\text{pub}, \text{priv}) \leftarrow \text{NISC.Init}(\mathbf{P})$, for a policy $\mathbf{P} \in \mathcal{P}$ of its choice, publishes pub for the sender S and sends the NISC private state to the auditor, A .¹⁴ Subsequently, each time the sender S wishes to send a message m to the receiver, encrypts m symmetrically with respect to SE , computing a ciphertext c and then encrypts its message via NISC by computing $c' \leftarrow \text{NISC.Enc}(\text{pub}, m)$, and sends (c, c') to the auditor. The auditor evaluates the policy $\mathbf{P}$ over m by executing decryption, i.e., by executing $\text{NISC}_f.\text{Dec}(\text{priv}, c)$. By the definition of f , the decryption evaluates the policy over m , and reveals m only if it's not policy compliant. If the decryption outputs "good", A sends c to the receiver, otherwise, sends m to the judge J .

Observe that, by the security of NISC_f , nothing about the policy $\mathbf{P}$ is revealed to the adversary beyond (i) what is leaked by $\mathcal{P}$ itself and (ii) whether messages sent by S satisfy $\mathbf{P}$ or not, thus the scheme achieves *weak policy privacy*. Furthermore, the decryption of c' only leaks $f(m, \mathbf{P})$, and the above protocol achieves *detectability* against honest but curious senders. However, the protocol does not satisfy *unframeability* and *verifiable reporting*, since it provides no evidence that m was indeed sent by S , and fails to provide *message privacy* since non-policy compliant messages are revealed to the auditor (recall that according to message privacy, only the judge should access non-policy compliant messages).

We extend the above protocol in a way that enables the computation of "evidence" that harmful content was indeed sent by S and that information should only be accessible by the judge J , as follows: the sender computes a digital signature σ over m and then encrypts (m, σ) under the public key of the judge, pk_J , with respect to an IND-CCA secure public key encryption scheme, i.e., computes $e_J \leftarrow \text{PKE.Enc}(\text{pk}_J, (m, \sigma))$ and provides e_J as private input to the NISC computation. In particular, we leverage a NISC scheme for the computation of the following function:

$$f'_{\text{pk}_J}((m, e_J), \mathbf{P}) := \text{If } m \text{ satisfies policy } \mathbf{P}, \text{ return "good", otherwise, return an encryption, } e_J, \text{ of } (m, \sigma) \text{ under the public key of the judge, } \text{pk}_J.$$

¹⁴ Our first protocol doesn't provide privacy against the auditor, therefore the NISC initialization operation, can be executed either by the auditor or the policy creator, as both have access to the policy. The roles of PC and A become distinct later, when we present a protocol that achieves the stronger notion of policy privacy.

Given a NISC scheme for f'_{pk_J} , the sender encrypts (m, σ) under the public key of the judge, computing e_J , and finally encrypts (m, e_J) under the encryption operation of NISC. Thus, assuming honest but curious senders, the NISC decryption executed by the auditor will produce either the keyword “good” if $P(m) = 1$, or an encryption of (m, σ) under the judge’s public key. In particular, the encryption of (m, σ) ensures that only the judge (not the auditor or any other party) obtains evidence about the harmful content, and using σ , the judge can verify this fact for themselves without relying on a (possibly corrupt) auditor that might try to frame the sender, ensuring the *verifiable reporting* property of PCSM. Furthermore, if S is honest and never sent a message m , then the judge J never accepts a report from (a possibly corrupt auditor) claiming that S sent m , ensuring *unframeability*.

Additional challenges. Observe that the above design can break in various ways when parties behave dishonestly:

1. A malicious policy creator could generate $\text{pub} \leftarrow \text{NISC.Init}$ that leaks the message m outright, regardless even for policy-compliant messages, thus breaking message-privacy.
2. A malicious sender could encrypt a harmful message symmetrically (i.e., w.r.t. SE) but a different benign message with NISC. The policy check by the auditor would pass but the receiver would still decrypt to a harmful message breaking *detectability*. Furthermore, it easily breaks verifiable reporting by encrypting an invalid signature under the public key of the judge.
3. A sender could compute the symmetric encryption ciphertext using a key k' , such that decryption w.r.t. k' outputs a policy compliant message, while decryption with the honest receiver’s key k outputs a harmful message. Here too, *detectability* is violated.

We address 1 by augmenting the computation of $\text{NISC.Init}(\mathbf{P})$ with a zero-knowledge proof created by PC and validated by the sender, that pub has been generated correctly, with input policy $\mathbf{P}$ such that $\mathbf{P} \in \mathcal{P}$, enforcing *policy validity*. We address issue 2, by augmenting the “**Send**” operation with a zero-knowledge proof created by the sender and validated by the auditor that the symmetric and NISC ciphertexts encrypt the same message, m , and that e_J is a valid PKE ciphertext encrypting m and a valid signature over it. To address 3 we use a committing symmetric encryption scheme; namely one for which it is infeasible to produce a ciphertext c and distinct pair of keys k and k' such that decrypting c using k and k' both succeed yet produce distinct plaintexts. Notably, our solution addresses issue 3 without compromising the forward secrecy and post-compromise privacy and authenticity of the symmetric layer.

We formally present our first protocol, called POCO, capturing arbitrary policy classes, in Sec. 4, and we refer to the theorem statement (cf. Theorem 1) and proof (cf. App. B.1) for further details about the exact security properties needed for each of the above components.

PCSM for hash-based policy classes via Apple’s PSI. We construct PCSM for hash-based policies, i.e., policies defined via a list of hash values of harmful messages,

using Apple’s *private set intersection with associated data* (PSI-AD) [15] primitive, as our main ingredient. PSI-AD was introduced by [15] as a variant of PSI with data transfer [23], and was proven UC-secure in [15]. At a high level, content moderation via PSI-AD works as follows. The server holds a set X of fingerprints of harmful content, and the client stores a set D of fingerprints of their data. For each element in D , the execution of PSI-AD requires the client to define what is called the “associated data”, and assuming the client follows the protocol, the security of the primitive ensures that the server learns the associated data of the elements that belong to the intersection, $X \cap D$, but nothing about the elements in D whose fingerprints are not in the intersection. The protocol *does not* guarantee detection of harmful content if the client deviates from the protocol specification.

In our “black-box” result, we show how to generically construct PCSM for hash-based policies using the PSI-AD UC functionality of [15], $\mathcal{F}_{\text{PSI-AD}}$, in a black-box way, deriving the protocol H-POCO, achieving detectability and verifiable reporting only against honest-but-curious senders, thus highlighting the limitations of the original protocol of [15]. At a high level, if the sender is allowed to deviate from the protocol, it can: (1) encrypt harmful content w.r.t. the symmetric encryption scheme and harmless content w.r.t. the PSI-AD encryption, leading the PSI-AD execution to compute an empty intersection, thus bypassing detection, and (2) use arbitrary, inconsistent, associated data, which is the data that will be sent to the judge, thus breaking verifiable reporting and unframeability. We analyze the security of H-POCO in the $\mathcal{F}_{\text{PSI-AD}}$ -hybrid world, where parties make direct calls to an ideal PSI functionality $\mathcal{F}_{\text{PSI-AD}}$. Besides the PSI-AD component, H-POCO also relies on existentially unforgeable signatures, IND-CCA-secure symmetric encryption and IND-CCA-secure public-key encryption.

Finally, the high level idea behind our non-black box result, is as follows. PSI-AD considers server and client initialization operations, as well as client encryption and server decryption, which are referred to as voucher computation and voucher processing, respectively, in [15]. Furthermore, the original version of the protocol considers a threshold version of PSI-AD in which the associated data is revealed to the server only if the number of elements in the intersection matches that threshold. By considering a modified version of PSI-AD that does not account for thresholds, i.e., one in which the associated data is revealed to the server if the data (or the sent message in our context) is in X , we derive an efficient NISC scheme for the PSI-AD functionality, such that, by instantiating POCO with PSI-AD, we derive (as a corollary) a PCSM protocol for hash-based policies, satisfying the exact same level of security as POCO.

Strong policy privacy. We modify protocol POCO to derive the protocol O-POCO that achieves the stronger notion of policy privacy against the auditor (plus all other PCSM properties). At a high level O-POCO works as follows. PC, instead of directly computing the initialization operation of NISC (as it does in POCO), it obfuscates the policy $\mathbf{P}$ using VBB obfuscation and computes a zero knowledge proof proving that it follows the computation and that $\mathbf{P}$ is in $\mathcal{P}$. Subsequently,

the auditor fetches the obfuscated policy, and if the verification of the NIZK proof succeeds, it computes the initialization operation of NISC, with input the obfuscated policy, as well as a NIZK proof of correctness of the computation. The protocol should prevent the auditor from initializing NISC using an arbitrary policy, and we achieve this by efficiently binding together the obfuscated policy (computed by the policy creator) with the NISC state (computed by the auditor). As a corollary, we derive PCSM with strong policy privacy for the class of lists of point functions, under standard assumptions.

The above exposition omits several details and we refer the reader to Sec. 4, App. B.1, Sec. 5, App. B.2, and Sec. 6, for formal presentations and security proofs of POCO, H-POCO, and O-POCO, respectively. Furthermore, in Sec. 1.4 we present an extensive discussion on the importance of policies and the security properties related to them, as well as deniability.

1.3 Comparison with Prior Work

The work of Green et al. [30] defines the notion of abuse-resistant law enforcement access system (ARLEAS), aiming to mitigate the possibility of unauthorized surveillance. The follow up work of Fetzer et al. [28], provides a modular approach on auditable surveillance, by presenting a UC-functionality which can be used to augment applications with auditable surveillance capabilities, together with a threshold based construction that relies on the blockchain and YOSO multi-party computation.

An important line of research builds so-called “traceback” mechanisms for E2EE systems [52,38,46,33], extending message franking [31,25,51]. Such systems aim for the following two objectives. First, they conceal messages (exchanged by users), even from the server. Second, they guaranty anonymity of the sender of any harmless content. The main drawbacks in those systems, already identified by [29], are the following: (1) a receiver can collude with the server to de-anonymize the originator of any (even harmless) message, and (2) existing solutions (besides [46]) require the help, and thus de-anonymize, users on the traceback path in order to identify the originator of the message.

All aforementioned works, are insecure against malicious servers, and this is addressed by the recent work of Bartusek et al. [14], that introduce the notion of pre-constrained group signatures, which enable tracing of users in messaging systems who send pre-defined illegal content, while providing security against malicious service providers. Note that our security definition is orthogonal to the one of Bartusek et al. [14], which is mainly focusing on hiding the identity (i.e., the public key) of the sender, until a successful report operation takes place. Since our definition requires the (additional) property of *detectability* (which is not covered by [14]), in our setting the auditor knows the identity that sent harmful content, therefore, we aim at protecting the content and the proof that such content was sent by that sender.¹⁵ In this setting, our *verifiable reporting* implies

¹⁵ Note that our model considers a single sender and we are not aiming at hiding its identity from the auditor. In the proposed construction the auditor knows the public-key of the sender.

a type of *client-client* and *client-server anonymity* (of [14]), in the sense that only the judge receives access to the harmful content including verifiable evidence of its provenance (i.e., w.r.t. the sender’s signature public key). Our *unframeability* and *policy privacy*, are along the lines of *unframeability* and *set-hiding*, properties of [14], respectively, *detectability* and *message privacy* are not covered by [14] (since it is report-based and is only focusing on group signatures). Finally, the security definition of [14] is not focusing on *policy-validity*.

Another related line of work on group signatures is the one with message-dependent opening (GS-MDO) [27,37], in which trust is split between the group manager and an additional entity called the “admitter”.

In a different related work [15] the PSI-AD protocols used in our second construction is introduced as a mechanism for detecting and reporting CSAM user media content prior to it being uploaded to the Apple cloud. The work of Kohlweiss et al. [35] introduces the notion of privacy-preserving blueprints, which enables tracing the identity of entities that perform illegal actions in systems that use anonymous credentials. The notion is extended by [22]. The work of Scheffle and Mayer [49] presents a survey on content moderation for E2EE systems, while [48] presents techniques for publicly verifiable private hash matching. Finally, the works of Persiano et al. [47] and Banfi et al. [11], study the notion of anamorphic encryption, introduced by the former, which enables private communication in the presence of an untrusted government (referred as a dictator), that can receive access to the private keys of users. The work of [41] introduces private hierarchical governance for encrypted group messaging, combining robust end-to-end encryption (E2EE) with strong privacy, integrity, and unframeability for both content and governance. The design ensures that even a malicious platform cannot identify moderators or secretly tamper with their actions.

We leave it as an open problem to extend this notion to the group setting [4,5,9], or to achieve additional properties as explored in [26].

1.4 Further Discussion

Prior works have made an important contribution in the area of content moderation from a practical point of view, however from a theoretical perspective lack a comprehensive security model, that would account for the different and often conflicting interests of the various stakeholders involved, and would capture the security guarantees that should be provided by such systems. One of the novelties of our work is that it fills this gap, by putting forth a realistic and comprehensive model for content moderation, aiming to provide the strongest possibly security guarantees for the involved stakeholders, as well as, present feasibility results for the newly proposed notion.

On the role of PC. The policy creator, PC, is the entity that plays the role of a national organization, such as NCMEC, that specifies the policy, $\mathbf{P}$, which the definition requires to belong to a predefined policy class, $\mathcal{P}$, and needs to be satisfied by the sent messages in order for them to get delivered to users. Our

definition intentionally separates the two entities (i.e., the auditor and the policy creator) as this type of modeling captures more accurately the real-world setting.

On honest judge. We model the judge as honest, reflecting its expected trustworthiness in real-world scenarios, without which PCSM would lack credibility. However, the constructions that we present in our paper ensure unframeability even if the judge is corrupted (as long as the sender is honest), since neither the auditor nor the judge can produce valid reports for a policy-compliant message. The constructions remain unchanged, with minor adjustments to security proofs so that they will only simulate (fake) non-policy-compliant messages only when the judge is honest.

On denial-of-service attacks by the auditor. Note that a malicious auditor could potentially refuse to deliver messages to parties, however, this is already a trivial denial-of-service (DoS) attack in messaging systems and is unrelated to the PCSM component, as real-world messaging systems, the server can simply refuse to deliver messages. The auditor in our system can be implemented by the messaging system’s delivery server or any other server. Importantly, our system does not introduce additional vulnerabilities to DoS attacks beyond those that any comparable system might face.

On honest receivers. Observe that, if a corrupt sender and receiver deviate and collude they might agree on an added layer of encryption around content, making PCSM hopeless. Moreover, for our constructions our goal was to consider schemes which augment existing popular 2-party E2EE protocols rather than replace them. But the most popular such protocol, Signal’s double-ratchet, has the property that corrupting R allows an adversary to trivially inject forged messages from S to R . Thus, the most basic goal of PCSM – that harmful content not be delivered – would already be trivially violated using no further corruption of policy privacy and validity.

We motivate our model capturing policies and the related security notions by mapping real world examples of content moderation to our model.

On policy privacy. Recall, that one intuitive security property we formalize is that of policy privacy; i.e. the property that when A and PC are honest, or only PC is honest (following the stronger notion of policy privacy), nothing about the policy $\mathbf{P} \in \mathcal{P}$ is revealed to the adversary beyond (i) what is leaked by the policy class $\mathcal{P}$ itself and (ii) whether messages sent by S satisfy $\mathbf{P}$ or not.

Policy privacy can prove useful when balancing the desire for privacy of sensitive content covered by policies vs. the need of senders to limit what type of policies they are willing to let others enforce. Consider a system to combat (unintentional) leaking of an organization’s sensitive documents via their secure messaging system (say, because an employee is the target of a social engineering attack). We must reconcile two, seemingly conflicting, interests. The organization which is trying to prevent the dissemination of its most sensitive documents and the users who expect privacy for all other data being communicated. To

accommodate the later we can use a policy class consisting only of policies where “bad” content must be watermarked (or even digitally signed) by the organization. However, revealing the particular sensitive documents being scanned for (via the policy) goes against the goal of preventing dissemination of the documents. Thus, policy privacy helps to avoid the spread of the documents as a side effect of users interacting with the messaging system. Indeed, while NCMEC works with a variety of partners to deploy their hash-based policy, the source CSAM documents used to create the policy are far more guarded.

Policy privacy can also prevent senders from evaluating the policy locally; for example, in an attempt at manipulating non-compliant content until it is no longer flagged as such (our notion prevents this). We note that a similar notion to policy hiding, called set hiding for hash-based policies, was considered by [14].

On policy validity. Recall that policy validity is the intuitive property that only a *single* policy can be enforced for an honest sender and it must be chosen from the policy class. In our schemes the sender verifies that only one policy can possibly be enforced on its traffic (despite, say, auditor, judge and policy creator are colluding), and also that this policy must be in the public policy class. Concretely, returning to the watermarked documents example above, the policy that defines *all* messages to be “bad” cannot be enforced by a corrupt and colluding auditor if this policy was not contained in the policy class. In particular, corrupt policy creators, auditors and judges can not break a sender’s privacy according to some non-trivial combination of policies from the policy class.

We note that there many technical challenges behind defining illegal content via policies, since for instance, for perceptual hash based policies, there could be collisions between legal and illegal content and words could potential have dual meaning. However, this is out of the scope of the preset work.

Useful policy classes. Consider the policy class in which bad messages are simply those that are signed by an organization. Then there is no way for the auditor to produce a valid report unless they possess a valid signature over the transmitted message. Moreover, expressing such policies is straightforward, and using zero-knowledge proofs to verify policy validity is practically efficient.

Hash-based policies classes present another interesting practical example of classes. They are efficient, naturally provide decent policy privacy yet also impose meaningful limitations on policies. The fact that for any policy in such a class, the policy’s description (i.e. its hash-list), grows linearly in the number of “bad” documents makes it all but impossible to implement (or at least trivial to circumvent) key-word search policies. Instead, a hash-based PCSM scheme guarantees its users that any message with sufficient entropy will always be policy compliant regardless of what else it may contain. So while, known bad content will be detected, private conversations are still easy to have using the system.

An extensive discussion on PCSM and deniability as well as future work is deferred for Sec. 1.6 and Sec. 1.5 , respectively.

1.5 Future Work

This work only represents an first step towards practical PCSM. Besides the obvious open question of practical efficiency, a truly practical PCSM system would account for several extensions we have left out in this work for the sake of clarity. Notably, a PCSM system should allow for many senders and receivers, groups messaging and regular updates to policies that have a cost proportionate to the difference in old and new policy instead of requiring a full setup from scratch. We view our work as initiating this line of investigation rather claiming any sort of “completeness” of the system we describe.

Another very important avenue of research concerns the social questions posed by PCSM. What are practical policy classes and concrete policies and what social impact and pitfalls might enforcing them have? What is the right balance between what is publicly revealed about a policy via the policy class vs. what about the policy should be kept secret? How reliable can different policies be? Who should act as the judge and policy creator? How should the judge act when bad messages are reported; in particular, in light of the possibility for false positives?

The results in this work do not answer or even address any of these issues. Instead they add a new dimension to these debates as they demonstrate new technological possibilities.

1.6 PCSM vs. Deniability

Deniability is a spectrum of formal notions broadly enabling users to plausibly deny their participation in a protocol execution. In the context of secure messaging, deniability ensures that if party P_1 receives a message from P_2 , then although P_1 knows that the message was sent by P_2 it can’t prove this fact to a third party. From a technical perspective, deniability essentially requires that, messages exchanged between users can be simulated without relying on any user-specific secrets, and simulated messages should be indistinguishable from real ones. Concrete notions vary in strength depending on what information is needed by the simulator and what is available when attempting to distinguish its output from real transcripts. Such data includes, sender’s, receiver’s (and, if applicable, server’s) longer term, medium term and even ephemeral keys and coins. Deniability can be highly desirable in encrypted messaging systems, particularly in situations where there is a risk of coercion from external entities. For example, deniability has been a focus for end-to-end encrypted communication systems with traceback functionalities [46].

PCSM involves several parties besides sender’s, so deniability could take many forms. At least some of these seem to inherently conflict with the goals of PCSM [51]. For example, most deniability notions only afford the simulator public data which means that, in particular, a malicious auditor could run the simulator to produce fake transcripts for any sender in the system. Yet, if those transcripts look real to a third party with no more information than the judge, then the messaging system cannot also provide unframeability; namely the property that malicious auditors cannot falsely convince a judge that an honest

sender sent illegal content. Given the, potentially very serious ramifications a successful accusation can have for a sender, our design of PCSM forgoes this type of deniability in favour of unframeability.

Nevertheless, both of our constructions do achieve at least some weak form of deniability: intuitively, the auditor learns nothing about the underlying message nor do they see the signature (even for non-compliant content). Only the judge or sender ever see those values. So it would be relatively straightforward to produce fake transcripts between the parties to convince a third party which does *not* know the receiver’s nor judge’s private keys. We leave exploring this intuition in more detail and a wider study of the relationship between PCSM and deniability, for interesting future work.

2 Preliminaries

In the current section we introduce the notation and the cryptographic tools that will be used throughout the paper. Due to space limitations, basic preliminary notions are deferred for App. A.

Notation. Keyword **req**, followed by a condition, requires the condition to be satisfied, otherwise all steps up to the **req** statement are reversed, the operation returns $\perp$ to the adversary and the execution halts. Keyword **try**, followed by an operation, tries to execute operation and in case of failure the execution proceeds as in **req**. For a set X , $X \leftarrow x$, adds x to X , while $X \leftarrow x$, removes x from X . The advantage of an adversary $\mathcal{A}$ in breaking property X of primitive Y will be denoted by $\text{Adv}_Y^X(\mathcal{A})$, and “ $X \stackrel{c}{\approx} Y$ ” denotes that the random variables X , Y , are computationally indistinguishable.

2.1 Non-Interactive Secure Computation

In the current section we recall the notion of non-interactive secure computation (NISC) [32,1,10], between a sender, S , with input x , and a receiver, R , with input y . The receiver should learn $f(x, y)$ and nothing else, while the sender should gain no information about y .

Real World. A NISC protocol NISC between a sender S and a receiver R , is defined by three probabilistic algorithms (**Init**, **Enc**, **Dec**) as follows. The receiver runs the algorithm **Init** which takes the receiver’s input, $y \in \{0, 1\}^k$, as input and outputs $(\text{nisc}_1, \text{priv})$. The receiver then sends nisc_1 to the sender, who obtains nisc_2 by evaluating $\text{Enc}(\text{nisc}_1, x)$, where $x \in \{0, 1\}^k$, is the sender’s input. The sender then sends nisc_2 to the receiver who obtains $f(x, y)$ by evaluating $\text{Dec}(\text{priv}, \text{nisc}_2)$.

At the onset of the computation the real world adversary $\mathcal{A}$ corrupting either the sender S or the receiver R , receives some auxiliary information z . Next, the computation proceeds as described above where the honest party sends messages as prescribed by the protocol and the adversary $\mathcal{A}$ sends arbitrary messages on behalf on the corrupted party. At the end of the computation the

uncorrupted party outputs whatever is specified in the protocol. The corrupted party outputs any arbitrary probabilistic polynomial time (PPT) function of the view of $\mathcal{A}$. The overall output of the real-world experiment consists of all the values output by all parties at the end of the protocol, and this random variable is denoted by $\text{REAL}_{\mathcal{A}}^{\text{NISC}(\mathcal{S}, \mathcal{R})}(1^\lambda, x, y, z)$. Let $\text{REAL}_{\mathcal{A}}^{\text{NISC}(\mathcal{S}, \mathcal{R})}$ denote the ensemble $\left\{ \text{REAL}_{\mathcal{A}}^{\text{NISC}(\mathcal{S}, \mathcal{R})}(1^\lambda, x, y, z) \right\}_{(\lambda \in \mathbb{N}, x, y \in \{0,1\}^k, z \in \{0,1\}^*)}$.

Ideal World. In the ideal world experiment, the sender $\mathcal{S}$ and the receiver $\mathcal{R}$ interact with a trusted party for computing a function $f : \{0,1\}^k \times \{0,1\}^k \rightarrow \{0,1\}^k$. The ideal world computation in presence of the ideal world adversary $\mathcal{S}$ corrupting either the sender $\mathcal{S}$ or the receiver $\mathcal{R}$, and an (incorruptible) trusted party $\mathcal{F}$, proceeds as follows. First, as in the real-life model, $\mathcal{S}$ gets auxiliary information z . Next, the ideal world adversary $\mathcal{S}$ generates any arbitrary input on behalf of the corrupted party, which it sends to the trusted party $\mathcal{F}$. The honest party sends its input to the trusted party $\mathcal{F}$. At this point the ideal functionality evaluates the output and sends it to the receiver. The honest receiver outputs this value. The adversarial receiver $\mathcal{S}$ outputs an arbitrary value. The overall output of the ideal process consists of all the values that are output by all parties at the end of the protocol. Let this random variable be denoted by $\text{IDEAL}_{\mathcal{S}}^{\mathcal{F}}(1^\lambda, x, y, z)$ and let $\text{IDEAL}_{\mathcal{S}}^{\mathcal{F}}$ denote the ensemble $\left\{ \text{IDEAL}_{\mathcal{S}}^{\mathcal{F}}(1^\lambda, x, y, z) \right\}_{(\lambda \in \mathbb{N}, x, y \in \{0,1\}^k, z \in \{0,1\}^*)}$.

Definition 1. Let f be any polynomial time computable function on two inputs and let $\text{NISC}(\mathcal{S}, \mathcal{R})$ be a protocol between a sender $\mathcal{S}$ and a receiver $\mathcal{R}$. We say that $\text{NISC}(\mathcal{S}, \mathcal{R})$ securely evaluates f , if for every PPT real world adversary $\mathcal{A}$ there exists an ideal world adversary $\mathcal{S}$, such that $\text{REAL}_{\mathcal{A}}^{\text{NISC}(\mathcal{S}, \mathcal{R})} \stackrel{c}{\approx} \text{IDEAL}_{\mathcal{S}}^{\mathcal{F}}$.

2.2 Obfuscation

We now revisit the definition of virtual black box obfuscation.

Definition 2 ([13,12]). Let $\mathcal{C} = \{\mathcal{C}_n\}_{n \in \mathbb{N}}$, be a family of polynomial-size circuits, where $\mathcal{C}_n$ is a set of boolean circuits operating on inputs of length n . Let $\mathcal{O}$ be a PPTM algorithm, which takes as input an input length $n \in \mathbb{N}$, a circuit $C \in \mathcal{C}_n$, a security parameter $\lambda \in \mathbb{N}$, and outputs a boolean circuit $\mathcal{O}(C)$ (not necessarily in $\mathcal{C}$). $\mathcal{O}$ is a (black-box) obfuscator for the circuit family $\mathcal{C}$ if it satisfies:

- **Functionality preservation:** for every $n \in \mathbb{N}$, every $C \in \mathcal{C}_n$, and every $x \in \{0,1\}^n$, with all but negligible probability over the coins of $\mathcal{O}$:

$$(\mathcal{O}(C, 1^n, 1^\lambda))(x) = C(x).$$

- **Polynomial slowdown:** for every $n, \lambda \in \mathbb{N}$ and $C \in \mathcal{C}$, the circuit $\mathcal{O}(C, 1^n, 1^\lambda)$ is of size at most $\text{poly}(|C|, n, \lambda)$.
- **Virtual black-box:** for every (non-uniform) polynomial size adversary $\mathcal{A}$, there exists a (non-uniform) polynomial size simulator $\mathcal{S}$, such that for every $n \in \mathbb{N}$ and for every $C \in \mathcal{C}_n$:

$$\left| \Pr_{\mathcal{O}, \mathcal{A}}[\mathcal{A}(\mathcal{O}(C, 1^n, 1^\lambda)) = 1] - \Pr_{\mathcal{S}}[\mathcal{O}^{\mathcal{S}}(|C|, 1^n, 1^\lambda)] \right| = \text{negl}(\lambda).$$

3 Policy Compliant Secure Messaging

In the current section we define the notion of *policy-compliant secure messaging*, in the UC framework [17]. In Sec. 3.1, we introduce the high level ideas, the syntax, the adversarial model and the security properties, for PCSM protocols, while in Sec. 3.2, we present our security definition.

3.1 PCSM Protocols

Following the *secure messaging* regime, PCSM protocols enable two parties to communicate *privately* and *authentically*, with the additional goal of *detecting* harmful content that does not satisfy policies within certain policy classes.

The parties. A PCSM protocol execution considers the following parties. The sender, S , who wishes to send messages to the receiver, R , via the execution of a secure messaging (SM) protocol. The policy creator, PC , which is the entity that specifies the policy, P , that needs to be satisfied by the sent messages in order for them to get delivered to R . The auditor, A , which is the entity that plays the role of the server that audits the output of S , and depending on whether the underlying message satisfies P , it either delivers the message to R , or creates a report for the judge, J . Finally, the judge J is processing incoming reports from A and verifies their validity.

Single sender/receiver and unidirectional channels. Following the two-party secure messaging literature [50], we consider a *single sender* and a *single receiver*. In addition, we assume a *unidirectional communication channel* between them, i.e., as the notation suggests, S is only acting as a sender, while R is only acting as a receiver. The main reason behind this choice (besides simplicity) is that we follow a modular approach that enables the construction of PCSM as a composition between a two-party SM layer and a policy compliance layer, where in contrast to two-party SM, in which parties share a common state, the policy compliance layer only requires a common state to be shared between the sender and the auditor (the server), i.e., the state of the policy compliance layer is independent of the common state of S , R . Therefore, switching the roles between the sender and the receiver would simply require an additional, independent state, and presenting our results using unidirectional channels suffices. Our definition can be easily extended to support switching roles w.r.t. send and receive.

Syntax. A PCSM protocol, $PCSM$, consists of the algorithms (**Initialization**, **SetPolicy**, **Send**, **Audit**, **Deliver**, **Report**), such that:

- **Initialization**: the operation receives no input and initializes the state of the parties.
- **SetPolicy**: executed by PC ; it receives a policy P and returns an acknowledgement *ack* if the operation is successful, or $\perp$.
- **Send**: executed by S ; it receives a message m and returns a ciphertext c .

- **Audit**: executed by A; it receives a ciphertext c and returns, either a “proof” π for J that the underlying message is not policy compliant, or a ciphertext c for R, or $\perp$.
- **Deliver**: executed by R; it receives a ciphertext c and outputs a message m or $\perp$.
- **Report**: executed by J; it receives a proof π and returns a message m or $\perp$.

Adversarial model. We consider a static adversary who is allowed to corrupt parties in $\{S, PC, A\}$. The reason why we consider honest R is that PCSM aims at protecting honest receivers from receiving harmful content. If R gets corrupted then the adversary can trivially inject and deliver arbitrary messages against any two-party SM protocol that shares a symmetric state between S, R, and we intentionally exclude this case from our model. Regarding the judge J, its role is to verify “proofs” proving that (1) certain messages have been sent by S and (2) those messages are not policy compliant. “Good” messages do not get leaked to J (if S is honest) and we assume this party to be honest, as one would normally expect in reality.

For any corrupted party in $\{S, PC, A\}$, the adversary learns its state and is allowed to send messages on behalf of it. If S and A are honest, then we don’t consider injections against A, while if A is honest, then our definition does not allow injections against R, J. The reason why we opt-in for this type of adversary, is that in practice there is an authenticated channel between the sender, S, and the server (auditor), A, as well as between A and R, J, thus we only allow injections only if the corresponding parties are corrupted. Our definition can be easily extended to allow injections if the parties are honest, and our constructions would only require the additional hybrids whose indistinguishability reduces to authenticity related properties of the underlying primitives.

Security. At a high level PCSM provides the following security properties:

- **Message privacy**: if the sender, S, is honest, i.e., its state is not leaked to the adversary, sent messages that satisfy the policy, $\mathbf{P}$, set by PC, remain private.
- **Weak Policy Privacy**: If PC and A are honest, nothing about the policy $\mathbf{P} \in \mathcal{P}$ is revealed to the adversary beyond (i) what is leaked by $\mathcal{P}$ itself and (ii) whether messages sent by S satisfy $\mathbf{P}$ or not.
- **Policy Privacy**: A strengthening of the above property that does not require A to be honest.
- **Policy validity**: The policy is guaranteed to belong to a public policy class $\mathcal{P}$, known to all parties, even if PC, A, are corrupted.
- **Detectability**: if A is honest, harmful content is detectable and not delivered to R.
- **Unframeability**: if S is honest, no valid report that would be accepted by the judge J can be generated by (a possibly corrupt auditor) A claiming that S sent a non-policy compliant message (if the message is not sent by S).
- **Verifiable reporting**: If reports computed by A are shared with the judge J, the latter can recover the sent non-compliant message, along with irrefutable evidence that S indeed sent that message.

3.2 PCSM Security Definition

We define PCSM security in the UC framework, i.e., we consider a real-world experiment where an environment¹⁶ $\mathcal{A}$ interacts with a PCSM protocol Π , where such an execution is denoted by $\text{REAL}_\Pi(\mathcal{A})$, and an ideal-world experiment where $\mathcal{A}$ interacts with an ideal functionality $\mathcal{F}_{\text{PCSM}}$ and a simulator $\mathcal{S}$, denoted by $\text{IDEAL}_{\mathcal{F}_{\text{PCSM}}, \mathcal{S}}(\mathcal{A})$. In this setting, a protocol Π is secure if for all $\mathcal{A}$ there exists a simulator $\mathcal{S}$ such that the difference between the probability that $\mathcal{A}$ outputs 1 in $\text{REAL}_\Pi(\mathcal{A})$ and the probability that $\mathcal{A}$ outputs 1 in $\text{IDEAL}_{\mathcal{F}_{\text{PCSM}}, \mathcal{S}}(\mathcal{A})$, is negligible in the security parameter.

We now formally define our ideal functionality $\mathcal{F}_{\text{PCSM}}$ presented in Fig. 1. The functionality is parameterized by the policy class $\mathcal{P}$, the set of parties $\mathcal{U} = \{\text{S}, \text{R}, \text{PC}, \text{A}, \text{J}\}$, and the set of corrupted parties $\mathcal{C} \subseteq \mathcal{U} \setminus \{\text{R}, \text{J}\}$.

Below we present the operations supported by the ideal functionality.

Initialization. This operation initializes the variables that will be used throughout the execution. In more detail, M will store all the messages sent by the sender, GM (resp. BM), stores messages that satisfy (resp. do not satisfy) the current policy, which is denoted by $\mathbf{P}$, and RM stores messages that have been successfully reported. $\hat{\mathbf{P}}$, $\hat{m}$ and v , are helper variables that we explain below.

Set policy. This operation receives as input a policy $\mathbf{P}'$ and if either the policy creator, PC , or the auditor, A , is corrupted, then it sets $\hat{\mathbf{P}}$ to $\mathbf{P}$, i.e., it leaks the policy to the ideal world adversary. Note that by omitting the “red” part of the first “if” condition (“ $\forall \text{A} \in \mathcal{C}$ ”), the definition captures the stronger notion of policy privacy, that hides the policy even against the auditor. Subsequently, if PC is corrupted, it is requested to set the policy, i.e., $\mathbf{P}'$ is set by the environment as input to the operation, still the ideal-world adversary might choose another policy, otherwise it simply acknowledges the completion of the operation. In any case the new policy is set to $\mathbf{P}$ and is required to belong to the set of admissible policies, $\mathcal{P}$, otherwise, the execution returns $\perp$.

Send. The operation receives as input a message m with message id mid that is required to be fresh, and additionally requires the policy to be non-empty. If the sender S is corrupted, the operation sets $\hat{m}$ to m , and thus the execution leaks m to the ideal world adversary. Subsequently, the functionality evaluates the policy $\mathbf{P}$ over the message m (denoted by b'), and if S is not corrupted (which implies the it will always follow the prescribed protocol), the functionality reveals b' to the ideal world adversary, which in turn returns a ciphertext c , a message m' and v , where v indicates whether c is a valid ciphertext. Note that, revealing b' to the adversary at this early stage of the execution, and before it decides the ciphertext that it wants to send, does not create any security issues since for $\text{S} \notin \mathcal{C}$, S always follows the prescribed protocol, even if $b' = 0$. On the other hand, if $\text{S} \in \mathcal{C}$, b' is revealed to the simulator only after it defines its ciphertext, therefore the

¹⁶ Here, we simplify notation and we refer to both the environment $\mathcal{Z}$ and real world adversary $\mathcal{A}$ of UC, using the notation $\mathcal{A}$.

Functionality $\mathcal{F}_{\text{PCSM}}$

Parameters: policy class $\mathcal{P}$, parties $\mathcal{U}$, corrupted parties $\mathcal{C}$

```

// Executed on first input
Initialization
   $M, GM, BM, RM \leftarrow \emptyset$ 
   $P, \hat{P}, \hat{m} \leftarrow \varepsilon; v \leftarrow \text{true}$ 

// Set policy
Input (SetPolicy,  $P'$ ) from PC
  if  $PC \in \mathcal{C} \vee A \in \mathcal{C}$  then  $\hat{P} \leftarrow P'$ 
  Send (SetPolicy,  $\hat{P}$ ) to sim; receive  $P^*$ 
  if  $PC \in \mathcal{C}$  then
     $P \leftarrow P^*$ 
  else
     $P \leftarrow P'$ 
  req  $P \in \mathcal{P}$ 
   $\hat{P} \leftarrow \varepsilon$ 
  return ack

// Send message
Input (Send, mid,  $m$ ) from S
  req  $P \neq \varepsilon, (\cdot, \cdot, \cdot, \text{mid}) \notin M$ 
  if  $S \in \mathcal{C}$  then  $\hat{m} \leftarrow m$ 
   $b \leftarrow \varepsilon; b' \leftarrow P(m)$ 
  if  $S \notin \mathcal{C}$  then  $b \leftarrow b'$ 
  Send (Send,  $\hat{m}, b$ ) to sim; receive  $(c, m', v)$  from sim
  if  $m' \neq \perp$  then
     $m \leftarrow m'; b' \leftarrow P(m')$ ; send  $b'$  to sim
   $M \leftarrow (m, c, v, \text{mid})$ 
  return c

// Audit ciphertext
Input (Audit, mid,  $c$ ) from A
  req  $P \neq \varepsilon \wedge (\cdot, c, \cdot, \text{mid}) \in M$ 
  if  $A \notin \mathcal{C}$  then req  $(\cdot, c, \text{true}, \text{mid}) \in M$ 
  Send (Audit, mid,  $c$ ) to sim; receive  $\pi$ 
  if  $P(m) = 1$  then
     $GM \leftarrow (m, c, \text{mid}); v \leftarrow c$ 
  else
     $BM \leftarrow (m, \pi, \text{mid}); v \leftarrow \pi$ 
  return v

// Deliver message
Input (Deliver, mid,  $c$ ) from R
  if  $A \notin \mathcal{C}$  then
    if  $\exists m : (m, c, \text{mid}) \in GM$  then
       $GM \leftarrow (m, c, \text{mid})$ 
      return  $m$ 
    else
      return  $\perp$ 
  else
    Send (Deliver, mid,  $c$ ) to sim; receive  $m$ 
    return  $m$ 

// Report message
Input (Report, mid,  $\pi$ ) from J
  if  $A \in \mathcal{C}$  then
    Send (Report, mid,  $\pi$ ) to sim
    Receive  $\pi'; \pi \leftarrow \pi'$ 
    if  $S \in \mathcal{C}$  then return  $\pi$ 
  if  $\exists m : (m, \pi, \text{mid}) \in BM \wedge (m, \text{mid}) \notin RM$  then
     $RM \leftarrow (m, \text{mid})$ 
    return  $m$ 
  else
    return  $\perp$ 

```

Fig. 1: The ideal functionality for PCSM, parameterized by the policy class $\mathcal{P}$, the set of parties $\mathcal{U} = \{S, R, PC, A, J\}$, and the set of corrupted parties $\mathcal{C} \subseteq \mathcal{U} \setminus \{R, J\}$.

adversary cannot choose the sent message depending on whether it is valid or not. m' is also returned to capture the fact that a malicious S might not choose to send the input message m , i.e., it might diverge and pick its own message m' . In this setting the simulator is required to “extract” that message and return it to $\mathcal{F}_{\text{PCSM}}$ in order to execute the policy evaluation. Finally, $\mathcal{F}_{\text{PCSM}}$ stores the message, ciphertext, message id, and v , into M and returns the ciphertext c .

Audit. The **Audit** operation initially requires the policy to be already set and also that the input ciphertext to be the output of the **Send** operation. The second requirement comes from fact that our definition excludes injections if S is honest (cf. the “Adversarial model” paragraph in Sec. 3.1). Additionally, if the auditor is honest, it requires c to be a valid ciphertext, i.e., v should be equal true, since honest auditors would simply drop invalid ciphertexts. Subsequently, $\mathcal{F}_{\text{PCSM}}$ informs the simulator about the operation and if the ciphertext encrypts a message

that satisfies the policy, the functionality stores to the set of “good messages”, GM , the input ciphertext and the corresponding information, (m, c, mid) , while if the message does not satisfy the policy it stores a proof of non-compliance, π , provided by the simulator. The operation returns either the ciphertext, which during an honest execution would be delivered to the receiver, R , or a proof that the message is “bad”, that would be sent to the judge, J , via **Report**.

Deliver. If the auditor is corrupted, then it can simply forward to the receiver arbitrary ciphertexts, therefore, in this setting $\mathcal{F}_{\text{PCSM}}$ simply forwards the input c to the simulator, which decides the message that should be output by the **Deliver** operation. On the other hand, if A is honest, then $\mathcal{F}_{\text{PCSM}}$ returns a non-bottom value only if for the input ciphertext c there exists a “good message” m in GM , in which case it returns that message. This enforces the fact that R only receives messages that satisfy the policy.

Report. If the auditor is corrupted, then **Report** forwards the input to the operation, (mid, π) , to the simulator, to receive back a potentially different proof π' , which is used in the rest of the execution. If in addition $\text{S} \in \mathcal{C}$, this implies that a malicious auditor who knows the private state of S can create valid reports for arbitrary messages for which the policy has not even been evaluated, therefore in this trivial setting, $\mathcal{F}_{\text{PCSM}}$ simply returns the value set by the simulator, and there is no security guarantee for the sender. In any other scenario, $\mathcal{F}_{\text{PCSM}}$ returns the message m that relates to the proof π , only if the message fails to satisfy the policy and is the first time that is being reported (for the specific unique mid), in which case it has been already added to BM , and this guarantees the unframeability (anti-framing) property of our primitive, if $\text{S} \notin \mathcal{C}$, i.e., the state of S is not leaked to the adversary.¹⁷ Finally, a message cannot be reported multiple times and this is captured by the requirement $(m, \text{mid}) \notin \text{RM}$.

Security properties. It is not hard to see that $\mathcal{F}_{\text{PCSM}}$ guarantees the security properties presented in the previous section (cf. Sec. 3.1). With regards to *message privacy*, observe that S ’s input message m is leaked to the adversary only if $\text{S} \in \mathcal{C}$. Similarly, if A and PC are honest, or if only PC is honest (for the stronger notion of policy privacy), $\mathcal{F}_{\text{PCSM}}$ leaks to the adversary only the policy evaluation outcome over the sent messages, enforcing *policy privacy*, while $\mathcal{F}_{\text{PCSM}}$ checks whether $\mathbf{P}$ is in $\mathcal{P}$, enforcing *policy validity*. If A is honest, then the **Deliver** operation only delivers messages stored into GM (i.e., the messages m for which $\mathbf{P}(m) = 1$), enforcing *detectability*, and if S is honest, then **Report** only outputs messages sent by S that do not satisfy $\mathbf{P}$ (those messages are stored in BM), i.e., no report would output a “good” message, enforcing *unframeability*, and finally,

¹⁷ In our definition, the judge outputs only the non-policy compliant message, however, in our constructions, the judge learns not just the message but also a signature under the public key of the sender providing publicly verifiable evidence that sender sent illegal content. Therefore, our definition can be easily extended to provide public verifiability of the fact that the sender sent illegal content.

if A is honest, J always outputs (learns) the “bad” messages sent by S, and no other entity learns those messages.

Post-compromise/forward security. Since our work is focusing on the policy-compliance layer, our definition considers static adversaries, against which the notions of post-compromise and forward security are not relevant, since if a user gets corrupted, this will hold for the entire execution. However, we stress that our protocols, when instantiated with any forward and post-compromise secure protocol against adaptive adversaries, such as Signal [50], achieves the same level of post-compromise and forward security (cf. Sec. 4.3).

4 The POCO Protocol

In the current section we define the PCSM protocol POCO that supports arbitrary policy classes and achieves weaker policy privacy. As most secure messaging protocols, it relies on a PKI infrastructure, that we model using an Authenticated Key Service functionality [9,7]. Our protocol is based on the following primitives: (1) a signature scheme, Σ , (2) a symmetric encryption scheme, SE, (3) a non-interactive secure computation scheme, NISC, (4) a public-key encryption scheme, PKE, and (5) a non-interactive zero-knowledge scheme, N, and is depicted in Fig. 4. Our protocol relies on three functionalities, which we present in Sec. 4.1. The main protocol operations are presented in Sec. 4.2.

4.1 Protocol Functionalities

The authenticated key service (AKS) functionality $\mathcal{F}_{\text{AKS}}$. The AKS is a type of PKI that enables the distribution of the users’ signature keys as well as the distribution of the public-key encryption key of the judge J, without requiring any interaction between the parties. It is modeled via the functionality $\mathcal{F}_{\text{AKS}}$ (cf. Fig. 2), supporting an arbitrary number of users and keys.¹⁸

$\mathcal{F}_{\text{AKS}}$ is parameterized by the signature and public-key encryption key generation algorithms, $\Sigma.\text{KeyGen}$, PKE.KeyGen , respectively, and works as follows. **Initialization** initializes two empty arrays that will be used to store public keys for Σ and PKE. When a party with id id wishes to fetch a public key of another party with id id' , it sends to $\mathcal{F}_{\text{AKS}}$ a query of the form $(\text{GetPK}, \text{id}', T)$, where T should be in $\{\Sigma, \text{PKE}\}$. When a party wishes to generate a key-pair for a scheme $T \in \{\Sigma, \text{PKE}\}$ it simply sends a (Gen, T) query to $\mathcal{F}_{\text{AKS}}$, and the functionality generates and stores the corresponding public-key. Our protocol is defined in the $\mathcal{F}_{\text{AKS}}$ -hybrid model, i.e., its operations make direct calls to the operations provided by $\mathcal{F}_{\text{AKS}}$.

¹⁸ $\mathcal{F}_{\text{AKS}}$ is simply a PKI functionality for signature and encryption keys, which for simplicity we combine into one functionality. This exact version is novel to our work but it is inspired by the PKI of [6] and [8].

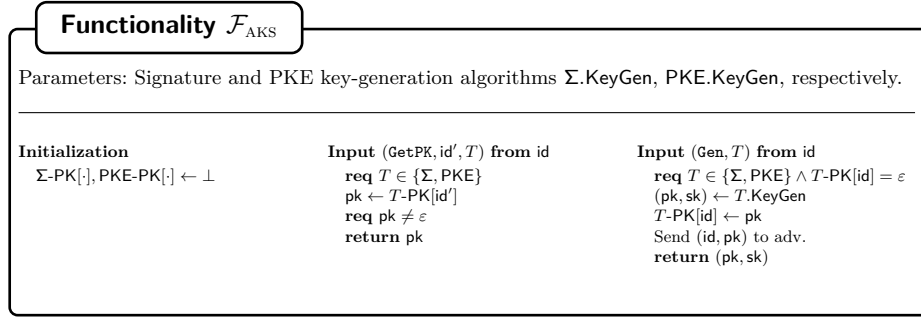


Fig. 2: The Authenticated Key Service functionality.

The two-party ratchet functionality $\mathcal{F}_{2\text{PR}}$. Our protocol builds on top of a symmetric encryption layer which we model as a two-party ratcheting execution that can be instantiated by any protocol such as Signal [50,3,16]. Signal enables users to initialize a common private state as well as ratchet the state forward each time a message is sent (or received). Whenever the state ratchets forward a new symmetric private key is derived and is used to encrypt the next message. For the sake of modularity (and for simplicity), we model those operations via the functionality $\mathcal{F}_{2\text{PR}}$, supporting the operations **Init** and **GetKey**. The **Init** operation receives no input and initializes the two-party shared state, while the **GetKey** operation with input index i returns the symmetric key that will be used by the symmetric encryption scheme, **SE**, to encrypt/decrypt the message with id mid, enabling the decryption of out-of-order messages (a property that is called immediate decryption [3]). The functionality simply enables the generation of independent symmetric keys with every **GetKey**.

The functionality $\mathcal{F}_{\text{CRS}}$ (cf. Fig. 3). This is a helper functionality that: (1) generates the common reference string (CRS) for the NIZK scheme and enables the parties to fetch it, (2) enables PC to non-interactively publish (via **SetPolicy**) the information that is required by the sender to send messages and by the auditor to audit messages, and enables the latter parties to non-interactively fetch that information via **GetPub** and **GetPolicy**, respectively.

4.2 Protocol Operations

Initialization & set policy. During initialization, the sender **S** samples a fresh signature key pair via a (Gen, Σ) query to $\mathcal{F}_{\text{AKS}}$, retrieves **J**'s public-key via the query $(\text{GetPK}, \text{J}, \text{PKE})$ to $\mathcal{F}_{\text{AKS}}$,¹⁹ initializes the two-party ratchet via an **Init** query to $\mathcal{F}_{2\text{PR}}$ and initializes the message id i to 0. The judge **J** samples a fresh

¹⁹ Here we assume that queries to $\mathcal{F}_{\text{AKS}}$ that fetch keys of other parties terminate only after successfully fetching those keys.

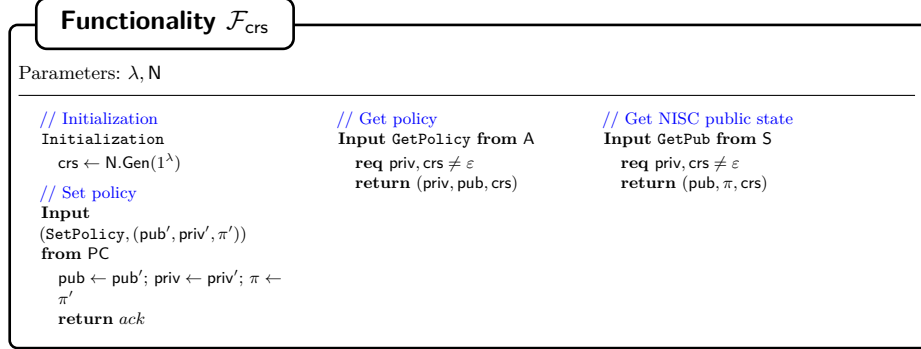


Fig. 3: The functionality that initializes the CRS, enables the policy creator to non-interactively communicate $(\text{pub}, \pi, \text{crs})$ to the sender, and $(\text{priv}, \text{pub}, \text{crs})$ to the auditor, parameterized by the security parameter λ and a NIZK scheme $N = (\text{Gen}, \text{Prv}, \text{Vrf}, \mathcal{S})$.

PKE key-pair via a (Gen, PKE) query to $\mathcal{F}_{\text{AKS}}$, fetches the sender's Σ public key and initializes the set of reported messages RM . Finally, the auditor retrieves spk_S via a $(\text{GetPK}, S, \Sigma)$ query to $\mathcal{F}_{\text{AKS}}$, and initializes the set that will store the set of observed message indices, M .

On **SetPolicy** with input $\mathbf{P}$, the policy creator executes the initialization of NISC with respect to the circuit $\mathcal{C}^{\mathcal{P}}$ and input $\mathbf{P}$ and creates a NIZK proof for $\mathcal{W}$, proving correctness of the computation and that $\mathbf{P} \in \mathcal{P}$. Here, $\mathcal{C}^{\mathcal{P}}$, denotes a universal circuit over the class of admissible possibly classes $\mathcal{P}$, that, on input a policy $\mathbf{P}$ and (m, x) , if $\mathbf{P}(m) = 1$, $\mathcal{C}^{\mathcal{P}}(m, x)$ outputs GOOD, otherwise outputs x (we define x below).

Send message. On **Send** with input m , the sender fetches the NISC public state pub , the NIZK proof π and the CRS, verifies π , increases the message index i , signs (m, i) , computes the next private key using the two-party double ratchet functionality $\mathcal{F}_{2\text{PR}}$, encrypts (m, σ) under PKE with pk_J , encrypts m under SE, computes NISC.Enc over (m, e_J) , and computes a zero-knowledge proof for $\mathcal{R}$, which informally states that S knows the SE private key, k , message m , signature σ over it, and both ciphertexts encrypt the same message m , and also that NISC.Enc encrypts m , as well as e_J .

Audit ciphertext. On **Audit** with input e , the auditor recovers the CRS and NISC related state, verifies the zero-knowledge proof and the uniqueness of the index i , executes the NISC decryption using priv , and if the decryption indicates that the ciphertext encrypts a “GOOD” message, the operation returns the input ciphertext, otherwise it returns the decryption output, which by the definition of $\mathcal{C}^{\mathcal{P}}$ is a public key encryption ciphertext under the judge's public key of the underlying message-signature pair, sent by S .

POCO

Parameters: $\lambda, \mathcal{P}, \mathcal{C}^{\mathcal{P}}, \mathcal{U}, \Sigma, \text{SE}, \text{NISC}, \text{PKE}, \text{N}$

```

// Executed on first input
Initialization
if id = S then
  try (spkS, sskS) ← query (Gen, Σ) to  $\mathcal{F}_{\text{AKS}}$ 
  try pkJ ← query (GetPK, J, PKE) to  $\mathcal{F}_{\text{AKS}}$ 
  try query (Init) to  $\mathcal{F}_{2\text{PR}}$ 
  i ← 0
if id = J then
  try (pkJ, skJ) ← query (Gen, PKE) to  $\mathcal{F}_{\text{AKS}}$ 
  try spkS ← query (GetPK, S, Σ) to  $\mathcal{F}_{\text{AKS}}$ 
  RM ← ∅
if id = A then
  try spkS ← query (GetPK, S, Σ) to  $\mathcal{F}_{\text{AKS}}$ 
  M ← ∅

// Set policy
Input (SetPolicy, P) for PC
r ← {0, 1}λ
(pub, priv) ← NISCCP.Init(1λ, P; r)
π ← NW.Prv(crs, pub, (r, P, priv))
try query (SetPolicy, (pub, priv, π)) to  $\mathcal{F}_{\text{crs}}$ 
return ack

// Send message
Input (Send, m) for S
try (pub, π, crs) ← query (GetPub) to  $\mathcal{F}_{\text{crs}}$ 
req NW.Vrf(crs, pub, π)
r, r' ← {0, 1}λ; i ← 1
σ ← Σ.Sign(sskS, (m, i))
try k ← query (GetKey, i) to  $\mathcal{F}_{2\text{PR}}$ 
c ← SE.Enc(k, m)
eJ ← PKE.Enc(pkJ, (m, σ); r')
c' ← NISC.Enc(pub, (m, eJ); r)
π ← NR.Prv(crs, (c, c', spkS, pub, pkJ, i), (m, k, σ, r, r', eJ))
return (c, c', π, i)

// Audit ciphertext
Input (Audit, e) for A
req spkS ≠ ε
try (priv, pub, crs) ← query (GetPolicy) to  $\mathcal{F}_{\text{crs}}$ 
(c, c', π, i) ← e
req i ∉ M ∧ NR.Vrf(crs, (c, c', spkS, pub, pkJ, i), π)
d ← NISC.Dec(priv, c'); M ← i
if d = GOOD then
  return e
else
  return d

// Deliver ciphertext
Input (Deliver, e) for R
try (c, c', π, i) ← e
try k ← query (GetKey, i) to  $\mathcal{F}_{2\text{PR}}$ 
m ← SE.Dec(k, c)
return m

// Report message
Input (Report, (eJ, i)) for J
req spkS ≠ ε ∧ i ∉ RM
(m, σ) ← PKE.Dec(skJ, eJ)
req Σ.Ver(spkS, (m, i), σ)
RM ← i
return m

// Policy evaluation helper
Helper  $\mathcal{C}^{\mathcal{P}}((m, e_J), \mathbf{P})$ 
if P(m) then
  return GOOD
else
  return e_J

// Witness relation for Send
 $\mathcal{R} := \{(c, c', \text{spk}_S, \text{pub}, \text{pk}_J, i) : \exists w = (m, k, \sigma, r, r', e_J) \text{ s.t.}$ 
 $\Sigma.\text{Ver}(\text{spk}_S, (m, i), \sigma) = 1, e_J \leftarrow \text{PKE}.\text{Enc}(\text{pk}_J, (m, \sigma); r'),$ 
 $c' = \text{NISC}.\text{Enc}(\text{pub}, (m, e_J); r), c = \text{SE}.\text{Enc}(k, m)\}$ 

// Witness relation for the SetPolicy
 $\mathcal{W} := \{\text{pub} : \exists w = (r, \mathbf{P}, \text{priv}) \text{ s.t. } (\text{pub}, \text{priv}) \leftarrow \text{NISC}.\text{Init}(1^\lambda, \mathbf{P}; r), \mathbf{P} \in \mathcal{P}\}$ 

```

Fig. 4: The protocol that realizes the ideal functionality of Fig. 1, parameterized by the security parameter λ , the policy class $\mathcal{P}$, the universal circuit $\mathcal{C}^{\mathcal{P}}$ over $\mathcal{P}$, the set of parties $\mathcal{U} = \{S, R, PC, A, J\}$, an EU-CMA signature scheme $\Sigma = (\text{KeyGen}, \text{Sign}, \text{Ver})$, a symmetric, committing encryption scheme $\text{SE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$, a NISC scheme $\text{NISC} = (\text{Init}, \text{Enc}, \text{Dec})$ for $\mathcal{C}^{\mathcal{P}}$, a public key encryption scheme $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$, and a NIZK scheme $\text{N} = (\text{Gen}, \text{Prv}, \text{Vrf}, \mathcal{S})$. The protocol is in the $(\mathcal{F}_{\text{AKS}}, \mathcal{F}_{\text{crs}}, \mathcal{F}_{2\text{PR}})$ -hybrid world.

Deliver & report. On **Deliver** with input e , the receiver first queries $\mathcal{F}_{2\text{PR}}$ for the SE private key for the message with id i , decrypts the ciphertext and returns the plaintext. On **report** with input (e_J, i) , the judge first checks if the input index i is fresh, and then decrypts the input ciphertext to recover (m, σ) , verifies the

signature over (m, i) and returns m on successful verification, otherwise returns $\perp$.

4.3 Security (High Level)

In the current section we provide the high level intuition behind the security of POCO.

Message privacy. This property is relevant only if S is honest. In this case, observe that S encrypts the input message m under the symmetric encryption scheme, SE , computing the ciphertext c , and also encrypts m together with the encryption of m and a signature, σ , over it, under the public key of the judge, under $NISC$, to compute c' . Then, it computes a zero-knowledge proof, π , proving that the two ciphertexts encrypt the same message m and the judge's ciphertext is correctly computed (the ZK proof is required for the case in which $S \in \mathcal{C}$, see below). Assuming the $NISC$ security of $NISC$, the CCA security of SE and the zero-knowledge property of N , the output of the encryption operation, $e := (c, c', \pi, i)$, hides the underlying message m . Note that the adversary could try to tamper with c' to create a related message, however, it would then be required to create a NIZK proof proving that the same new message is also encrypted under the SE and that it knows a valid signature of that message. Clearly, this cannot happen since SE is authenticated, and the adversary does not have access to the sender's SE key and signature key. It remains to argue about message privacy after e is audited by A . If A is malicious, the fact that the $NISC$ state $priv$ is consistent with the input policy $\mathbf{P}$ and $\mathbf{P}$ is admissible, i.e., $\mathbf{P} \in \mathcal{P}$ (this is enforced by the zero-knowledge proof computed by PC), enforces the fact that the output of the audit operation computes no more information than what is leaked by the $NISC$ decryption, i.e., the computation only leaks $\mathbf{P}(m)$, and if the message is not policy compliant it outputs e_J . However, there is still a remaining issue to be addressed: a malicious auditor can possibly distinguish the fake c' (that encrypts an independent message m') from the original one (that encrypts m), since it could be the case that $\mathbf{P}(m) \neq \mathbf{P}(m')$. Here we use the property that for each policy class $\mathcal{P}$ and policy $\mathbf{P}$, a “good” and a “bad” message are publicly known, and the fake c' should encrypt m' s.t. $\mathbf{P}(m') = \mathbf{P}(m)$, where $\mathbf{P}(m)$ is leaked by the ideal functionality. In this way, the output of the audit operation leaks only whether m is policy compliant.

Finally, we additionally fake the PKE ciphertext output by A assuming the CCA security of PKE. After faking the ciphertexts, ensuring that the outputs of **Delivery** and **Report** via $\mathcal{F}_{PCSM}$ are consistent with those in the real-world requires additional properties (such as CCA security) and we refer the reader to the full proof (cf. App. B.1) for more details.

Policy privacy & validity. With regards to policy privacy, observe that, if A and PC , are honest, the policy $\mathbf{P}$ remains private against a corrupted sender since by the $NISC$ security pub doesn't leak any information about $\mathbf{P}$, and the same holds for π assuming the zero-knowledge property of NIZK. The only information that

is leaked is the policy evaluation outcome over the sent messages, since this is the only information leaked by $\text{NISC.Dec}(\text{priv}, c')$. Policy validity is enforced by the fact that PC is required to prove that the input policy $\mathbf{P}$ is in $\mathcal{P}$, when executing NISC.Enc .

Detectability. Detectability is relevant only if A is honest. Observe that, in that case, if S is malicious, there are two ways in which it could possibly deviate from the protocol and violate detectability. The first would be to encrypt different messages w.r.t. NISC and SE, so that the functional encryption evaluation over c' would be w.r.t. a policy compliant message m , while the symmetric encryption SE ciphertext c encrypts an arbitrary, possibly bad, message (this would result in R receiving that message). We mitigate this attack by requiring S to provide a zero-knowledge proof that both ciphertexts are w.r.t. the same message m .

Unframeability & verifiable reporting. Observe that, if S is honest, in which case its signing key is private, assuming the unforgeability of Σ and the security of NISC, the only way to compute a valid signature over a message m , is to evaluate NISC over a c' that encrypts a bad message, therefore no valid report that would be accepted by J can be generated by A for any policy-compliant message sent by S, and the protocol satisfies the unframeability property. Furthermore, if A is honest, the judge J learns the “bad” messages sent by the sender by decrypting the PKE ciphertext e_J to recover (m, σ) and verifies their validity by verifying the signature σ over (m, i) , and the protocol satisfies verifiable reporting. Note, that since the S is signing m together with the unique index i , and J keeps a record of reported messages, a malicious auditor cannot report the same message multiple times. Furthermore, since the uniqueness of the index i is checked by the A, a malicious S cannot reuse the same index, to avoid reporting and achieve delivery of non-policy compliant messages.

Post-compromise/forward security. Observe that, besides the symmetric state, the only additional private information that is leaked to the adversary if a sender or receiver gets corrupted, is the secret key of the signature scheme, Σ , therefore, by periodically updating the keys of Σ , our protocol can achieve the same level of post-compromise and forward security as the underlying symmetric encryption scheme. Also, the **SetPolicy** operation provides healing with respect to corrupted A and PC.

Our result is formally summarized in the below theorem.

Theorem 1. *Let $\mathcal{F}_{\text{PCSM}}$ be the functionality of Fig. 1 w.r.t. weak policy privacy, Σ be an EU-CMA-secure signature scheme, SE be a CCA-secure committing encryption scheme, NISC be a non-interactive secure computation scheme for $\mathcal{C}^{\mathcal{P}}$, PKE be an IND-CCA-secure PKE scheme, and N be a simulation sound extractable zero-knowledge scheme, instantiating the protocol POCO of Fig. 4 in the $(\mathcal{F}_{\text{AKS}}, \mathcal{F}_{\text{CRS}}, \mathcal{F}_{\text{2PR}})$ -hybrid world. Then, for any $\mathcal{A}$, there exists an $\mathcal{S}_{\text{POCO}}$ and*

adversaries $\mathcal{B}_1, \dots, \mathcal{B}_7$, s.t.

$$\begin{aligned} & \Pr[\text{IDEAL}_{\mathcal{F}_{\text{PCSM}}, \mathcal{S}_{\text{POCO}}}(\mathcal{A}) \Rightarrow 1] - \Pr[\text{REAL}_{\text{POCO}, \mathcal{F}_{\text{AKS}}, \mathcal{F}_{\text{crs}}, \mathcal{F}_{2\text{PR}}}(\mathcal{A}) \Rightarrow 1] \leq \\ & + \text{Adv}_{\text{N}}^{\text{ZK}}(\mathcal{B}_1) + \text{Adv}_{\text{N}}^{\text{SSE}}(\mathcal{B}_2) + \text{Adv}_{\Sigma}^{\text{EU-CMA}}(\mathcal{B}_3) + \text{Adv}_{\text{SE}}^{\text{COM}}(\mathcal{B}_4) \\ & + \text{Adv}_{\text{NISC}}^{\text{NISC}}(\mathcal{B}_5) + q \cdot \left(\text{Adv}_{\text{PKE}}^{\text{IND-CCA}}(\mathcal{B}_6) + \text{Adv}_{\text{SE}}^{\text{CCA}}(\mathcal{B}_7) \right) \\ & + \varepsilon_{\text{se-cor}} + \varepsilon_{\text{nisc-cor}} + \varepsilon_{\text{pke-cor}} + \varepsilon_{\Sigma\text{-cor}} + \varepsilon_{\text{N-com}}, \end{aligned}$$

where q denotes the number of queries made by $\mathcal{A}$ and $\varepsilon_{\text{se-cor}}$, $\varepsilon_{\text{nisc-cor}}$, $\varepsilon_{\text{pke-cor}}$, $\varepsilon_{\Sigma\text{-cor}}$, $\varepsilon_{\text{N-com}}$, denote upper bounds on the probability of violating the correctness (or completeness) of SE, NISC, PKE, Σ and N, respectively.

Non black-box hash-based PCSM. The protocol of [15] considers server and client initialization operations, as well as client encryption and server decryption, while an additional operation checks if the number of harmful messages has reached a specific threshold, and only then reveals evidence about the harmful content. By removing the threshold requirement, we derive a simplified version of the PSI-AD protocol of [15], which is an instantiation of NISC for hash-based policies. Therefore we derive the following corollary.

Corollary 1. *Let $\mathcal{F}_{\text{PCSM}}$ be the functionality of Fig. 1 w.r.t. weak policy privacy, Σ be an EU-CMA-secure signature scheme, SE be a CCA-secure committing encryption scheme, NISC := PSI-AD be the private set intersection with associated data scheme of [15], PKE be an IND-CCA-secure PKE scheme, and N be a simulation sound extractable zero-knowledge scheme, instantiating the protocol POCO of Fig. 4 in the $(\mathcal{F}_{\text{AKS}}, \mathcal{F}_{\text{crs}}, \mathcal{F}_{2\text{PR}})$ -hybrid world. Then, for any $\mathcal{A}$, there exists an $\mathcal{S}_{\text{POCO}}$ s.t. $\Pr[\text{IDEAL}_{\mathcal{F}_{\text{PCSM}}, \mathcal{S}_{\text{POCO}}}(\mathcal{A}) \Rightarrow 1] - \Pr[\text{REAL}_{\text{POCO}, \mathcal{F}_{\text{AKS}}, \mathcal{F}_{\text{crs}}, \mathcal{F}_{2\text{PR}}}(\mathcal{A}) \Rightarrow 1]$ is negligible in the security parameter, where, the exact bound matches that of Theorem 1.*

5 Black-box Hash-Based PCSM

In the current section we present a black-box construction of PCSM for hash-based policies, based on *private set intersection with associated data* (PSI-AD), which is proven to be UC secure [15]. In fact, our construction relies on a simplified version of Apple’s PSI-AD that supports the **InSet** operation (instead of the more general set intersection operation). Our protocol is presented in the $\mathcal{F}_{\text{PSI-AD}}$ -hybrid world, where parties make direct calls to $\mathcal{F}_{\text{PSI-AD}}$, presented in Fig. 5. Similar to our first construction, H-POCO makes also direct calls to $\mathcal{F}_{2\text{PR}}$ and $\mathcal{F}_{\text{AKS}}$.

The functionality $\mathcal{F}_{\text{PSI-AD}}$. The functionality is parameterized by the parties S, A, PC, and a hash function $\text{Hash} : \mathcal{X} \rightarrow \mathcal{Y}$. **Initialization** simply initializes variables X , V , **SetDB** sets the database to a new value X' after checking whether it is a hash list in $\mathcal{Y}$, **SetHash** with input (y, a, id) , stores the hash value y , with associated data a and id id , so that the functionality at a later point can

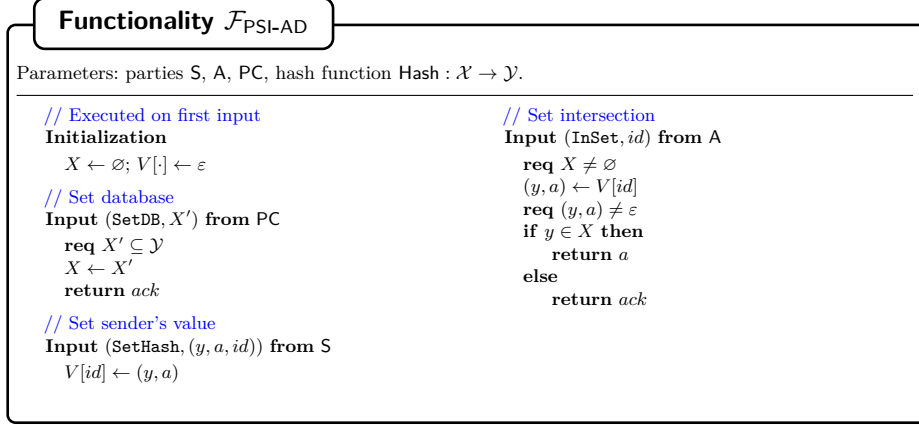


Fig. 5: The functionality for PSI-AD.

check via the **InSet** operation whether a value with id id belongs to X , in which case the functionality returns the associated data a , otherwise it returns an acknowledgement.

The high level idea is that, if the message sent by the sender contains harmful content (in which case its hash value belongs to the database X), then the functionality returns the associated data which we define to be the encryption of the harmful content and a signature over it, over the public key of the judge. On the other hand, if the sent message is policy compliant, the functionality returns an acknowledgement.

Protocol operations. Our protocol is depicted in figure 6. **Initialization** initializes the PKE, Σ and SE key material, as in POCO, with the difference being that in H-POCO there is no need for NISC related key material, and NIZKs. The **SetPolicy** operation with input $\mathbf{P}$, makes a **SetBD** query to $\mathcal{F}_{\text{PSI-AD}}$, requiring that $\mathbf{P}$ is a list of hash values in the range of Hash . **Send** with input m , signs (m, i) and computes its hash y , computes the SE and PKE encryptions of m , c , e_J , respectively, queries $\mathcal{F}_{\text{PSI-AD}}$ with $(\text{SetHash}, (y, e_J, i))$ (i.e., sets e_J as the associated data in $\mathcal{F}_{\text{PSI-AD}}$) and returns (c, i) . **Audit** on input (c, i) queries $\mathcal{F}_{\text{PSI-AD}}$ with i , to check whether the hash value of the underlying message in c is in the database, and if this is the case it returns the associated data, which is the PKE ciphertext for J , otherwise it returns the input ciphertext e for delivery. **Deliver** and **Report** are straightforward.

Security. *Message privacy* (for the case in which S is honest), holds due to the security of SE and PKE and the fact that $\mathcal{F}_{\text{PSI-AD}}$ leaks only whether m satisfies the policy $\mathbf{P}$. *Policy privacy* is straightforward as $\mathbf{P}$ is only known to PC and $\mathcal{F}_{\text{PSI-AD}}$, where the latter checks whether X is a valid hash list, while for

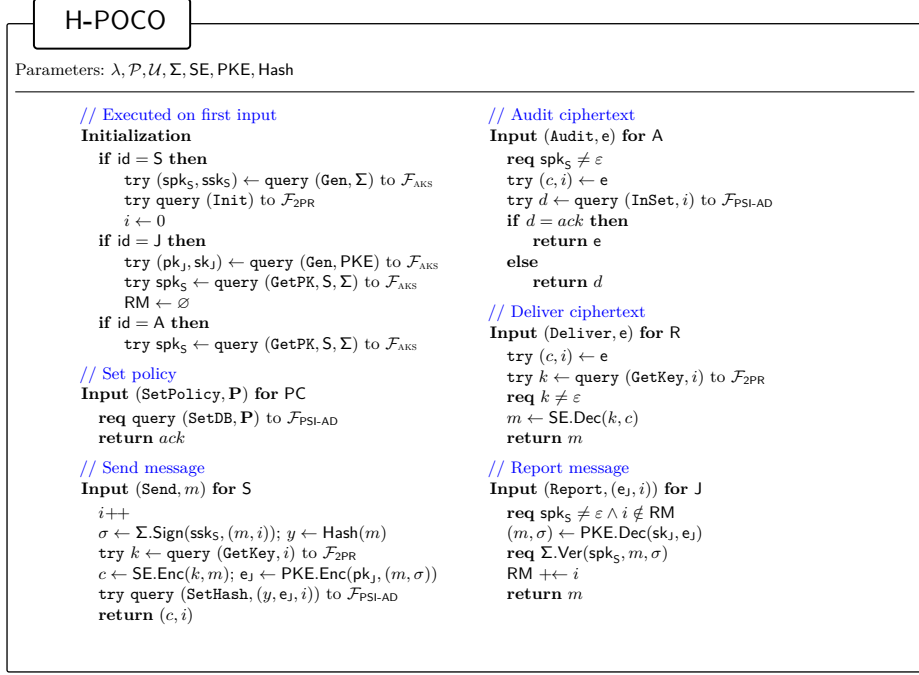


Fig. 6: The protocol that realizes the ideal functionality of Fig. 1 in the $\mathcal{F}_{\text{PSI-AD}}, \mathcal{F}_{\text{AKS}}, \mathcal{F}_{2\text{PR}}$ -hybrid world, parameterized by the security parameter λ , the policy class $\mathcal{P}$, the set of parties $\mathcal{U} = \{S, R, PC, A, J\}$ an EU-CMA signature scheme $\Sigma = (\text{KeyGen}, \text{Sign}, \text{Ver})$, a symmetric encryption scheme $\text{SE} = (\text{Init}, \text{Enc}, \text{Dec})$, a public key encryption scheme $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$, and a hash function Hash.

unframeability we observe that the only way for **A** to submit a valid report to **J** is to retrieve an encryption of a valid (m, σ) pair, which only happens if the hash of the message is in the database of bad messages (assuming unforgeability of signatures). Finally, *detectability* and *verifiable reporting* hold against honest but curious **S**: if **S** follows the protocol then no message with message id i will reach the receiver if the hash value of it is in X (if **A** is honest), and for any message that does not satisfy **P**, the auditor will retrieve the associated data e_J , which is a valid encryption of (m, σ) under **J**'s public key, and this enables the judge to verify that a bad message was sent by **S**.

Our result is formally presented in the following theorem.

Theorem 2. *Let $\mathcal{F}_{\text{PCSM}}$ be the functionality of Fig. 1 w.r.t. weak policy privacy, Σ be an EU-CMA secure signature scheme, SE be a CCA-secure encryption scheme, PKE be a CCA-secure PKE scheme, instantiating the protocol H-POCO of Fig. 6 in the $\mathcal{F}_{\text{AKS}}, \mathcal{F}_{2\text{PR}}, \mathcal{F}_{\text{PSI-AD}}$ -hybrid world. Then, for any **A**, there exists*

an $\mathcal{S}$ and adversaries $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3$, s.t.

$$\begin{aligned} \Pr[\text{IDEAL}_{\mathcal{F}_{\text{PCSM}}, \mathcal{S}}(\mathcal{A}) \Rightarrow 1] - \Pr[\text{REAL}_{\text{H-POCO}, (\mathcal{F}_{\text{AKS}}, \mathcal{F}_{\text{2PR}}, \mathcal{F}_{\text{PSI-AD}})}(\mathcal{A}) \Rightarrow 1] \leq \\ q \cdot \left(\text{Adv}_{\text{PKE}}^{\text{CCA}}(\mathcal{B}_1) + \text{Adv}_{\text{SE}}^{\text{CCA}}(\mathcal{B}_2) \right) + \text{Adv}_{\Sigma}^{\text{EU-CMA}}(\mathcal{B}_3) \\ + \varepsilon_{\text{se-cor}} + \varepsilon_{\text{pke-cor}} + \varepsilon_{\Sigma\text{-cor}}, \end{aligned}$$

where q denotes the number of queries made by $\mathcal{A}$ and $\varepsilon_{\text{se-cor}}, \varepsilon_{\text{pke-cor}}, \varepsilon_{\Sigma\text{-cor}}$, denote upper bounds on the probability of violating the correctness of SE, PKE and Σ , respectively.

Due to space limitations the proof of the above theorem is deferred for App. B.2.

6 Strong Policy Privacy

In the current section we show how to modify protocol POCO in order to achieve policy privacy against the auditor (and all other parties besides PC). Our protocol, O-POCO, is depicted in Fig. 8 and makes direct calls to $\mathcal{F}'_{\text{crs}}$, presented in Fig. 7. Operations that don't appear in Fig. 8 are identical to the those of POCO, presented in Fig. 4, while the code in gray again matches that of POCO.

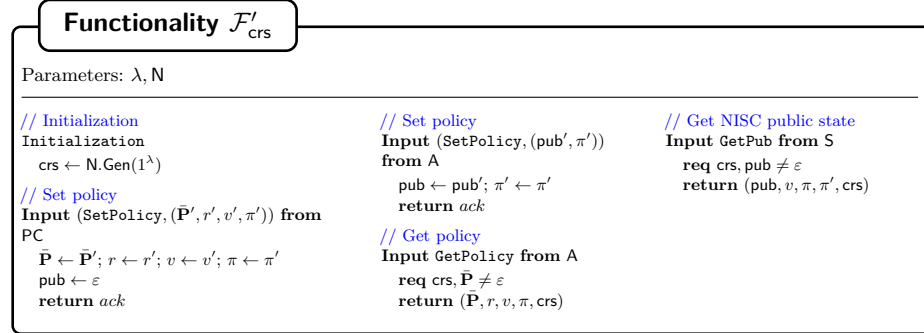


Fig. 7: The functionality that initializes the CRS, enables the policy creator and the auditor to non-interactively communicate $(\text{pub}, v, \pi, \pi', \text{crs})$ to the sender, and $(\bar{\mathbf{P}}, r, v, \pi, \text{crs})$ to the auditor, parameterized by the security parameter λ and a NIZK scheme $\mathbf{N} = (\text{Gen}, \text{Prv}, \text{Vrf}, \mathcal{S})$.

The protocol. At a high level O-POCO works as follows. PC first obfuscates the policy $\mathbf{P}$ using VBB obfuscation (cf. Sec. 2.2) in order to derive $\bar{\mathbf{P}}$, while in addition computes the hash of $\bar{\mathbf{P}}$ and a zero knowledge proof w.r.t. the witness relation $\mathcal{W}$, proving that it follows the computation and that $\mathbf{P}$ is in $\mathcal{P}$. The

O-POCO

Parameters: $\lambda, \mathcal{P}, \mathcal{C}^{\mathcal{P}}, \mathcal{U}, \Sigma, \text{SE}, \text{NISC}, \text{PKE}, \text{N}, \text{Hash}$

```

// Set policy
Input (SetPolicy, P) for PC
 $r, r' \leftarrow \{0, 1\}^\lambda$ 
 $\bar{\mathbf{P}} \leftarrow \mathcal{O}(\mathbf{P}, 1^n, 1^\lambda; r')$ 
 $v \leftarrow \text{Hash}(\bar{\mathbf{P}}; r)$ 
 $\pi \leftarrow \text{N}_{\mathcal{W}}.\text{Prv}(\text{crs}, v, (r, r', \mathbf{P}, \bar{\mathbf{P}}))$ 
try query (SetPolicy,  $(\bar{\mathbf{P}}, r, v, \pi)$ ) to  $\mathcal{F}_{\text{crs}}$ 
return ack

// Send message
Input (Send, m) for S
try  $(\text{pub}, v, \pi, \pi', \text{crs}) \leftarrow \text{query}(\text{GetPub})$  to  $\mathcal{F}_{\text{crs}}$ 
req  $\text{N}_{\mathcal{W}}.\text{Vrf}(\text{crs}, v, \pi)$ 
req  $\text{N}_{\mathcal{W}}.\text{Vrf}(\text{crs}, (\text{pub}, v), \pi')$ 
 $r, r' \leftarrow \{0, 1\}^\lambda; i \leftarrow$ 
 $\sigma \leftarrow \Sigma.\text{Sign}(\text{ssk}_S, (m, i))$ 
try  $k \leftarrow \text{query}(\text{GetKey}, i)$  to  $\mathcal{F}_{2\text{PR}}$ 
 $c \leftarrow \text{SE}.\text{Enc}(k, m)$ 
 $e_j \leftarrow \text{PKE}.\text{Enc}(\text{pk}_J, (m, \sigma); r')$ 
 $c' \leftarrow \text{NISC}.\text{Enc}(\text{pub}, (m, e_j); r)$ 
 $\pi \leftarrow \text{N}_{\mathcal{R}}.\text{Prv}(\text{crs}, (c, c', \text{spk}_S, \text{pub}, \text{pk}_J, i), (m, k, \sigma, r, r', e_j))$ 
return  $(c, c', \pi, i)$ 

// Audit ciphertext
Input (Audit, e) for A
req  $\text{spk}_S \neq \varepsilon$ 
if  $e = \varepsilon$  then
try  $(\bar{\mathbf{P}}, r, v, \pi, \text{crs}) \leftarrow \text{query}(\text{GetPolicy})$  to  $\mathcal{F}_{\text{crs}}$ 
req  $\text{N}_{\mathcal{W}}.\text{Vrf}(\text{crs}, v, \pi)$ 
 $r' \leftarrow \{0, 1\}^\lambda$ 
 $(\text{pub}, \text{priv}) \leftarrow \text{NISC}_{\mathcal{C}^{\mathcal{P}}}.\text{Init}(1^\lambda, \bar{\mathbf{P}}; r')$ 
 $\pi \leftarrow \text{N}_{\mathcal{W}}.\text{Prv}(\text{crs}, (\text{pub}, v), (r, r', \mathbf{P}, \text{priv}))$ 
try query (SetPolicy,  $(\text{pub}, \pi)$ ) to  $\mathcal{F}_{\text{crs}}$ 
return
 $(c, c', \pi, i) \leftarrow e$ 
req  $i \notin M \wedge \text{N}_{\mathcal{R}}.\text{Vrf}(\text{crs}, (c, c', \text{spk}_S, \text{pub}, \text{pk}_J, i), \pi)$ 
 $d \leftarrow \text{NISC}.\text{Dec}(\text{priv}, c')$ ;  $M \leftarrow i$ 
if  $d = \text{GOOD}$  then
return e
else
return  $(d, i)$ 

// Policy evaluation helper
Helper  $\mathcal{C}^{\mathcal{P}}((m, e_j), \mathcal{O}(\mathbf{P}, 1^n, 1^\lambda))$ 
if  $(\mathcal{O}(\mathbf{P}, 1^n, 1^\lambda))(m)$  then
return GOOD
else
return  $e_j$ 

// Witness relations for set policy
 $\mathcal{W} := \{v : \exists w = (r, r', \mathbf{P}, \bar{\mathbf{P}}) \text{ s.t. } \bar{\mathbf{P}} = \mathcal{O}(\mathbf{P}, 1^n, 1^\lambda; r'), v = \text{Hash}(\bar{\mathbf{P}}; r), \mathbf{P} \in \mathcal{P}\}$ 
 $\bar{\mathcal{W}} := \{(\text{pub}, v) : \exists w = (r, r', \bar{\mathbf{P}}, \text{priv}) \text{ s.t. } (\text{pub}, \text{priv}) \leftarrow \text{NISC}.\text{Init}(1^\lambda, \bar{\mathbf{P}}; r'), v = \text{Hash}(\bar{\mathbf{P}}; r)\}$ 

```

Fig. 8: The protocol that realizes the ideal functionality of Fig. 1, parameterized by the security parameter λ , the policy class $\mathcal{P}$, the universal circuit $\mathcal{C}^{\mathcal{P}}$ over $\mathcal{P}$, the set of parties $\mathcal{U} = \{\text{S}, \text{R}, \text{PC}, \text{A}, \text{J}\}$, an EU-CMA signature scheme $\Sigma = (\text{KeyGen}, \text{Sign}, \text{Ver})$, a symmetric, committing encryption scheme $\text{SE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$, a NISC scheme $\text{NISC} = (\text{Init}, \text{Enc}, \text{Dec})$ for $\mathcal{C}^{\mathcal{P}}$, a public key encryption scheme $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$, a hash function Hash , a VBB obfuscator $\mathcal{O}$, and a NIZK scheme $\text{N} = (\text{Gen}, \text{Prv}, \text{Vrf}, \mathcal{S})$. The protocol is in the $(\mathcal{F}_{\text{AKS}}, \mathcal{F}'_{\text{crs}}, \mathcal{F}_{2\text{PR}})$ -hybrid world.

Audit operation, when executed with empty input $e \neq \varepsilon$, matches that of POCO. Otherwise, the auditor recovers $(\bar{\mathbf{P}}, r, v, \pi, \text{crs})$, where, besides crs , the rest of the values were set by PC. The auditor verifies that the obfuscated policy $\bar{\mathbf{P}}$, with hash value v is in $\mathcal{P}$, and computes the initialization operation of NISC with input $\bar{\mathbf{P}}$, as well as a NIZK proof of the correctness of the computation and the fact that the hash of $\mathbf{P}$ and r , matches v . Finally, the sender fetches $(\text{pub}, v, \pi, \pi', \text{crs})$, and verifies the two proofs.

Security. Regarding *policy privacy*, observe that the auditor learns $(\bar{\mathbf{P}}, r, v, \pi, \text{crs})$, where $\bar{\mathbf{P}}$ is the obfuscated version of $\mathbf{P}$, therefore, assuming security of VBB obfuscation scheme $\mathcal{O}$ and the zero-knowledge property of NIZK scheme, the only information related to $\mathbf{P}$ that is leaked to A is the policy evaluation outcome

over the sent messages, as required by *strong policy privacy*. Furthermore, the sender learns $(\text{pub}, v, \pi, \pi', \text{crs})$, where by the NISC security pub doesn't leak anything about $\bar{\mathbf{P}}$, π and π' do not leak any information about $\mathbf{P}$ and $\bar{\mathbf{P}}$ assuming the zero-knowledge property of $\mathbf{N}$, and the same holds for v , assuming Hash is modeled as a random oracle.

Regarding *policy validity*, assuming the soundness property of $\mathbf{N}$, π verifies correctly only if v is the hash value of a policy $\mathbf{P} \in \mathcal{P}$. Furthermore, a malicious auditor is required to execute the NISC initialization operation over an obfuscated policy $\bar{\mathbf{P}}'$, such that for some r' , $\text{Hash}(\bar{\mathbf{P}}'; r') = v$, therefore, assuming that Hash is modeled as a random oracle and the soundness property of $\mathbf{N}$, π' verifies only if $(\bar{\mathbf{P}}', r') = (\bar{\mathbf{P}}, r)$ and $\mathbf{A}$ follows the computation, ensuring policy validity. I.e., NISC encryptions computed under pub and then decrypted under priv , will only leak the policy evaluation outcome w.r.t. a policy $\mathbf{P} \in \mathcal{P}$, chosen by PC .

The remaining properties of PCSM follow as in POCO. Our result is formalized in the following theorem.

Theorem 3. *Let $\mathcal{F}_{\text{PCSM}}$ be the functionality of Fig. 1 w.r.t. strong policy privacy, Σ be an EU-CMA-secure signature scheme, SE be a CCA-secure committing encryption scheme, NISC be a non-interactive secure computation scheme for $\mathcal{C}^{\mathcal{P}}$, PKE be an IND-CCA-secure PKE scheme, $\mathbf{N}$ be a simulation sound extractable zero-knowledge scheme, Hash is modeled as a random oracle, and $\mathcal{O}$ be a VBB obfuscation scheme, instantiating the protocol O-POCO of Fig. 8 in the $(\mathcal{F}_{\text{AKS}}, \mathcal{F}'_{\text{crs}}, \mathcal{F}_{2\text{PR}})$ -hybrid world. Then, for any $\mathcal{A}$, there exists an $\mathcal{S}_{\text{O-POCO}}$ and adversaries $\mathcal{B}_1, \dots, \mathcal{B}_8$, s.t.*

$$\begin{aligned} & \Pr[\text{IDEAL}_{\mathcal{F}_{\text{PCSM}}, \mathcal{S}_{\text{O-POCO}}}(\mathcal{A}) \Rightarrow 1] - \Pr[\text{REAL}_{\text{O-POCO}, \mathcal{F}_{\text{AKS}}, \mathcal{F}'_{\text{crs}}, \mathcal{F}_{2\text{PR}}}(\mathcal{A}) \Rightarrow 1] \leq \\ & \quad + \text{Adv}_{\mathbf{N}}^{\text{ZK}}(\mathcal{B}_1) + \text{Adv}_{\mathbf{N}}^{\text{SSE}}(\mathcal{B}_2) + \text{Adv}_{\Sigma}^{\text{EU-CMA}}(\mathcal{B}_3) + \text{Adv}_{\text{SE}}^{\text{COM}}(\mathcal{B}_4) \\ & \quad + \text{Adv}_{\text{NISC}}^{\text{NISC}}(\mathcal{B}_5) + q \cdot \left(\text{Adv}_{\text{PKE}}^{\text{IND-CCA}}(\mathcal{B}_6) + \text{Adv}_{\text{SE}}^{\text{CCA}}(\mathcal{B}_7) + \text{Adv}_{\mathcal{O}}^{\text{VBB}}(\mathcal{B}_8) \right) \\ & \quad + \varepsilon_{\text{se-cor}} + \varepsilon_{\text{nisc-cor}} + \varepsilon_{\text{pke-cor}} + \varepsilon_{\Sigma\text{-cor}} + \varepsilon_{\mathbf{N}\text{-com}} + \varepsilon_{\mathcal{O}\text{-cor}}, \end{aligned}$$

where q denotes the number of queries made by $\mathcal{A}$ and $\varepsilon_{\text{se-cor}}$, $\varepsilon_{\text{nisc-cor}}$, $\varepsilon_{\text{pke-cor}}$, $\varepsilon_{\Sigma\text{-cor}}$, $\varepsilon_{\mathbf{N}\text{-com}}$, $\varepsilon_{\mathcal{O}\text{-cor}}$, denote upper bounds on the probability of violating the correctness (or completeness) of SE, NISC, PKE, Σ , $\mathbf{N}$, and $\mathcal{O}$, respectively.

The proof of the above theorem is along the lines of the proof of Theorem 1, plus the additional hybrid that substitutes $\mathbf{P}$ with the obfuscated $\bar{\mathbf{P}}$, and the indistinguishability uses the VBB security of $\mathcal{O}$.

As a corollary of the above theorem, we derive a PCSM scheme for lists of point functions, that achieves strong policy privacy, under standard assumptions.

Corollary 2. *Let $\mathcal{F}_{\text{PCSM}}$ be the functionality of Fig. 1 w.r.t. strong policy privacy, Σ be an EU-CMA-secure signature scheme, SE be a CCA-secure committing encryption scheme, NISC be a non-interactive secure computation scheme for $\mathcal{C}^{\mathcal{P}}$, where $\mathcal{P}$ is the class point functions, PKE be an IND-CCA-secure PKE scheme, $\mathbf{N}$ be a simulation sound extractable zero-knowledge scheme, Hash is modeled as*

a random oracle, and $\mathcal{O}$ be a VBB obfuscation scheme for lists of point functions, instantiating the protocol O-POCO of Fig. 8 in the $(\mathcal{F}_{\text{AKS}}, \mathcal{F}'_{\text{CRS}}, \mathcal{F}_{2\text{PR}})$ -hybrid world. Then, for any $\mathcal{A}$, there exists an $\mathcal{S}_{\text{O-POCO}}$ and adversaries $\mathcal{B}_1, \dots, \mathcal{B}_8$, s.t. $\Pr[\text{IDEAL}_{\mathcal{F}_{\text{PCSM}}, \mathcal{S}_{\text{O-POCO}}}(\mathcal{A}) \Rightarrow 1] - \Pr[\text{REAL}_{\text{O-POCO}, \mathcal{F}_{\text{AKS}}, \mathcal{F}'_{\text{CRS}}, \mathcal{F}_{2\text{PR}}}(\mathcal{A}) \Rightarrow 1]$ is negligible in the security parameter, where the exact bound matches that of Theorem 3.

References

1. A. Afshar, P. Mohassel, B. Pinkas, and B. Riva. Non-interactive secure computation based on cut-and-choose. In P. Q. Nguyen and E. Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 387–404. Springer, Heidelberg, May 2014.
2. W. G. Alliance. Global threat assessment 2019: Working together to end the sexual exploitation of children online, 2019.
3. J. Alwen, S. Coretti, and Y. Dodis. The double ratchet: Security notions, proofs, and modularization for the Signal protocol. In Y. Ishai and V. Rijmen, editors, *EUROCRYPT 2019, Part I*, volume 11476 of *LNCS*, pages 129–158. Springer, Heidelberg, May 2019.
4. J. Alwen, S. Coretti, Y. Dodis, and Y. Tselekounis. Security analysis and improvements for the IETF MLS standard for group messaging. In D. Micciancio and T. Ristenpart, editors, *CRYPTO 2020, Part I*, volume 12170 of *LNCS*, pages 248–277. Springer, Heidelberg, Aug. 2020.
5. J. Alwen, S. Coretti, Y. Dodis, and Y. Tselekounis. Modular design of secure group messaging protocols and the security of MLS. In G. Vigna and E. Shi, editors, *ACM CCS 2021*, pages 1463–1483. ACM Press, Nov. 2021.
6. J. Alwen, S. Coretti, D. Jost, and M. Mularczyk. Continuous group key agreement with active security. In R. Pass and K. Pietrzak, editors, *TCC 2020, Part II*, volume 12551 of *LNCS*, pages 261–290. Springer, Heidelberg, Nov. 2020.
7. J. Alwen, D. Hartmann, E. Kiltz, and M. Mularczyk. Server-aided continuous group key agreement. In H. Yin, A. Stavrou, C. Cremers, and E. Shi, editors, *ACM CCS 2022*, pages 69–82. ACM Press, Nov. 2022.
8. J. Alwen, D. Jost, and M. Mularczyk. On the insider security of MLS. In Y. Dodis and T. Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 34–68. Springer, Heidelberg, Aug. 2022.
9. J. Alwen, M. Mularczyk, and Y. Tselekounis. Fork-resilient continuous group key agreement. In H. Handschuh and A. Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023*, pages 396–429, Cham, 2023. Springer Nature Switzerland.
10. S. Badrinarayanan, S. Garg, Y. Ishai, A. Sahai, and A. Wadia. Two-message witness indistinguishability and secure computation in the plain model from new assumptions. In T. Takagi and T. Peyrin, editors, *ASIACRYPT 2017, Part III*, volume 10626 of *LNCS*, pages 275–303. Springer, Heidelberg, Dec. 2017.
11. F. Banfi, K. Gegier, M. Hirt, and U. Maurer. Anamorphic encryption, revisited. *Cryptology ePrint Archive*, 2023.
12. B. Barak, O. Goldreich, R. Impagliazzo, S. Rudich, A. Sahai, S. Vadhan, and K. Yang. On the (im) possibility of obfuscating programs. *Journal of the ACM (JACM)*, 59(2):1–48, 2012.
13. B. Barak, O. Goldreich, R. Impagliazzo, S. Rudich, A. Sahai, S. P. Vadhan, and K. Yang. On the (im)possibility of obfuscating programs. In J. Kilian, editor, *CRYPTO 2001*, volume 2139 of *LNCS*, pages 1–18. Springer, Heidelberg, Aug. 2001.

14. J. Bartusek, S. Garg, A. Jain, and G.-V. Policharla. End-to-end secure messaging with traceability only for illegal content. In C. Hazay and M. Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 35–66. Springer, Heidelberg, Apr. 2023.
15. A. Bhowmick, D. Boneh, S. Myers, K. Talwar, and K. Tarbe. The apple psi system. *Apple, Inc., Tech. Rep*, 2021.
16. A. Bienstock, J. Fairuze, S. Garg, P. Mukherjee, and S. Raghuraman. A more complete analysis of the Signal double ratchet algorithm. In Y. Dodis and T. Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 784–813. Springer, Heidelberg, Aug. 2022.
17. R. Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd FOCS*, pages 136–145. IEEE Computer Society Press, Oct. 2001.
18. R. Canetti, P. Jain, M. Swanberg, and M. Varia. Universally composable end-to-end secure messaging. In Y. Dodis and T. Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 3–33. Springer, Heidelberg, Aug. 2022.
19. K. Cohn-Gordon, C. Cremers, B. Dowling, L. Garratt, and D. Stebila. A formal security analysis of the signal messaging protocol. *Journal of Cryptology*, 33(4):1914–1983, Oct. 2020.
20. J. Comey. Going dark: Are technology, privacy, and public safety on a collision course? *US Department of Justice*, 2014.
21. E. Commission. Child sexual abuse, 2022.
22. B. David, F. Engelmann, T. Frederiksen, M. Kohlweiss, E. Pagnin, and M. Volkhov. Updatable privacy-preserving blueprints. *Cryptology ePrint Archive*, 2023.
23. E. De Cristofaro and G. Tsudik. Practical private set intersection protocols with linear complexity. In R. Sion, editor, *FC 2010*, volume 6052 of *LNCS*, pages 143–159. Springer, Heidelberg, Jan. 2010.
24. A. De Santis, G. Di Crescenzo, R. Ostrovsky, G. Persiano, and A. Sahai. Robust non-interactive zero knowledge. In J. Kilian, editor, *CRYPTO 2001*, volume 2139 of *LNCS*, pages 566–598. Springer, Heidelberg, Aug. 2001.
25. Y. Dodis, P. Grubbs, T. Ristenpart, and J. Woodage. Fast message franking: From invisible salamanders to encryptment. In H. Shacham and A. Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 155–186. Springer, Heidelberg, Aug. 2018.
26. Y. Dodis, B. Magri, N. Stephens-Davidowitz, and Y. Tselekounis. Guarding the signal: Secure messaging with reverse firewalls. In *Annual International Cryptology Conference*, pages 39–74. Springer, 2025.
27. K. Emura, G. Hanaoka, Y. Kawai, T. Matsuda, K. Ohara, K. Omote, Y. Sakai, et al. Group signatures with message-dependent opening: Formal definitions and constructions. *Security and communication networks*, 2019, 2019.
28. V. Fetzer, M. Klooß, J. Müller-Quade, M. Raiber, and A. Rupp. Universally composable auditable surveillance. *Cryptology ePrint Archive*, 2023.
29. M. Green. Thinking about “traceability”., 2021.
30. M. Green, G. Kaptchuk, and G. V. Laer. Abuse resistant law enforcement access systems. In A. Canteaut and F.-X. Standaert, editors, *EUROCRYPT 2021, Part III*, volume 12698 of *LNCS*, pages 553–583. Springer, Heidelberg, Oct. 2021.
31. P. Grubbs, J. Lu, and T. Ristenpart. Message franking via committing authenticated encryption. In J. Katz and H. Shacham, editors, *CRYPTO 2017, Part III*, volume 10403 of *LNCS*, pages 66–97. Springer, Heidelberg, Aug. 2017.
32. Y. Ishai, E. Kushilevitz, R. Ostrovsky, M. Prabhakaran, and A. Sahai. Efficient non-interactive secure computation. In K. G. Paterson, editor, *EUROCRYPT 2011*, volume 6632 of *LNCS*, pages 406–425. Springer, Heidelberg, May 2011.

33. R. Issa, N. Alhaddad, and M. Varia. Hecate: Abuse reporting in secure messengers with sealed sender. In K. R. B. Butler and K. Thomas, editors, *USENIX Security 2022*, pages 2335–2352. USENIX Association, Aug. 2022.
34. N. Kobeissi, K. Bhargavan, and B. Blanchet. Automated verification for secure messaging protocols and their implementations: A symbolic and computational approach. In *2017 IEEE European symposium on security and privacy (EuroS&P)*, pages 435–450. IEEE, 2017.
35. M. Kohlweiss, A. Lysyanskaya, and A. Nguyen. Privacy-preserving blueprints. In C. Hazay and M. Stam, editors, *EUROCRYPT 2023, Part II*, volume 14005 of *LNCS*, pages 594–625. Springer, Heidelberg, Apr. 2023.
36. A. Kulshrestha and J. R. Mayer. Identifying harmful media in end-to-end encrypted communication: Efficient private membership computation. In M. Bailey and R. Greenstadt, editors, *USENIX Security 2021*, pages 893–910. USENIX Association, Aug. 2021.
37. B. Libert and M. Joye. Group signatures with message-dependent opening in the standard model. In J. Benaloh, editor, *CT-RSA 2014*, volume 8366 of *LNCS*, pages 286–306. Springer, Heidelberg, Feb. 2014.
38. L. Liu, D. S. Roche, A. Thériault, and A. Yerukhimovich. Fighting fake news in encrypted messaging with the fuzzy anonymous complaint tally system (FACTS). Cryptology ePrint Archive, Report 2021/1148, 2021. <https://eprint.iacr.org/2021/1148>.
39. B. Lynn, M. Prabhakaran, and A. Sahai. Positive results and techniques for obfuscation. In C. Cachin and J. Camenisch, editors, *EUROCRYPT 2004*, volume 3027 of *LNCS*, pages 20–39. Springer, Heidelberg, May 2004.
40. F. C. Ministerial. Joint meeting of fcm and quintet of attorneys-general. *London: Government of the United Kingdom*, 2019.
41. A. Namavari, B. Wang, S. Menda, B. Nassi, N. Tyagi, J. Grimmelmann, A. Zhang, and T. Ristenpart. Private hierarchical governance for encrypted messaging. In *2024 IEEE Symposium on Security and Privacy (SP)*, page 2610–2629. IEEE, May 2024.
42. C. of the European Union. Council conclusions on combating the sexual abuse of children, 2019.
43. P. of the United Kingdom. Online safety bill, 2021.
44. O. of the United Nations High Commissioner for Human Rights. The right to privacy in the digital age, 2022.
45. P. Patel, W. Barr, P. Dutton, A. Little, and B. Blair. International statement: End-to-end encryption and public safety, 2021.
46. C. Peale, S. Eskandarian, and D. Boneh. Secure complaint-enabled source-tracking for encrypted messaging. In G. Vigna and E. Shi, editors, *ACM CCS 2021*, pages 1484–1506. ACM Press, Nov. 2021.
47. G. Persiano, D. H. Phan, and M. Yung. Anamorphic encryption: Private communication against a dictator. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 34–63. Springer, 2022.
48. S. Scheffler, A. Kulshrestha, and J. Mayer. Public verification for private hash matching. *Cryptology ePrint Archive*, 2023.
49. S. Scheffler and J. Mayer. Sok: Content moderation for end-to-end encryption. *Proceedings on Privacy Enhancing Technologies*, 2:403–429, 2023.
50. O. W. Systems. The signal application, 2023. <https://github.com/signalapp>.
51. N. Tyagi, P. Grubbs, J. Len, I. Miers, and T. Ristenpart. Asymmetric message franking: Content moderation for metadata-private end-to-end encryption. In

- A. Boldyreva and D. Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 222–250. Springer, Heidelberg, Aug. 2019.
52. N. Tyagi, I. Miers, and T. Ristenpart. Traceback for end-to-end encrypted messaging. In L. Cavallaro, J. Kinder, X. Wang, and J. Katz, editors, *ACM CCS 2019*, pages 413–430. ACM Press, Nov. 2019.
53. N. Watt, R. Mason, and I. Traynor. David cameron pledges anti-terror law for internet after paris attacks. *The Guardian*, 12, 2015.

Supplementary Material

A Deferred Preliminaries

A.1 Committing Symmetric Encryption

Our definition follows a simplified version of the receiver's binding property of Grubbs et al. [31]. Our definition is simplified in the sense that it omits the verification operation (i.e., we use the decryption operation instead), as well as the associated data from the operations' input.

Syntax. A symmetric encryption (SE) scheme SE consists of three algorithms:

- $k \leftarrow \text{KeyGen}(1^\lambda)$: receives a security parameter and outputs a key $k \leftarrow \text{KeyGen}(1^\lambda)$.
- $c \leftarrow \text{Enc}(k, m)$: receives key k and message m and outputs a ciphertext c .
- $m \leftarrow \text{Dec}(k, c)$: receives key k and ciphertext c and outputs a message m or $\perp$.

Correctness. An SE scheme $\text{SE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ is correct if for every message m , $\Pr[\text{Dec}(k, \text{Enc}(k, m)) = m] = 1$, where $k \leftarrow \text{KeyGen}(1^\lambda)$.

Our setting requires one-time security, i.e., only one ciphertext per secret key is generated. The security games defined below capture this notion.

Definition 3 (Security). An SE scheme SE is an ε -CCA secure committing encryption scheme if:

- **CCA security:** For every PPT attacker $\mathcal{A}$,

$$\Pr \left[b = b' \mid \begin{array}{l} k \leftarrow \text{KeyGen}(1^\lambda); (m_0, m_1) \leftarrow \mathcal{A}(1^\lambda); \\ b \leftarrow \{0, 1\}; c \leftarrow \text{Enc}(k, m_b); b' \leftarrow \mathcal{A}^{\text{Dec}(k, \cdot)}(c) \end{array} \right] \leq \varepsilon,$$

where $\mathcal{A}$ is not allowed to provide the challenge ciphertext c as input to the decryption oracle, $\text{Dec}(k, \cdot)$. The advantage of $\mathcal{A}$ in the above game is denoted by $\text{Adv}_{\text{SE}}^{\text{CCA}}(\mathcal{A})$.

- **Commitment:** For every PPT attacker $\mathcal{A}$,

$$\Pr \left[\begin{array}{l} k \neq k' \wedge \\ \perp \neq \text{Dec}(k, c) \wedge \text{Dec}(k', c) \neq \perp \\ \text{Dec}(k, c) \neq \text{Dec}(k', c) \end{array} \mid (k, k', c) \leftarrow \mathcal{A}(1^\lambda) \right] \leq \varepsilon.$$

The advantage of $\mathcal{A}$ in the above game is denoted by $\text{Adv}_{\text{SE}}^{\text{COM}}(\mathcal{A})$.

A.2 Non-Interactive Zero-Knowledge

Definition 4 (Robust NIZK [24]). Let $\mathcal{R}$ be a witness relation for a language $\mathcal{L} \in \text{NP}$. A non-interactive zero-knowledge argument system for $\mathcal{R}$ is a vector of algorithms $(\text{Gen}, \text{Prv}, \text{Vrf}, \mathcal{NS} = (\mathcal{NS}_1, \mathcal{NS}_2))$, satisfying the following properties:

- **Completeness.** For any $y \in \mathcal{L}$ and any w such that $(y, w) \in \mathcal{R}$,

$$\Pr \left[\text{Vrf}(\text{crs}, y, \pi) = 1 \mid \begin{array}{l} \text{crs} \leftarrow \text{Gen}(1^\lambda) \\ \pi \leftarrow \text{Prv}(y, w, \text{crs}) \end{array} \right] \geq 1 - \text{negl}(\lambda).$$

- **Zero-knowledge.** For all non-uniform PPT adversaries $\mathcal{A}$, we have

$$\Pr \left[\mathcal{A}^{\text{Prv}(\text{crs}, \cdot, \cdot)}(\text{crs}) = 1 \mid \text{crs} \leftarrow \text{Gen}(1^\lambda) \right] \approx \Pr \left[\mathcal{A}^{\mathcal{NS}(\hat{\text{crs}}, \tau, \cdot, \cdot)}(\hat{\text{crs}}) = 1 \mid (\hat{\text{crs}}, \tau) \leftarrow \mathcal{NS}_1(1^\lambda) \right],$$

where $\mathcal{NS}(\hat{\text{crs}}, \tau, y, w) = \mathcal{NS}_2(\hat{\text{crs}}, \tau, y)$ if $(y, w) \in \mathcal{R}$, otherwise outputs $\perp$.

- **Simulation sound extractability.** There exists a PPT algorithm $\mathcal{E}$, such that for all PPT algorithms $\mathcal{A}$, we have

$$\Pr \left[\begin{array}{l} \text{Vrf}(\hat{\text{crs}}, y, \pi) = 1 \wedge \\ w \notin \mathcal{R}(y) \wedge \\ (y, \pi) \notin Q \end{array} \mid \begin{array}{l} (\hat{\text{crs}}, \tau) \leftarrow \mathcal{NS}_1(1^\lambda) \\ (y, \pi) \leftarrow \mathcal{A}^{\mathcal{NS}_2(\hat{\text{crs}}, \tau, \cdot)}(\hat{\text{crs}}) \\ w \leftarrow \mathcal{E}(\tau, y, \pi) \end{array} \right] \leq \text{negl}(\lambda),$$

where Q denotes the simulation queries and answers (y_i, π_i) , produced by the interaction between $\mathcal{A}$ and $\mathcal{NS}_2$.

We define as the advantage of $\mathcal{A}$ in breaking the zero-knowledge and simulation sound extractability properties, the bounds in the above probabilities, and we denote them by $\text{Adv}_N^{\text{ZK}}(\mathcal{A})$ and $\text{Adv}_N^{\text{SSE}}(\mathcal{A})$, respectively.

A.3 Digital Signatures

In the current section we define existential (strong) unforgeability against chosen message attacks (EU-CMA).

Syntax. A digital signature (DS) scheme is a triple of algorithms $\Sigma = (\text{KeyGen}, \text{Sign}, \text{Ver})$ with the following syntax:

- $(\text{spk}, \text{ssk}) \leftarrow \text{KeyGen}(1^\lambda)$: receives a security parameter λ and outputs a fresh public and secret keypair $(\text{spk}, \text{ssk}) \leftarrow \text{KeyGen}(\lambda)$.
- $\sigma \leftarrow \text{Sign}(\text{ssk}, m)$: receives a signing key ssk and a message m and produces a signature $\sigma \leftarrow \text{Sign}(\text{ssk}, m)$.
- $b \leftarrow \text{Ver}(\text{spk}, m, \sigma)$: receives a public key spk , a message m and signature σ , and outputs a bit $b \in \{0, 1\}$.

A DS scheme must satisfy the following correctness property.

Correctness. For any message m ,

$$\Pr[(\text{spk}, \text{ssk}) \leftarrow \text{KeyGen}(1^\lambda); \sigma \leftarrow \text{Sign}(\text{ssk}, m) : \text{Ver}(\text{spk}, m, \sigma) = 1] = 1.$$

Strong unforgeability game. For any PPT adversary $\mathcal{A}$ and DS scheme Σ , we consider the following security game:

- $(\text{spk}, \text{ssk}) \leftarrow \text{KeyGen}(1^\lambda)$.
- $\mathcal{A}$ receives spk and oracle access to $\text{Sign}(\text{ssk}, \cdot)$. Let Q be the set of queries made by $\mathcal{A}$.
- $\mathcal{A}$ outputs (m, σ) .
- If $\text{Ver}(\text{spk}, m, \sigma) = 1$ and $(m, \sigma) \notin Q$, output 1, otherwise, output 0.

$\mathcal{A}$ wins if the game outputs 1. The advantage of $\mathcal{A}$ in winning the above game is denoted by $\text{Adv}_\Sigma^{\text{EU-CMA}}(\mathcal{A})$.

Definition 5 (Strong unforgeability). A DS scheme Σ is ϵ -EU-CMA-secure, if for all PPT adversaries $\mathcal{A}$, $\text{Adv}_\Sigma^{\text{EU-CMA}}(\mathcal{A}) \leq \epsilon$.

B Deferred Proofs

B.1 Security Proof of POCO

In the current section we prove security of POCO (cf. Fig. 4), formalized in the following theorem.

Theorem 4. Let $\mathcal{F}_{\text{PCSM}}$ be the functionality of Fig. 1 w.r.t. weaker policy privacy, Σ be an EU-CMA-secure signature scheme, SE be a CCA-secure committing encryption scheme, NISC be a non-interactive secure computation scheme for $\mathcal{C}^{\mathcal{P}}$, PKE be an IND-CCA-secure PKE scheme, and N be a simulation sound extractable zero-knowledge scheme, instantiating the protocol POCO of Fig. 4 in the $(\mathcal{F}_{\text{AKS}}, \mathcal{F}_{\text{CRS}}, \mathcal{F}_{2\text{PR}})$ -hybrid world. Then, for any $\mathcal{A}$, there exists an $\mathcal{S}_{\text{POCO}}$ and adversaries $\mathcal{B}_1, \dots, \mathcal{B}_7$, s.t.

$$\begin{aligned} & \Pr[\text{IDEAL}_{\mathcal{F}_{\text{PCSM}}, \mathcal{S}_{\text{POCO}}}(\mathcal{A}) \Rightarrow 1] - \Pr[\text{REAL}_{\text{POCO}, \mathcal{F}_{\text{AKS}}, \mathcal{F}_{\text{CRS}}, \mathcal{F}_{2\text{PR}}}(\mathcal{A}) \Rightarrow 1] \leq \\ & \quad + \text{Adv}_{\text{N}}^{\text{ZK}}(\mathcal{B}_1) + \text{Adv}_{\text{N}}^{\text{SSE}}(\mathcal{B}_2) + \text{Adv}_{\Sigma}^{\text{EU-CMA}}(\mathcal{B}_3) + \text{Adv}_{\text{SE}}^{\text{COM}}(\mathcal{B}_4) \\ & \quad + \text{Adv}_{\text{NISC}}^{\text{NISC}}(\mathcal{B}_5) + q \cdot \left(\text{Adv}_{\text{PKE}}^{\text{IND-CCA}}(\mathcal{B}_6) + \text{Adv}_{\text{SE}}^{\text{CCA}}(\mathcal{B}_7) \right) \\ & \quad + \epsilon_{\text{se-cor}} + \epsilon_{\text{nisc-cor}} + \epsilon_{\text{pke-cor}} + \epsilon_{\Sigma\text{-cor}} + \epsilon_{\text{N-com}}, \end{aligned}$$

where q denotes the number of queries made by $\mathcal{A}$ and $\epsilon_{\text{se-cor}}, \epsilon_{\text{nisc-cor}}, \epsilon_{\text{pke-cor}}, \epsilon_{\Sigma\text{-cor}}, \epsilon_{\text{N-com}}$, denote upper bounds on the probability of violating the correctness (or completeness) of SE, NISC, PKE, Σ and N, respectively.

Our proof proceeds in hybrids, where each hybrid $\mathbf{H}^i$ is parameterized by a functionality $\mathcal{F}^i$ and simulator $\mathcal{S}^i$, that will eventually converge to the ideal-world execution IDEAL, with respect to the ideal functionality $\mathcal{F}_{\text{PCSM}}$ and simulator $\mathcal{S}_{\text{POCO}}$. The simulator $\mathcal{S}_{\text{POCO}}$ is depicted in Fig. 9. The hybrids are as follows.

$\mathcal{S}_{\text{POCO}}$

Parameters: $\lambda, \mathcal{P}, \mathcal{U}, \Sigma, \text{SE}, \text{NISC}, \text{PKE}, \text{N}$

```

// Executed on first input
Initialization
  (spkS, sskS) ←  $\Sigma$ .KeyGen
  (skJ, skJ) ←  $\text{PKE}$ .KeyGen
   $\mathcal{F}_{2\text{PR}}$ .init;  $i \leftarrow 0$ 
   $M, \text{RM}, I \leftarrow \emptyset$ ;  $v \leftarrow \text{false}$ 
  if  $S \in \mathcal{C}$  then
    On query (Gen,  $\Sigma$ ) from  $S$  return (spkS, sskS)
    On query (GetPK,  $J, \text{PKE}$ ) from  $S$  return pkJ
    On query (Init) from  $S$  to  $\mathcal{F}_{2\text{PR}}$  return ack
  if  $A \in \mathcal{C}$  then
    On query (GetPK,  $S, \Sigma$ ) from  $A$  return spkS

// Set policy
Input (SetPolicy,  $P$ ) from  $\mathcal{F}_{\text{PCSM}}$ 
  if  $PC \notin \mathcal{C}$  then
    if  $A \in \mathcal{C}$  then
       $r \leftarrow \{0, 1\}^\lambda$ 
      (pub, priv) ←  $\text{NISC}$ .Init( $1^\lambda, P; r$ )
       $\pi \leftarrow \text{N}_{\text{WV}}$ .Prv(crs, pub, ( $r, P$ , priv))
       $\mathcal{F}_{\text{crs}}$ (pub, priv,  $\pi$ )
    else
      pub ←  $\text{NISC}$ . $S_1(1^\lambda)$ 
       $\pi \leftarrow \text{N}_{\text{WV}}$ . $\mathcal{N}S_2(\text{crs}, \tau, \text{pk})$ 
       $\mathcal{F}_{\text{crs}}$ (pub,  $\varepsilon, \pi$ )
    return
  Send (SetPolicy,  $P$ ) to  $A$ ; receive (pub, priv,  $\pi$ )
  if  $\text{N}_{\text{WV}}$ .Vrf(crs, pub,  $\pi$ ) then
    ext ←  $\text{N}_{\text{WV}}$ . $\mathcal{E}(\tau, \text{pub}, \pi)$ 
    if ext  $\neq \perp$  then
      ( $r, P, \cdot$ ) ← ext
    else
      Abort
   $\mathcal{F}_{\text{crs}}$ (pub, priv,  $\pi$ )
  return  $P$ 

// Audit ciphertext – generated via Send
Input (Audit, mid,  $e$ )
  if  $A \notin \mathcal{C}$  then
    ( $\cdot, \cdot, \cdot, i$ ) ←  $e$ ; req  $i \notin I$ 
    req  $\exists(m, \sigma, e_j, b) : (m, \sigma, e, e_j, b, i, \text{mid}) \in M$ 
    if  $b = 1$  then
      return  $e$ 
    else
      return  $e_j$ 
   $I \leftarrow I \cup i$ 
  else
    Send (Audit, mid,  $e$ ) to  $A$ 
    On receive query (GetPolicy) from  $A$  send
      (priv, pub, crs) to  $A$ ; receive  $v$ 
    return  $v$ 

// Send message
Input (Send, mid,  $m, b$ ) from  $\mathcal{F}_{\text{PCSM}}$ 
  req crs  $\neq \varepsilon$ 
  if  $S \in \mathcal{C}$  then
    Send (Send,  $m$ ) to  $A$ 
    On query (GetPub) from  $A$  to  $\mathcal{F}_{\text{crs}}$ , send (pub,  $\pi$ , crs)
    to  $A$ 
    On query (GetKey,  $i$ ) from  $A$  to  $\mathcal{F}_{2\text{PR}}$ ,  $k \leftarrow \mathcal{F}_{2\text{PR}}$ .GetKey( $i$ )
    Send  $k$  to  $A$ 
    Receive  $e$ ; ( $c, c', \pi, i$ ) ←  $e$ 
    if  $\text{N}_{\text{R}}$ .Vrf(crs, ( $c, c', \text{spk}_S, \text{pub}, \text{pk}_J, i$ ),  $\pi$ ) = 1 then
      ext ←  $\text{N}_{\text{R}}$ . $\mathcal{E}(\tau, (c, c', \text{spk}_S, \text{pub}, \text{pk}_J, i), \pi)$ 
      if ext  $\neq \perp$  then
        ( $m, k', \sigma, r, r', e_j$ ) ← ext;  $v \leftarrow \text{true}$ 
        req  $k' = k \vee \text{SE}.\text{Dec}(k, c) = \text{SE}.\text{Dec}(k', c)$ 
        Send ( $e, m, v$ ) to  $\mathcal{F}_{\text{PCSM}}$ ; receive  $b$ 
      else
        Abort
    else
       $v \leftarrow \text{false}$ 
      Send ( $e, \perp, v$ ) to  $\mathcal{F}_{\text{PCSM}}$ 
  else
    (pub,  $\pi$ ) ←  $\mathcal{F}_{\text{crs}}$ .GetPub
    if  $PC \in \mathcal{C}$  then req  $\text{N}_{\text{WV}}$ .Vrf(crs, pub,  $\pi$ )
     $i \leftarrow i + 1$ 
     $k \leftarrow \mathcal{F}_{2\text{PR}}$ .GetKey( $i$ )
     $c \leftarrow \text{SE}.\text{Enc}(k, 0)$ 
    if  $b = 1$  then
      out ← GOOD
       $m, \sigma, e_j \leftarrow \perp$ 
    else
       $m \leftarrow 0$ 
       $\sigma \leftarrow \Sigma$ .Sign(sskS, ( $m, i$ ))
       $e_j \leftarrow \text{PKE}.\text{Enc}(\text{pk}_J, (m, \sigma))$ 
      out ←  $e_j$ 
       $c' \leftarrow \text{NISC}.\mathcal{S}_2(1^\lambda, \text{out})$ 
       $\pi \leftarrow \text{N}_{\text{WV}}$ . $\mathcal{N}S_2(\text{crs}, \tau, (c, c', \text{spk}_S, \text{pk}, \text{pk}_J, i))$ 
       $e \leftarrow (c, c', \pi, i)$ ;  $v \leftarrow \text{true}$ 
      Send ( $e, \perp, v$ ) to  $\mathcal{F}_{\text{PCSM}}$ 
  if  $v$  then  $M \leftarrow M \cup (m, \sigma, e, e_j, b, i, \text{mid})$ 
  return

// Deliver message – only deliver if  $A \in \mathcal{C}$ 
Input (Deliver, mid,  $e$ )
  ( $c, c', \pi, i$ ) ←  $e$ 
   $k \leftarrow \mathcal{F}_{2\text{PR}}$ .GetKey( $i$ )
   $m \leftarrow \text{SE}.\text{Dec}(k, c)$ 
  return  $m$ 

// Report message – only receive if  $A \in \mathcal{C}$ 
Input (Report, mid, ( $e_j, i$ )))
  req spkS  $\neq \varepsilon$ 
  ( $m, \sigma$ ) ←  $\text{PKE}.\text{Dec}(\text{sk}_J, e_j)$ 
  if  $S \notin \mathcal{C}$  then
    req  $\exists e, e'_j : (m, \sigma, e, e'_j, 0, i, \text{mid}) \in M \wedge i \notin \text{RM}$ 
     $\text{RM} \leftarrow \text{RM} \cup i$ 
    return  $e'_j$ 
  else
    req  $\Sigma$ .Ver(spkS, ( $m, i$ ),  $\sigma$ )
    return  $m$ 

```

Fig. 9: The simulator for the proof of Theorem 4.

$\mathbf{H}^0$: This hybrid matches the real world execution, $\text{REAL}_{\text{POCO}, \mathcal{F}_{\text{AKS}}, \mathcal{F}_{\text{crs}}, \mathcal{F}_{2\text{PR}}}(\mathcal{A})$.

$\mathbf{H}_{\mathcal{F}^1, \mathcal{S}^1}^1$: This hybrid is identical to $\mathbf{H}^0$, but translated to the ideal-world language. That is, we replace the real world execution of POCO , $\mathcal{F}_{\text{AKS}}$, $\mathcal{F}_{\text{crs}}$, $\mathcal{F}_{2\text{PR}}$, by an execution with a “trivial” ideal functionality $\mathcal{F}^1$ and a “trivial” simulator $\mathcal{S}^1$. $\mathcal{F}^1$ provides no security guarantees but matches the syntax of $\mathcal{F}^1$. That is, $\mathcal{F}^1$ forwards all inputs to $\mathcal{S}^1$ and allows it to choose all outputs. $\mathcal{S}^1$ uses the inputs from $\mathcal{F}^1$ to run the code of $\text{REAL}_{\text{POCO}, \mathcal{F}_{\text{AKS}}, \mathcal{F}_{\text{crs}}, \mathcal{F}_{2\text{PR}}}$ for honest parties, i.e., $\mathcal{S}^1$, using the honest parties inputs (forwarded from $\mathcal{F}^1$), it fully simulates their execution for corrupted ones, and computes the outputs exactly as in the real world.

Lemma 1. *The view of the adversary in $\mathbf{H}^0$ is identical to the one in $\mathbf{H}_{\mathcal{F}^1, \mathcal{S}^1}^1$.*

Proof. The proof is straightforward by inspection.

$\mathbf{H}_{\mathcal{F}^2, \mathcal{S}^2}^2$: This hybrid uses $\mathcal{F}^2$ which is identical to $\mathcal{F}^1$. $\mathcal{S}^2$ diverges from $\mathcal{S}^1$ as follows: if $S \notin \mathcal{C}$, for each honestly generated ciphertext e , $\mathcal{S}^2$ computes $b \leftarrow \mathbf{P}(m)$ and stores $(m, \sigma, e, e_J, b, i, \text{mid})$ to a list $\mathbf{M}$ (the adversarial view remains the same after this modification). Additionally, assuming $S \notin \mathcal{C}$, if $A \notin \mathcal{C}$, during the audit operation instead of calling $\mathbf{N}_{\mathcal{R}}.\text{Vrf}(\text{crs}, (c, c', \text{spk}_S, \text{pub}, \text{pk}_J, i), \pi)$, the simulator of the current hybrid simply checks whether there exist (m, σ, e_J, b) such that $(m, \sigma, e, e_J, b, i, \text{mid}) \in \mathbf{M}$. The indistinguishability between the hybrids follows by the completeness of $\mathbf{N}$ with respect to $\mathcal{R}$ and is formalized in the following lemma.

Lemma 2. *Assuming $\varepsilon_{\text{N-com}}$ -completeness of $\mathbf{N}$ with respect to $\mathcal{R}$, $\mathbf{H}_{\mathcal{F}^1, \mathcal{S}^1}^1$ cannot be distinguished from $\mathbf{H}_{\mathcal{F}^2, \mathcal{S}^2}^2$ with probability more than $\varepsilon_{\text{N-com}}$, where the probability runs over the randomness used by the two experiments.*

Proof. We define an adversary $\mathcal{B}$ that simulates $\mathbf{H}_{\mathcal{F}^2, \mathcal{S}^2}^2$ for $\mathcal{A}$ and outputs the first honestly generated NIZK proof $\pi \leftarrow \mathbf{N}_{\mathcal{R}}.\text{Prv}(\text{crs}, (c, c', \text{spk}_S, \text{pub}, \text{pk}_J, \text{mid}), (m, k, \sigma, r, r', e_J))$, which is computed by an honest sender according to the protocol specification, for which the verification fails. Clearly, the advantage of any adversary in distinguishing between the two hybrids is bounded by the probability of breaking the completeness property of $\mathbf{N}$, and the proof of the lemma is concluded.

$\mathbf{H}_{\mathcal{F}^3, \mathcal{S}^3}^3$: This hybrid uses $\mathcal{F}^3$ which is identical to $\mathcal{F}^2$. $\mathcal{S}^3$ diverges from $\mathcal{S}^2$ as follows: if $\text{PC}, S \notin \mathcal{C}$, an honest S skips the verification $\mathbf{N}_{\mathcal{W}}.\text{Vrf}(\text{crs}, \text{pub}, \pi)$. The indistinguishability between the hybrids follows by the completeness of $\mathbf{N}$ with respect to $\mathcal{W}$ and is formalized in the following lemma.

Lemma 3. *Assuming $\varepsilon_{\text{N-com}}$ -completeness of $\mathbf{N}$ with respect to $\mathcal{W}$, $\mathbf{H}_{\mathcal{F}^2, \mathcal{S}^2}^2$ cannot be distinguished from $\mathbf{H}_{\mathcal{F}^3, \mathcal{S}^3}^3$ with probability more than $\varepsilon_{\text{N-com}}$, where the probability runs over the randomness used by the two experiments.*

Proof. We define an adversary $\mathcal{B}$ that simulates $\mathbf{H}_{\mathcal{F}^3, \mathcal{S}^3}^3$ for $\mathcal{A}$ and outputs the first honestly generated NIZK proof $\pi \leftarrow \mathbf{N}_{\mathcal{W}}.\text{Prv}(\text{crs}, \text{pub}, (r, \mathbf{P}, \text{priv}))$, computed by an honest PC , for which the verification fails. Clearly, the advantage of

any adversary in distinguishing between the two hybrids is bounded by the probability of breaking the completeness property of $\mathbf{N}$, and the proof of the lemma is concluded.

$\mathbf{H}_{\mathcal{F}^4, \mathcal{S}^4}^4$: In this hybrid $\mathcal{F}^4$ is identical to $\mathcal{F}^3$, while $\mathcal{S}^4$, instead of computing the CRS via $\text{crs} \leftarrow \mathbf{N}_{\mathcal{R}}.\text{Gen}(1^\lambda)$, computes $(\text{cfs}, \tau) \leftarrow \mathbf{N}_{\mathcal{R}}.\mathcal{NS}_1(1^\lambda)$, and calls to the NIZK prover for honest senders, i.e., if $S \notin \mathcal{C}$, use τ , cfs , while if $S \in \mathcal{C}$, uses cfs . The rest of the execution is identical. The indistinguishability between the current hybrid and previous one follows from the zero-knowledge property of $\mathbf{N}_{\mathcal{R}}$. More formally, we prove the following:

Lemma 4. *For any adversary $\mathcal{A}$, there exists an adversary $\mathcal{B}$ such that*

$$\Pr \left[\mathbf{H}_{\mathcal{F}^3, \mathcal{S}^3}^3(\mathcal{A}) \Rightarrow 1 \right] - \Pr \left[\mathbf{H}_{\mathcal{F}^4, \mathcal{S}^4}^4(\mathcal{A}) \Rightarrow 1 \right] \leq \text{Adv}_{\mathbf{N}}^{\text{ZK}}(\mathcal{B}).$$

The probability runs over the randomness used by the experiments.

Proof. We build an adversary $\mathcal{B}$ against the zero-knowledge property of $\mathbf{N}_{\mathcal{R}}$, as follows: $\mathcal{B}$ receives crs from the zero-knowledge challenger and fully simulates the execution of $\mathbf{H}_{\mathcal{F}^3, \mathcal{S}^3}^3$ (or $\mathbf{H}_{\mathcal{F}^4, \mathcal{S}^4}^4$), diverging only as follows: if $S \notin \mathcal{C}$, on input message m for the **Send** operation, $\mathcal{B}$ computes $(c, c', \text{spk}_S, \text{pub}, \text{pk}_J, i), (m, k, \sigma, r, r', \mathbf{e}_J)$ as the protocol specifies, and forwards it to the challenger of $\mathbf{N}_{\mathcal{R}}$, receives π , and returns (c, c', π, i) as the output of the **Send** operation. When the execution halts, it outputs the bit output by $\mathcal{A}$. Clearly, if the zero-knowledge challenger of $\mathbf{N}_{\mathcal{R}}$ computes the real proofs, $\mathcal{B}$ perfectly simulates for $\mathcal{A}$ $\mathbf{H}_{\mathcal{F}^3, \mathcal{S}^3}^3$, otherwise, it perfectly simulates $\mathbf{H}_{\mathcal{F}^4, \mathcal{S}^4}^4$, therefore, the distinguishing probability is bounded by $\text{Adv}_{\mathbf{N}}^{\text{ZK}}(\mathcal{B})$, and the proof is concluded.

$\mathbf{H}_{\mathcal{F}^5, \mathcal{S}^5}^5$: In this hybrid $\mathcal{F}^5$ is identical to $\mathcal{F}^4$, while $\mathcal{S}^5$, instead of computing the CRS via $\text{crs} \leftarrow \mathbf{N}_{\mathcal{W}}.\text{Gen}(1^\lambda)$, computes $(\text{cfs}, \tau) \leftarrow \mathbf{N}_{\mathcal{W}}.\mathcal{NS}_1(1^\lambda)$, and calls to the NIZK prover for policy creators, i.e., if $\text{PC} \notin \mathcal{C}$, use τ , cfs , while if $\text{PC} \in \mathcal{C}$, uses cfs . The rest of the execution is identical. The indistinguishability between the current hybrid and previous one follows from the zero-knowledge property of $\mathbf{N}_{\mathcal{W}}$. More formally, we prove the following:

Lemma 5. *For any adversary $\mathcal{A}$, there exists an adversary $\mathcal{B}$ such that*

$$\Pr \left[\mathbf{H}_{\mathcal{F}^4, \mathcal{S}^4}^4(\mathcal{A}) \Rightarrow 1 \right] - \Pr \left[\mathbf{H}_{\mathcal{F}^5, \mathcal{S}^5}^5(\mathcal{A}) \Rightarrow 1 \right] \leq \text{Adv}_{\mathbf{N}}^{\text{ZK}}(\mathcal{B}).$$

The probability runs over the randomness used by the experiments.

Proof. We build an adversary $\mathcal{B}$ against the zero-knowledge property of $\mathbf{N}_{\mathcal{W}}$, as follows: $\mathcal{B}$ receives crs from the zero-knowledge challenger and fully simulates the execution of $\mathbf{H}_{\mathcal{F}^4, \mathcal{S}^4}^4$ (or $\mathbf{H}_{\mathcal{F}^5, \mathcal{S}^5}^5$), diverging only as follows: if $\text{PC} \notin \mathcal{C}$, on input policy $\mathbf{P}$ for the **SetPolicy** operation, $\mathcal{B}$ computes $(\text{pub}, (r, \mathbf{P}, \text{priv}))$ as the protocol specifies, and forwards it to the challenger of $\mathbf{N}_{\mathcal{W}}$, receives π , and

returns *ack* as the output of the **SetPolicy** operation. When the execution halts, it outputs the bit output by $\mathcal{A}$. Clearly, if the zero-knowledge challenger of $\mathbf{N}_{\mathcal{W}}$ computes the real proofs, $\mathcal{B}$ perfectly simulates for $\mathcal{A}$ $\mathbf{H}_{\mathcal{F}^4, \mathcal{S}^4}^4$, otherwise, it perfectly simulates $\mathbf{H}_{\mathcal{F}^5, \mathcal{S}^5}^5$, therefore, the distinguishing probability is bounded by $\text{Adv}_{\mathbf{N}}^{\text{ZK}}(\mathcal{B})$, and the proof is concluded.

$\mathbf{H}_{\mathcal{F}^6, \mathcal{S}^6}^6$: In this hybrid, if $\mathbf{S} \in \mathcal{C}$, then after $\mathbf{S}$ returns $\mathbf{e}$, if $\mathbf{N}_{\mathcal{R}}.\text{Vrf}(\text{cfs}, (c, c', \text{spk}_{\mathbf{S}}, \text{pub}, \text{pk}_{\mathbf{J}}, i), \pi)$ outputs 0 the execution continuous as in the previous hybrid, otherwise, the extractor is executed $\text{ext} \leftarrow \mathbf{N}_{\mathcal{R}}.\mathcal{E}(\tau, (c, c', \text{spk}_{\mathbf{S}}, \text{pub}, \text{pk}_{\mathbf{J}}, i), \pi)$, and if ext is $\perp$, then the execution aborts. Moreover, if $\mathbf{PC} \in \mathcal{C}$, after the adversary returns pub , π , if $\mathbf{N}_{\mathcal{W}}.\text{Vrf}(\text{cfs}, \text{pub}, \pi) = 1$, execute $\text{ext} \leftarrow \mathbf{N}_{\mathcal{W}}.\mathcal{E}(\tau, \text{pub}, \pi)$, and if $\text{ext} \neq \perp$, return the extracted policy $\mathbf{P}$ to $\mathcal{F}_{\text{PCSM}}$, otherwise abort.

Furthermore, some additional bookkeeping takes place: $\mathcal{S}^6$ returns to $\mathcal{F}^6$, $(\mathbf{e}, m, v)$, where v signifies whether $\mathbf{e}$ is a valid ciphertext, and m is a potentially extracted message (or $\perp$). If the ciphertext is valid, $\mathcal{S}^6$ stores $\mathbf{M} \leftarrow (m, \sigma, \mathbf{e}, \mathbf{e}_{\mathbf{J}}, b, i, \text{mid})$, where b is provided by $\mathcal{F}^6$.

Clearly, the additional bookkeeping steps do not affect the adversarial view and the current hybrid is distinguishable from the previous one only if $\mathbf{N}.\text{Vrf}$ outputs 1 and $\text{ext} = \perp$ (i.e., the extracted value is not a valid witness), which happens with negligible probability assuming the simulation soundness extractability of $\mathbf{N}$. This idea is formalized in the following lemma.

Lemma 6. *For any adversary $\mathcal{A}$, there exists an adversary $\mathcal{B}$ such that*

$$\Pr \left[\mathbf{H}_{\mathcal{F}^5, \mathcal{S}^5}^5(\mathcal{A}) \Rightarrow 1 \right] - \Pr \left[\mathbf{H}_{\mathcal{F}^6, \mathcal{S}^6}^6(\mathcal{A}) \Rightarrow 1 \right] \leq \text{Adv}_{\mathbf{N}}^{\text{SSE}}(\mathcal{B}).$$

The probability runs over the randomness used by the experiments.

Proof. We build an adversary $\mathcal{B}$ against the simulation sound extractability of $\mathbf{N}$, as follows: $\mathcal{B}$ receives cfs from the SSE challenger and fully simulates the execution of $\mathbf{H}_{\mathcal{F}^3, \mathcal{S}^3}^3$, diverging only as follows: if $\mathbf{S} \in \mathcal{C}$, on receive (c, c', π) from $\mathbf{S}$ if the proof verifies and the extracted value ext is invalid, output $((c, c', \text{spk}_{\mathbf{S}}, \text{pub}, \text{pk}_{\mathbf{J}}, i), \pi)$, ext , and halt. $\mathbf{PC} \in \mathcal{C}$, on receive (pub, π) from $\mathbf{PC}$, if the proof verifies and the extracted value ext is invalid, output (pub, π) , ext , and halt. The simulation is perfect, therefore the advantage of $\mathcal{A}$ in distinguishing between the two hybrids is bounded by $\text{Adv}_{\mathbf{N}}^{\text{SSE}}(\mathcal{B})$.

$\mathbf{H}_{\mathcal{F}^7, \mathcal{S}^7}^7$: $\mathcal{F}^7$ is the same with $\mathcal{F}^6$. $\mathcal{S}^7$ diverges from $\mathcal{S}^6$ as follows: if $\mathbf{A} \notin \mathcal{C}$, during audit, instead of evaluating the policy by decrypting the input ciphertexts using $d \leftarrow \mathbf{NISC}.\text{Dec}(\text{priv}, c')$, the hybrid checks whether $\exists (m, \sigma, \mathbf{e}_{\mathbf{J}}, b) : (m, \sigma, \mathbf{e}, \mathbf{e}_{\mathbf{J}}, b, i, \text{mid}) \in \mathbf{M}$, and then checks the value of b , where $b = \mathbf{P}(m)$, computed by $\mathcal{F}^7$. Note that $\mathbf{M}$ contains all valid ciphertexts, generated by honest as well as malicious senders, therefore, for inputs to **Audit** not in $\mathbf{M}$, an honest auditor would simply reject. The requirement, **req** $\exists (m, \sigma, \mathbf{e}_{\mathbf{J}}, b) : (m, \sigma, \mathbf{e}, \mathbf{e}_{\mathbf{J}}, b, i, \text{mid}) \in \mathbf{M}$, does not introduce any distance between the hybrids since it only enforces that the input to **Audit** must have been generated by **Send**, enforced by both the real

and ideal world executions. Therefore, the current hybrid can be distinguished from the previous one only if NISC.Init is honestly executed, then a potentially malicious sender computes a valid NISC.Enc output, while the output of an honest auditor on NISC.Dec is incorrect, which clearly violates the correctness of NISC . This idea is formalized as follows.

Lemma 7. *Assuming NISC satisfies $\varepsilon_{\text{nisc-cor}}$ -correctness, $\mathbf{H}_{\mathcal{F}^7, \mathcal{S}^7}^7$ cannot be distinguished from $\mathbf{H}_{\mathcal{F}^6, \mathcal{S}^6}^6$ with probability more than $\varepsilon_{\text{nisc-cor}}$, over the randomness used by the two experiments.*

Proof. Straightforward assuming the correctness of NISC .

$\mathbf{H}_{\mathcal{F}^8, \mathcal{S}^8}^8$: $\mathcal{F}^8$ is the same with $\mathcal{F}^7$. If $A \notin \mathcal{C}$, during report, instead of decrypting e_J , track the message (m, σ) that produced e_J , and use directly (m, σ) as the output of the decryption. Note here that, all the input ciphertexts e_J to J have been computed by the honest auditor since we don't allow injections in the channel between honest A and J (the ideal functionality only returns m if it stores a valid proof π for that m otherwise returns $\perp$), and the simulator knows both all the extracted messages chosen by $\mathcal{A}$ (if $S \in \mathcal{C}$) as well as the messages of honest S (if $S \notin \mathcal{C}$). The current hybrid is indistinguishable from the previous one assuming correctness of PKE .

Lemma 8. *Assuming PKE satisfies $\varepsilon_{\text{pke-cor}}$ -correctness, then the adversarial view in $\mathbf{H}_{\mathcal{F}^8, \mathcal{S}^8}^8$ is indistinguishable to that in $\mathbf{H}_{\mathcal{F}^7, \mathcal{S}^7}^7$ with probability $1 - \varepsilon_{\text{pke-cor}}$, over the randomness used by the two experiments.*

Proof. Straightforward assuming the correctness of PKE .

$\mathbf{H}_{\mathcal{F}^9, \mathcal{S}^9}^9$: $\mathcal{F}^9$ is the same with $\mathcal{F}^8$. If $A \notin \mathcal{C}$, during report, $\mathcal{S}^9$ does not execute the signature verification algorithm over $((m, i), \sigma)$. Here, the inputs to report have been honestly computed by the honest auditor thus the signatures should verify since we don't allow injections when $A \notin \mathcal{C}$. This idea is formalized below.

Lemma 9. *Assuming Σ satisfies $\varepsilon_{\Sigma\text{-cor}}$ -correctness, the adversarial view in $\mathbf{H}_{\mathcal{F}^9, \mathcal{S}^9}^9$ is indistinguishable to that in $\mathbf{H}_{\mathcal{F}^8, \mathcal{S}^8}^8$ with probability $1 - \varepsilon_{\Sigma\text{-cor}}$, over the randomness used by the two hybrids.*

Proof. Straightforward assuming the completeness of Σ .

$\mathbf{H}_{\mathcal{F}^{10}, \mathcal{S}^{10}}^{10}$: In the current hybrid, if $A \notin \mathcal{C}$, $\mathcal{F}^{10}, \mathcal{S}^{10}$, are such that the **Send**, **Audit** and **Report** operations' outputs are handled by $\mathcal{F}^{10}$ ($\mathcal{F}^9$ was only acting as a relay and bookkeeping functionality) as depicted in Fig. 9.

During **Send**, $\mathcal{S}^{10}$ computes (e, m, v) as in the previous hybrid, i.e., by executing the actual send operation for honest senders, and extracting for corrupted ones; for honest senders $\mathcal{S}^{10}$ receives $b := \mathbf{P}(m)$ as input from $\mathcal{F}^{10}$, while for corrupted ones, $\mathcal{S}^{10}$ extracts m , sends it to $\mathcal{F}^{10}$ and receives b . $\mathcal{S}^{10}$ returns (e, m, v) to $\mathcal{F}^{10}$, which executes the code of $\mathcal{F}_{\text{PCSM}}$ that simply checks if there

has been an extracted message m' (if $S \in \mathcal{C}$), in which case it evaluates the policy over m' and sends the outcome to the simulator. Furthermore it stores (m, c, v, mid) in $\mathbf{M}$, where v signifies if c is a valid ciphertext. Thus, the output of **Send** matches the output of the previous hybrid.

During **Audit**, $\mathcal{F}^{10}$ executes the code of $\mathcal{F}_{\text{PCSM}}$ which enforces the fact that the input ciphertext must have been already generated via **Send** ($\text{req}(\cdot, c, \cdot, \text{mid}) \in \mathbf{M}$) and if $A \notin \mathcal{C}$ it requires the input ciphertext to be valid ($(\cdot, c, \text{true}, \text{mid}) \in \mathbf{M}$). This is consistent with the real world protocol execution and the simulator. Then it evaluates the policy, performs bookkeeping, and then returns $v := c$ or $v := \mathbf{e}_J$, as in the previous hybrid.

On **Report** with input $\mathbf{e}_J$, the previous hybrid tracks (m, σ) that generated $\mathbf{e}_J$ and outputs m . Clearly, if there is no valid (m, σ) for $\mathbf{e}_J$, or the message index i is not fresh, then the previous hybrid outputs $\perp$, and this is the case for the current hybrid which outputs $\perp$ via $\mathcal{F}^{10}$ either $\nexists m : (m, \pi, \text{mid}) \in \mathbf{BM}$ or $(m, \text{mid}) \in \mathbf{RM}$. On the other hand if there exists $((m, i), \sigma)$ for $\mathbf{e}_J$, this implies that $\mathbf{P}(m) = 0$ and if the message has not been reported yet, $\mathcal{F}^{10}$ would add m to $\mathbf{BM}$, thus **Report** outputs m in both hybrids. The idea is formalized in the following lemma.

Lemma 10. *The adversarial view is identical in $\mathbf{H}_{\mathcal{F}^{10}, \mathcal{S}^{10}}^{10}$, $\mathbf{H}_{\mathcal{F}^9, \mathcal{S}^9}^9$.*

Proof. Follows by inspection as discussed above.

$\mathbf{H}_{\mathcal{F}^{11}, \mathcal{S}^{11}, i}^{11, i}$, for $0 \leq i \leq q : \mathcal{F}^{11}$ matches $\mathcal{F}^{10}$ and $\mathcal{S}^{11, 0}$ matches $\mathcal{S}^{10}$. Furthermore, for $i > 0$, if $S \notin \mathcal{C}$, during the **Send** of the i -th sent message m for which, if $\mathbf{P}(m) = 0$, i.e., $b = 0$ (the total number of such messages cannot be more than q but for the ease of exposition we assume it is exactly q), $\mathcal{S}^{11, i}$ fakes $\mathbf{e}_J$ to be the encryption of the zero message, and signature over that message and its unique index i . Note that from the previous hybrid $\mathbf{e}_J$ only acts as a proof that $\mathbf{P}(m) = 0$, and both the previous and the current hybrid do not decide the output of the report operation based on what it is encrypted inside $\mathbf{e}_J$. Furthermore, if $b = 1$, the hybrid fakes $\mathbf{e}_J$ to be the encryption of the zero message, i.e., no signature is computed for policy compliant messages. We set $\mathbf{H}_{\mathcal{F}^{11}, \mathcal{S}^{11}}^{11} := \mathbf{H}_{\mathcal{F}^{11}, \mathcal{S}^{11}, q}^{11, q}$ and we formalize this idea in the following lemma.

Lemma 11. *For any adversary $\mathcal{A}$, $i \in \{0, \dots, q-1\}$, there exists an adversary $\mathcal{B}_i$ such that*

$$\Pr \left[\mathbf{H}_{\mathcal{F}^{11}, \mathcal{S}^{11}, i+1}^{11, i+1}(\mathcal{A}) \Rightarrow 1 \right] - \Pr \left[\mathbf{H}_{\mathcal{F}^{11}, \mathcal{S}^{11}, i}^{11, i}(\mathcal{A}) \Rightarrow 1 \right] \leq \text{Adv}_{\text{PKE}}^{\text{IND-CCA}}(\mathcal{B}_i).$$

The probability runs over the randomness used by the experiments.

Proof. We define an adversary $\mathcal{B}_i$ against the IND-CCA security of PKE, as follows: $\mathcal{B}_i$ interacts with the CCA challenger and receives pk . It then simulates $\mathbf{H}_{\mathcal{F}^{11}, \mathcal{S}^{11}, i}^{11, i}$ up to the point where the **Send** operation takes place for the i -th message. If $b = 0$, instead of encrypting m, σ , under the J 's public key, it sends to the IND-CCA challenger of PKE $M_0 := (m, \sigma)$, $M_1 := (0, \sigma')$, receives back $\mathbf{e}_J^*$

and uses it for the rest of the execution. If $b = 0$, $M_1 = 0$. On a **Report** operation with input an honestly generated $e_j \neq e_j^*$, $\mathcal{B}_i$ simply simulates the operation using the plaintext that it used to create e_j ; on **Report** with input e_j^* , $\mathcal{B}_i$ simulates the operation using (m, σ) , while on input a newly created ciphertext e_j , $\mathcal{B}_i$ sends a decryption query to the IND-CCA challenger, and uses the output of the decryption oracle to simulate the **Report** operation. $\mathcal{B}_i$ outputs to the IND-CCA challenger the bit that is output by $\mathcal{A}$ when the execution halts. If the IND-CCA challenger encrypts M_0 , $\mathcal{A}$ simulates perfectly $\mathbf{H}_{\mathcal{F}^{11}, \mathcal{S}^{11}, i}^{11, i}$, otherwise it perfectly simulates $\mathbf{H}_{\mathcal{F}^{11}, \mathcal{S}^{11}, i+1}^{11, i+1}$, therefore the advantage of $\mathcal{A}$ in distinguishing between the two experiments is bounded by $\text{Adv}_{\text{PKE}}^{\text{IND-CCA}}(\mathcal{B})$, and the proof of the lemma is concluded.

$\mathbf{H}_{\mathcal{F}^{12}, \mathcal{S}^{12}}^{12}$: In the current hybrid, if $A \in \mathcal{C}$, $S \notin \mathcal{C}$, then during **Report**, after decryption of e_j and recovery of m, σ , $\mathcal{S}^{12}$ substitutes “**req** $\Sigma.\text{Ver}(\text{spk}, (m, i), \sigma)$ ” by “**req** $e, e'_j : (m, \sigma, e, e'_j, 0, i, \text{mid}) \in \mathbf{M}$ ”

The high level idea is that, $\mathcal{A}$ cannot mail a report that encrypts $((m, i), \sigma)$ to one that encrypts a related message, otherwise there is a forgery against Σ . This idea is formalized in the following lemma.

Lemma 12. *For any adversary $\mathcal{A}$, there exists an adversary $\mathcal{B}$ such that*

$$\Pr \left[\mathbf{H}_{\mathcal{F}^{12}, \mathcal{S}^{12}}^{12}(\mathcal{A}) \Rightarrow 1 \right] - \Pr \left[\mathbf{H}_{\mathcal{F}^{11}, \mathcal{S}^{11}}^{11}(\mathcal{A}) \Rightarrow 1 \right] \leq \text{Adv}_{\Sigma}^{\text{EU-CMA}}(\mathcal{B}) + \varepsilon_{\Sigma\text{-cor}}.$$

The probability runs over the randomness used by the experiments.

Proof. Observe that, if $e, e'_j : (m, \sigma, e, e'_j, 0, i, \text{mid}) \in \mathbf{M}$ holds, then by the definition of the hybrid, $((m, i), \sigma)$ has been generated by the simulator while simulating the honest sender, therefore $((m, i), \sigma)$ verifies correctly assuming correctness of Σ .

First observe that the hybrid computes signatures only if $b = 0$ for the input message m . If $\Sigma.\text{Ver}(\text{spk}, (m, i), \sigma) = 1$, then for (m, σ) there should exist e, e'_j , such that $(m, \sigma, e, e'_j, 0, i, \text{mid}) \in \mathbf{M}$, otherwise, we build an adversary $\mathcal{B}$ against the EU-CMA security of Σ , as follows: $\mathcal{B}$ simulates the execution of $\mathbf{H}_{\mathcal{F}^{12}, \mathcal{S}^{12}}^{12}$, and initially $\mathcal{B}$ interacts with the challenger of the unforgeability experiment of Σ to receive the public verification key, which is then used as the public verification key for the sender, and whenever the **Send** operation is executed over message m , $\mathcal{B}$ sends a sign query to the challenger of the unforgeability experiment and uses the returned signature σ for the rest of the execution. In the first call to **Report** for which $\Sigma.\text{Ver}(\text{spk}, (m, i), \sigma) = 1$ and $(m, \sigma, e, e'_j, 0, i, \text{mid}) \notin \mathbf{M}$, $\mathcal{B}$ outputs $((m, i), \sigma)$ as a forgery to the challenger. The simulation is perfect, therefore the advantage of $\mathcal{A}$ in distinguishing between the two hybrids is bounded by $\mathcal{B}$'s advantage against the EU-CMA security of Σ , and the proof is concluded.

$\mathbf{H}_{\mathcal{F}^{13}, \mathcal{S}^{13}}^{13}$: If $A \notin \mathcal{C}$, $S \in \mathcal{C}$, $\mathcal{S}^{13}$ diverges from $\mathcal{S}^{12}$ as follows: after extracting a valid witness k' from the senders proof, it aborts if $k \neq k'$ (here k is the key that will be used by R for decryption during **Deliver**), while $\text{SE.Dec}(k, c) \neq$

$\text{SE.Dec}(k', c)$ (and the output of the decryption is non- $\perp$). The indistinguishability between the current hybrid and the previous one relies on the commitment property of SE, and is formalized below.

Lemma 13. *For any adversary $\mathcal{A}$, there exists an adversary $\mathcal{B}$ such that*

$$\Pr \left[\mathbf{H}_{\mathcal{F}^{13}, \mathcal{S}^{13}}^{13}(\mathcal{A}) \Rightarrow 1 \right] - \Pr \left[\mathbf{H}_{\mathcal{F}^{12}, \mathcal{S}^{12}}^{12}(\mathcal{A}) \Rightarrow 1 \right] \leq \text{Adv}_{\text{SE}}^{\text{COM}}(\mathcal{B}).$$

The probability runs over the randomness used by the experiments.

Proof. We build an adversary $\mathcal{B}$ against the commitment property of SE, as follows: $\mathcal{B}$ simulates $\mathbf{H}_{\mathcal{F}^{13}, \mathcal{S}^{13}}^{13}$ and the first time $\mathbf{S}$ outputs (c', c, π, i) for which the extraction succeeds with extracted key $k' \neq k$ and message m' (here $k \leftarrow \mathcal{F}_{2\text{PR}}.\text{NextKey}(m)$) and $\text{SE.Dec}(k, c) \neq \text{SE.Dec}(k', c)$, it outputs $((k, m), (k', m'), c)$ and halts. The simulation is perfect, therefore, the advantage of $\mathcal{A}$ in breaking the commitment property of SE is bounded by $\text{Adv}_{\text{SE}}^{\text{COM}}(\mathcal{B})$, and the proof is concluded.

$\mathbf{H}_{\mathcal{F}^{14}, \mathcal{S}^{14}}^{14}$: If $\mathbf{A} \notin \mathcal{C}$, $\mathcal{F}^{14}$ diverges from $\mathcal{F}^{13}$ as follows: during **Deliver**, $\mathcal{F}^{14}$ does not forward anything to the simulator, and simply outputs the message that has been recorded w.r.t. the input ciphertext during the send operation. The current hybrid is indistinguishable from the previous one assuming correctness of SE, and we formalize this idea in the below lemma.

Lemma 14. *Assuming $\varepsilon_{\text{se-cor}}$ -correctness of SE, $\mathbf{H}_{\mathcal{F}^{14}, \mathcal{S}^{14}}^{14}$ is indistinguishable from $\mathbf{H}_{\mathcal{F}^{13}, \mathcal{S}^{13}}^{13}$ with probability $1 - \varepsilon_{\text{se-cor}}$, over the randomness used by the two experiments.*

Proof. Since $\mathbf{A} \notin \mathcal{C}$, during the **Send** operation, $\mathcal{S}^{14}$ returns to $\mathcal{F}^{14}$ the real symmetric ciphertext over the real message m (here the message is either extracted if $\mathbf{S} \in \mathcal{C}$ or provided as input by $\mathcal{F}^{14}$ if $\mathbf{S} \notin \mathcal{C}$). $\mathcal{F}^{14}$ records c and m and since $\mathbf{A} \notin \mathcal{C}$, the ideal functionality would output a non- $\perp$ message only if c has been already recorded, i.e., $(m, c, \cdot) \in \text{GM}$, therefore the current hybrid diverges from the previous one only if $\mathcal{F}^{14}$ returns the recorded m still in the previous hybrid $\text{SE.Dec}(k, c) \neq m$, which clearly violates the correctness property of SE. Recall here that we don't allow injections from $\mathbf{A}$ to $\mathbf{R}$ if $\mathbf{A} \notin \mathcal{C}$, therefore we avoid the case in which in the real world the adversary outputs a new (not recorded) ciphertext and decryption could possibly output a non- $\perp$ value, diverging from our ideal-world execution. This would require the additional authenticity property for SE (and one more reduction), which we omit on purpose.

$\mathbf{H}_{\mathcal{F}^{15}, \mathcal{S}^{15, i}}^{15, i}$, for $0 \leq i \leq q$: We define $\mathbf{H}_{\mathcal{F}^{15}, \mathcal{S}^{15, 0}}^{15, 0} := \mathbf{H}_{\mathcal{F}^{14}, \mathcal{S}^{14}}^{14}$ and if $\mathbf{S} \notin \mathcal{C}$, $\mathcal{S}^{15, i+1}$ diverges from $\mathcal{S}^{15, i}$ as follows: for the first i **Send** operations, $\mathcal{S}^{15, i}$ encrypts for SE the zero message. The indistinguishability between the current hybrid and the previous one follows from the IND-CCA security of SE and is formalized the following lemma.

Lemma 15. *For any adversary $\mathcal{A}$, $i \in \{0, \dots, q\}$, there exists an adversary $\mathcal{B}_i$ such that*

$$\Pr \left[\mathbf{H}_{\mathcal{F}^{15}, \mathcal{S}^{15, i+1}}^{15, i+1}(\mathcal{A}) \Rightarrow 1 \right] - \Pr \left[\mathbf{H}_{\mathcal{F}^{15}, \mathcal{S}^{15, i}}^{15, i}(\mathcal{A}) \Rightarrow 1 \right] \leq q \cdot \text{Adv}_{\text{SE}}^{\text{IND-CCA}}(\mathcal{B}_i).$$

The probability runs over the randomness used by the experiments.

Proof. We build an adversary $\mathcal{B}_i$ against the IND-CCA security of SE, as follows: $\mathcal{B}_i$ simulates $\mathbf{H}_{\mathcal{F}^{15}, \mathcal{S}^{15, i}}^{15, i}$ up to the point where it needs to execute the i -th **Send** operation, where instead of encrypting the actual message m w.r.t. SE, it sends to the IND-CCA challenger of SE $M_0 := m$, $M_1 := 0$, receives back c^* , uses it for the rest of the execution, and outputs to the IND-CCA challenger the bit that is output by $\mathcal{A}$ when the execution halts. In addition, if $A \in \mathcal{C}$, for all the delivery operations for $c \neq c^*$, $\mathcal{S}^{15}$ simulates delivery by querying the decryption oracle of SE. If the IND-CCA challenger encrypts M_0 , $\mathcal{B}$ simulates perfectly $\mathbf{H}_{\mathcal{F}^{15}, \mathcal{S}^{15, i}}^{15, i}$, otherwise it perfectly simulates $\mathbf{H}_{\mathcal{F}^{15}, \mathcal{S}^{15, i+1}}^{15, i+1}$, and the proof is concluded.

$\mathbf{H}_{\mathcal{F}^{16}, \mathcal{S}^{16}}^{16}$. If $S \notin \mathcal{C}$, $\mathcal{S}^{16}$ instead of computing the NISC ciphertext, c' , over m, e_J , simulates c' by executing the sender's simulator for NISC, i.e., computes $c' \leftarrow \text{NISC.S}_2(1^\lambda, \text{out})$, where, if $b = 1$ (i.e., m is policy compliant), then $\text{out} = \text{GOOD}$, otherwise, $\text{out} = e_J$, where e_J (from the previous hybrid), encrypts the zero message and a signature over it.

Lemma 16. *For any adversary $\mathcal{A}$, there exists an adversary $\mathcal{B}$ such that*

$$\Pr \left[\mathbf{H}_{\mathcal{F}^{16}, \mathcal{S}^{16}}^{16}(\mathcal{A}) \Rightarrow 1 \right] - \Pr \left[\mathbf{H}_{\mathcal{F}^{15}, \mathcal{S}^{15}}^{15}(\mathcal{A}) \Rightarrow 1 \right] \leq \text{Adv}_{\text{NISC}}^{\text{NISC}}(\mathcal{B}).$$

The probability runs over the randomness used by the experiments.

Proof. We define $\mathcal{B}$ that plays against the NISC, i.e., will either participate in $\text{REAL}_{\mathcal{B}}^{\langle S, A \rangle}(1^\lambda, (m, e_J), \mathbf{P})$ or $\text{IDEAL}_{\mathcal{S}}^{\mathcal{F}_{\text{sc}}}(1^\lambda, (m, e_J), \mathbf{P})$, where $\mathcal{F}_{\text{sc}}$ is the trusted party that computes $\mathcal{C}^{\mathcal{P}}((m, e_J), \mathbf{P})$, and $\mathcal{S}$ is an arbitrary simulator for $\mathcal{B}$. $\mathcal{B}$ plays the role of a corrupted party that sets $\mathbf{P}$; it receives as input a random variable X , where either $X = \text{REAL}_{\mathcal{B}}^{\langle S, A \rangle}(1^\lambda, (m, e_J), \mathbf{P})$ or $X = \text{IDEAL}_{\mathcal{S}}^{\mathcal{F}_{\text{sc}}}(1^\lambda, (m, e_J), \mathbf{P})$ and fully simulates $\mathbf{H}_{\mathcal{F}^{16}, \mathcal{S}^{16}}^{16}$, diverging only as follows: instead of computing c' on its own it returns the value of c' according to X . $\mathcal{B}$ outputs the bit output by $\mathcal{A}$ when the execution halts. Since $\mathcal{F}^{16}, \mathcal{S}^{16}$ is independent of m the simulation is perfect, and if $X = \text{REAL}_{\mathcal{B}}^{\langle S, A \rangle}(1^\lambda, (m, e_J), \mathbf{P})$, then $\mathcal{B}$ simulates $\mathbf{H}_{\mathcal{F}^{15}, \mathcal{S}^{15}}^{15}$, while if $X = \text{IDEAL}_{\mathcal{S}}^{\mathcal{F}_{\text{sc}}}(1^\lambda, (m, e_J), \mathbf{P})$, it perfectly simulates $\mathbf{H}_{\mathcal{F}^{16}, \mathcal{S}^{16}}^{16}$, therefore the distinguishing advantage of $\mathcal{A}$ against the two hybrids is bounded by $\text{Adv}_{\text{NISC}}^{\text{NISC}}(\mathcal{B})$.

$\mathbf{H}_{\mathcal{F}^{17}, \mathcal{S}^{17}}^{17}$. If $A \notin \mathcal{C}$, $\mathcal{S}^{17}$ instead of computing $(\text{pub}, \text{priv}) \leftarrow \text{NISC.Init}(1^\lambda, \mathbf{P}; r)$, simulates **pub** by executing the auditor's simulator for NISC, i.e., computes $\text{pub} \leftarrow \text{NISC.S}_1(1^\lambda)$.

Lemma 17. *For any adversary $\mathcal{A}$, there exists an adversary $\mathcal{B}$ such that*

$$\Pr \left[\mathbf{H}_{\mathcal{F}^{17}, \mathcal{S}^{17}}^{17}(\mathcal{A}) \Rightarrow 1 \right] - \Pr \left[\mathbf{H}_{\mathcal{F}^{16}, \mathcal{S}^{16}}^{16}(\mathcal{A}) \Rightarrow 1 \right] \leq \text{Adv}_{\text{NISC}}^{\text{NISC}}(\mathcal{B}).$$

The probability runs over the randomness used by the experiments.

Proof. We define $\mathcal{B}$ that plays NISC, i.e., will either participate in $\text{REAL}_B^{<S, A>}(1^\lambda, (m, e_J), \mathbf{P})$ or $\text{IDEAL}_{\mathcal{S}^{\mathcal{F}_{\text{sc}}}}(1^\lambda, (m, e_J), \mathbf{P})$, where $\mathcal{F}_{\text{sc}}$ is the trusted party that computes $\mathcal{C}^{\mathcal{P}}((m, e_J), \mathbf{P})$, and $\mathcal{S}$ is an arbitrary simulator for $\mathcal{B}$. $\mathcal{B}$ plays the role of a corrupted party that sets (m, e_J) ; it receives as input a random variable X , where either $X = \text{REAL}_B^{<S, A>}(1^\lambda, (m, e_J), \mathbf{P})$ or $X = \text{IDEAL}_{\mathcal{S}^{\mathcal{F}_{\text{sc}}}}(1^\lambda, (m, e_J), \mathbf{P})$ and fully simulates $\mathbf{H}_{\mathcal{F}^{16}, \mathcal{S}^{16}}^{16}$, diverging only as follows: instead of computing pub on its own it returns the value of pub according to X . $\mathcal{B}$ outputs the bit output by $\mathcal{A}$ when the execution halts. Since $\mathcal{F}^{16}, \mathcal{S}^{16}$ is independent of $\mathbf{P}$ the simulation is perfect, and if $X = \text{REAL}_B^{<S, A>}(1^\lambda, (m, e_J), \mathbf{P})$, then $\mathcal{B}$ simulates $\mathbf{H}_{\mathcal{F}^{16}, \mathcal{S}^{16}}^{16}$, while if $X = \text{IDEAL}_{\mathcal{S}^{\mathcal{F}_{\text{sc}}}}(1^\lambda, (m, e_J), \mathbf{P})$, it perfectly simulates $\mathbf{H}_{\mathcal{F}^{17}, \mathcal{S}^{17}}^{17}$, therefore the distinguishing advantage of $\mathcal{A}$ against the two hybrids is bounded by $\text{Adv}_{\text{NISC}}^{\text{NISC}}(\mathcal{B})$.

The proof is concluded.

B.2 Security Proof of H-POCO

In the current section we prove PCSM security of H-POCO (cf. Fig. 6), presented in the $\mathcal{F}_{\text{AKS}}$ (cf. Fig. 2), $\mathcal{F}_{2\text{PR}}$, $\mathcal{F}_{\text{PSI-AD}}$ (cf. Fig. 5)-hybrid world.

Theorem 5. *Let $\mathcal{F}_{\text{PCSM}}$ be the functionality of Fig. 1 w.r.t. weaker policy privacy, Σ be an EU-CMA secure signature scheme, SE be an IND-CCA-secure encryption scheme, PKE be an IND-CCA-secure PKE scheme, instantiating the protocol H-POCO of Fig. 6 in the $\mathcal{F}_{\text{AKS}}$, $\mathcal{F}_{2\text{PR}}$, $\mathcal{F}_{\text{PSI-AD}}$ -hybrid world. Then, for any $\mathcal{A}$, there exists an $\mathcal{S}$ and adversaries $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3$, s.t.*

$$\begin{aligned} & \Pr[\text{IDEAL}_{\mathcal{F}_{\text{PCSM}}, \mathcal{S}}(\mathcal{A}) \Rightarrow 1] - \Pr[\text{REAL}_{\text{H-POCO}, (\mathcal{F}_{\text{AKS}}, \mathcal{F}_{2\text{PR}}, \mathcal{F}_{\text{PSI-AD}})}(\mathcal{A}) \Rightarrow 1] \leq \\ & q \cdot \left(\text{Adv}_{\text{PKE}}^{\text{IND-CCA}}(\mathcal{B}_1) + \text{Adv}_{\text{SE}}^{\text{IND-CCA}}(\mathcal{B}_2) \right) + \text{Adv}_{\Sigma}^{\text{EU-CMA}}(\mathcal{B}_3), \\ & + \varepsilon_{\text{se-cor}} + \varepsilon_{\text{pke-cor}} + \varepsilon_{\Sigma\text{-cor}}, \end{aligned}$$

where q denotes the number of queries made by $\mathcal{A}$ and $\varepsilon_{\text{se-cor}}, \varepsilon_{\text{pke-cor}}, \varepsilon_{\Sigma\text{-cor}}$, denote upper bounds on the probability of violating the correctness of SE, PKE and Σ , respectively.

Our proof proceeds in hybrids, where each hybrid $\mathbf{H}^i$ is parameterized by a functionality $\mathcal{F}^i$ and simulator $\mathcal{S}^i$, that will eventually converge to the ideal-world execution IDEAL, with respect to the ideal functionality $\mathcal{F}_{\text{PCSM}}$ and simulator $\mathcal{S}_{\text{H-POCO}}$. The simulator $\mathcal{S}_{\text{H-POCO}}$ is depicted in Fig. 10. The hybrids are as follows.

$\mathbf{H}^0$: This hybrid is equal to the real world execution, $\text{REAL}_{\text{H-POCO}, \mathcal{F}_{\text{AKS}}, \mathcal{F}_{2\text{PR}}, \mathcal{F}_{\text{PSI-AD}}}(\mathcal{A})$.

$\mathcal{S}_{\text{H-POCO}}$

Parameters: $\lambda, \mathcal{P}, \mathcal{U}, \Sigma, \text{SE}, \text{PKE}, \text{Hash}$

```

// Executed on first input
Initialization
 $(\text{spk}_S, \text{ssk}_S) \leftarrow \Sigma.\text{KeyGen}$ 
 $(\text{sk}_J, \text{sk}_J) \leftarrow \text{PKE}.\text{KeyGen}$ 
 $\mathcal{F}_{2\text{PR}}.\text{init}; i \leftarrow 0; \text{RM} \leftarrow \emptyset$ 
if  $S \in \mathcal{C}$  then
  On query  $(\text{Gen}, \Sigma)$  from  $S$  return  $(\text{spk}_S, \text{ssk}_S)$ 
if  $A \in \mathcal{C}$  then
  On query  $(\text{GetPK}, S, \Sigma)$  from  $A$  return  $\text{spk}_S$ 

// Set policy
Input  $(\text{SetPolicy}, \mathcal{P})$  from  $\mathcal{F}_{\text{PCSM}}$ 
if  $\mathcal{P} \notin \mathcal{C}$  then return ack
else
  Send  $(\text{SetPolicy}, \mathcal{P})$  to  $\text{PC}$ 
  On receive  $(\text{SetPolicy}, \mathcal{P}')$  from  $\text{PC}$  send
   $(\text{ack}, \mathcal{P}')$  to  $\mathcal{F}_{\text{PCSM}}$ 
  if receive  $\perp$  from  $\mathcal{F}_{\text{PCSM}}$  then
    Send  $\perp$  to  $\text{PC}$ 
    return
  else
    Send ack to  $\text{PC}$ 
  return

// Send message
Input  $(\text{Send}, m, b)$  from  $\mathcal{F}_{\text{PCSM}}$ 
 $i++$ 
 $k \leftarrow \mathcal{F}_{2\text{PR}}.\text{GetKey}(i)$ 
if  $S \in \mathcal{C}$  then
  Send  $(\text{Send}, m)$  to  $\mathcal{A}$ 
  On query  $(\text{GetKey}, i)$  from  $\mathcal{A}$  send  $k$  to  $\mathcal{A}$ 
  Receive  $e$ 
  On query  $(\text{SetHash}, (y, e_J, i))$  from  $S$  to  $\mathcal{F}_{\text{PSI-AD}}$ 
   $(m, \sigma) \leftarrow \text{PKE}.\text{Dec}(\text{sk}_J, e_J)$ 
else
   $\sigma \leftarrow \Sigma.\text{Sign}(\text{ssk}_S, (0, i)); c \leftarrow \text{SE}.\text{Enc}(k, 0)$ 
   $e \leftarrow (c, i);$ 
  if  $b = 0$  then
     $e_J \leftarrow \text{PKE}.\text{Enc}(\text{pk}_J, (0, i))$ 
   $X \leftarrow (m, \sigma, e, e_J, i, b)$ 
   $m \leftarrow \perp$ 
  return  $(e, m)$ 

// Audit ciphertext – generated via Send
Input  $(\text{Audit}, e)$ 
 $(\cdot, i) \leftarrow e$ 
if  $A \in \mathcal{C}$  then
  Send  $(\text{Audit}, e)$  to  $A$ 
  Receive query  $(\text{InSet}, i)$  from  $A$ 
  req  $\exists (m, \sigma, e, e_J, b) : (m, \sigma, e, e_J, i, b) \in X \wedge e = (\cdot, i)$ 
  if  $b = 1$  then
     $d \leftarrow e; d' \leftarrow \text{ack}$ 
  else
     $d, d' \leftarrow e_J$ 
  if  $A \notin \mathcal{C}$  then
    return  $d$ 
  else
    Send  $d'$  to  $A$ ; receive  $v$ 
    return  $v$ 

// Deliver message – only deliver if  $A \in \mathcal{C}$ 
Input  $(\text{Deliver}, e)$ 
 $(c, i) \leftarrow e$ 
 $k \leftarrow \mathcal{F}_{2\text{PR}}.\text{GetKey}(i)$ 
 $m \leftarrow \text{SE}.\text{Dec}(k, c)$ 
return  $m$ 

// Report message – only receive if  $A \in \mathcal{C}$ 
Input  $(\text{Report}, (e_J, i))$ 
req  $\text{spk}_S \neq \varepsilon$ 
 $(m, \sigma) \leftarrow \text{PKE}.\text{Dec}(\text{sk}_J, e_J)$ 
if  $S \notin \mathcal{C}$  then
  req  $\exists e : (m, \sigma, e, \cdot, i, 0) \in X \wedge i \notin \text{RM}$ 
   $\text{RM} \leftarrow i$ 
  return  $e_J$ 
else
  req  $\Sigma.\text{Ver}(\text{spk}_S, m, \sigma)$ 
  return  $m$ 

```

Fig. 10: The simulator for the security proof of Theorem 5.

$\mathbf{H}_{\mathcal{F}^1, \mathcal{S}^1}^1$: This hybrid is identical to $\mathbf{H}^0$, but translated to the ideal-world language. That is, we replace the real world execution with H-POCO , $\mathcal{F}_{\text{AKS}}$, $\mathcal{F}_{2\text{PR}}$, $\mathcal{F}_{\text{PSI-AD}}$, by an execution with a “trivial” ideal functionality $\mathcal{F}^1$ and a “trivial” simulator $\mathcal{S}^1$. $\mathcal{F}^1$ provides no security guarantees but matches the syntax of $\mathcal{F}^1$. That is, $\mathcal{F}^1$ forwards all inputs to $\mathcal{S}^1$ and allows it to choose all outputs. $\mathcal{S}^1$ uses the inputs from $\mathcal{F}^1$ to run the code of $\text{REAL}_{\text{H-POCO}, \mathcal{F}_{\text{AKS}}, \mathcal{F}_{2\text{PR}}, \mathcal{F}_{\text{PSI-AD}}}$ for honest parties, i.e., $\mathcal{S}^1$, using the honest parties inputs (forwarded from $\mathcal{F}^1$), it fully simulates their execution for corrupted ones, and computes the outputs exactly as in the real world.

Lemma 18. *The view of the adversary in $\mathbf{H}^0$ is identical to the one in $\mathbf{H}_{\mathcal{F}^1, \mathcal{S}^1}^1$.*

Proof. The proof is straightforward by inspection.

$\mathbf{H}_{\mathcal{F}^2, \mathcal{S}^2}^2$: In this hybrid, $\mathcal{S}^2$ receives b from $\mathcal{F}_{\text{PCSM}}$ and for the **Send** operation, if $\mathbf{S} \in \mathcal{C}$, it fully simulates the execution with a corrupted $\mathbf{S}$, otherwise it fully simulates the **Send** operation for $\mathbf{S} \notin \mathcal{C}$ (as in the previous hybrid), as depicted in Fig. 10, with the additional step being that after receiving **query** (**SetHash**, (y, e_J, i)) from $\mathbf{S}$ to $\mathcal{F}_{\text{PSI-AD}}$ (if $\mathbf{S} \in \mathcal{C}$), executes $(m, \sigma) \leftarrow \text{PKE.Dec}(\text{sk}_J, e_J)$, and stores $(m, \sigma, e, e_J, i, b)$ to a list X . If $\mathbf{S} \notin \mathcal{C}$, stores $(m, \sigma, e, e_J, i, b)$ to X , where m is the original message input to $\mathbf{S}$. Thus, for **Send**, $\mathcal{S}^2$ does some additional bookkeeping, and besides that, it behaves as in the previous hybrid.

With respect to the **Audit** operation, $\mathcal{S}^2$, instead of using $\mathcal{F}_{\text{PSI-AD}}$ to decide the output of the audit, is using the policy evaluation output, b , provided by $\mathcal{F}_{\text{PCSM}}$ (cf. Fig. 10). In more detail, if $\mathbf{A} \notin \mathcal{C}$, then $\mathcal{S}^2$, if $b = 1$, returns the input ciphertext e , otherwise it returns e_J , where the later has been consistently computed by the **Send** operation. Then $\mathcal{F}^2$, if $b = 1$, returns the stored ciphertext in GM , otherwise it returns e_J , and this matches the output of the previous hybrid. If $\mathbf{A} \in \mathcal{C}$, $\mathcal{S}^2$ forwards the input to **Audit** to $\mathbf{A}$, receives back i , and perfectly simulates the **InSet** operation using the bit b . The two executions provide identical outputs and this is formalized in the following lemma.

Lemma 19. *The view of the adversary in $\mathbf{H}_{\mathcal{F}^1, \mathcal{S}^1}^1$ is identical to the one in $\mathbf{H}_{\mathcal{F}^2, \mathcal{S}^2}^2$.*

Proof. The proof is straightforward by inspection.

$\mathbf{H}_{\mathcal{F}^3, \mathcal{S}^3}^3$: $\mathcal{F}^3$ is the same with $\mathcal{F}^2$. If $\mathbf{A} \notin \mathcal{C}$, during report, instead of decrypting e_J , track the message (m, σ) that produced e_J , and use directly (m, σ) as the output of the decryption. Note here that, all the input ciphertexts e_J to $\mathbf{J}$ must have been computed by the honest auditor since we don't allow injections in the channel between honest $\mathbf{A}$ and $\mathbf{J}$ (the ideal functionality only returns m if it stored a valid proof π for that m otherwise returns $\perp$), and the simulator knows both all the messages chosen by $\mathcal{A}$ (if $\mathbf{S} \in \mathcal{C}$) as well as the messages of honest $\mathbf{S}$ (if $\mathbf{S} \notin \mathcal{C}$) (note that the output of **Report** is not decided by $\mathcal{F}_{\text{PCSM}}$). The current hybrid is indistinguishable from the previous one assuming correctness of PKE.

Lemma 20. *Assuming PKE satisfies $\varepsilon_{\text{pke-cor}}$ -correctness, then $\mathbf{H}_{\mathcal{F}^3, \mathcal{S}^3}^3$ is indistinguishable from $\mathbf{H}_{\mathcal{F}^2, \mathcal{S}^2}^2$ with probability $1 - \varepsilon_{\text{pke-cor}}$, where the probability runs over the randomness used by the two experiments.*

Proof. Straightforward assuming the correctness of PKE.

$\mathbf{H}_{\mathcal{F}^4, \mathcal{S}^4}^4$: $\mathcal{F}^4$ is the same with $\mathcal{F}^3$. If $\mathbf{A} \notin \mathcal{C}$, during report, $\mathcal{S}^4$ does not execute the signature verification algorithm over (m, σ) . Here, the inputs to report have been honestly computed by the honest auditor thus the signatures should verify since we don't allow injections when $\mathbf{A} \notin \mathcal{C}$. This idea is formalized below.

Lemma 21. *Assuming Σ satisfies $\varepsilon_{\Sigma\text{-cor}}$ -correctness, then $\mathbf{H}_{\mathcal{F}^4, \mathcal{S}^4}^4$ is indistinguishable from $\mathbf{H}_{\mathcal{F}^3, \mathcal{S}^3}^3$ with probability $1 - \varepsilon_{\Sigma\text{-cor}}$, where the probability runs over the randomness used by the two experiments.*

Proof. Straightforward assuming the completeness of Σ .

$\mathbf{H}_{\mathcal{F}^5, \mathcal{S}^5}^5$: In the current hybrid, if $A \notin \mathcal{C}$, $\mathcal{F}^5, \mathcal{S}^5$, are such that the **Send**, **Audit** and **Report** operations' outputs are handled by $\mathcal{F}^5$ ($\mathcal{F}^4$ was only acting as a relay in the previous hybrid without computing anything) as depicted in Fig. 10.

During **Send**, $\mathcal{S}^5$ computes (e, m) as in the previous hybrid, i.e., by executing the actual send operation for honest senders, and acting as a relay for corrupted ones. $\mathcal{S}^5$ returns (e, m) to $\mathcal{F}^5$, which executes the code of $\mathcal{F}_{\text{PCSM}}$ that simply performs the bookkeeping and returns e , matching the output of the previous hybrid.

During **Audit**, $\mathcal{F}^5$ executes the code of $\mathcal{F}_{\text{PCSM}}$ which enforces the fact that the input ciphertext has been already generated via **Send** and sends it to $\mathcal{S}^5$, which, given the policy evaluation bit b ($\mathcal{S}^5$ is still using the original message m) returns either c or e_J to $\mathcal{F}^5$, which performs the required bookkeeping and then returns c or e_J , as in the previous hybrid.

On **Report** with input e_J , the previous hybrid tracks (m, σ) that generated e_J and outputs m . Clearly, if there is no valid (m, σ) for e_J the previous hybrid outputs $\perp$, and this is the case for the current hybrid which outputs $\perp$ via $\mathcal{F}^5$. On the other hand if there exists (m, σ) for e_J , this implies that $\mathbf{P}(m) = 0$ and $\mathcal{F}^5$ would add m to BM, thus **Report** outputs m in both hybrids. The idea is formalized in the following lemma.

Lemma 22. *The adversarial view is identical in $\mathbf{H}_{\mathcal{F}^5, \mathcal{S}^5}^5, \mathbf{H}_{\mathcal{F}^4, \mathcal{S}^4}^4$.*

Proof. Follows by inspection as discussed above.

$\mathbf{H}_{\mathcal{F}^6, \mathcal{S}^6, i}^{6, i}$, for $0 \leq i \leq q$: $\mathcal{F}^6$ matches $\mathcal{F}^5$ and $\mathcal{S}^{6,0}$ matches $\mathcal{S}^5$. Furthermore, for $i > 0$, if $S \notin \mathcal{C}$, during **Send** of the i -th message m for which $\mathbf{P}(m) = 0$, $\mathcal{S}^{6, i}$ fakes e_J to be the encryption of the zero message and signature of that message and its unique index i . Note that from the previous hybrid e_J only possibly acts as a proof that $\mathbf{P}(m) = 0$, and both the previous and the current hybrid do not decide the output of the report operation based on what it is encrypted inside e_J . We set $\mathbf{H}_{\mathcal{F}^6, \mathcal{S}^6}^{6, q} := \mathbf{H}_{\mathcal{F}^6, \mathcal{S}^{6, q}}^{6, q}$ and we formalize this idea in the following lemma.

Lemma 23. *For any adversary $\mathcal{A}$, there exists an adversary $\mathcal{B}$ such that*

$$\Pr \left[\mathbf{H}_{\mathcal{F}^6, \mathcal{S}^{6, q}}^{6, q}(\mathcal{A}) \Rightarrow 1 \right] - \Pr \left[\mathbf{H}_{\mathcal{F}^5, \mathcal{S}^5}^5(\mathcal{A}) \Rightarrow 1 \right] \leq q \cdot \text{Adv}_{\text{PKE}}^{\text{IND-CCA}}(\mathcal{B}).$$

The probability runs over the randomness used by the experiments.

Proof. For intermediate hybrids, we define for the i -th hybrid $\mathcal{B}_i$, for $i \in \{0, \dots, q-1\}$ as follows: $\mathcal{B}_i$ interacts with the PKE IND-CCA challenger and receives pk . Then, fully simulates $\mathbf{H}_{\mathcal{F}^6, \mathcal{S}^{6, i}}^{6, i}$ up to the point where the send operation takes place for the i -th message for which $\mathbf{P}(m) = 0$, where instead of encrypting m, σ , under the J 's public key, it sends to the IND-CCA challenger of PKE $M_0 := (m, \sigma)$, $M_1 := (0, \sigma)$, receives back e_J^* and uses it for the rest of the execution, and outputs to the IND-CCA challenger the bit that is output by the adversary when the execution halts. On a **Report** operation with input an

honestly generated $e_J \neq e_J^*$, $\mathcal{B}_i$ simply simulates the operation using the plaintext that it used to create e_J ; on **Report** with input e_J^* , $\mathcal{B}_i$ simulates the operation using (m, σ) , while on input a newly created ciphertext e_J , $\mathcal{B}_i$ sends a decryption query to the IND-CCA challenger, and uses the output of the decryption oracle to simulate the **Report** operation. If the IND-CCA challenger encrypts M_0 , $\mathcal{B}$ simulates perfectly $\mathbf{H}_{\mathcal{F}^6, S^6, i}^{6, i}$, otherwise it perfectly simulates $\mathbf{H}_{\mathcal{F}^6, S^6, i+1}^{6, i+1}$, and the advantage of the adversary in distinguishing between the hybrids is bounded by the advantage of breaking the IND-CCA security of PKE. Using a standard probabilistic argument the proof of the lemma is concluded.

$\mathbf{H}_{\mathcal{F}^7, S^7}^7$: In the current hybrid, if $A \in \mathcal{C}$, $S \notin \mathcal{C}$, then during **Report**, after decryption of e_J and recovery of m, σ , S^7 substitutes “**req** $\Sigma.\text{Ver}(\text{spk}, m, \sigma)$ ” by “**req** $\exists e, e_J (m, \sigma, e, e_J, i, 0) \in X$ ”. Furthermore, the output of **Report** is returned to the adversary via $\mathcal{F}^7$. We prove the following lemma.

Lemma 24. *Assuming $\varepsilon_{\Sigma\text{-cor}}$ -correctness of Σ , for any adversary $\mathcal{A}$, there exists an adversary $\mathcal{B}$ such that*

$$\Pr \left[\mathbf{H}_{\mathcal{F}^7, S^7}^7(\mathcal{A}) \Rightarrow 1 \right] - \Pr \left[\mathbf{H}_{\mathcal{F}^6, S^6}^6(\mathcal{A}) \Rightarrow 1 \right] \leq \text{Adv}_{\Sigma}^{\text{EU-CMA}}(\mathcal{B}) + \varepsilon_{\Sigma\text{-cor}}.$$

The probability runs over the randomness used by the experiments.

Proof. For the first direction, if $\exists e, e_J (m, \sigma, e, e_J, i, 0) \in X$ holds, then by the definition of the hybrid, (m, σ) has been created by the simulator simulating an honest sender or by an honest but curious S , therefore (m, σ) verifies correctly assuming correctness of Σ .

For the second direction, if $\Sigma.\text{Ver}(\text{spk}, m, \sigma) = 1$, then for (m, σ) there should exist e, e_J , such that $(m, \sigma, e, e_J, i, 0) \in X$. The reason why $b = 0$ is that S is honest but curious, therefore, even if $A \in \mathcal{C}$, J would never receive via d' (computed by the simulator) an e_J that encrypts a “good” message together with its signature. We define $\mathcal{B}$ as follows: initially $\mathcal{B}$ interacts with the challenger of the unforgeability experiment of Σ to receive the public verification key, which is then used as the public verification key for the sender, and whenever the **Send** operation is executed over message m , $\mathcal{B}$ sends a signature query to the challenger of the unforgeability experiment and uses the returned signature σ for the rest of the execution. In the first call to **Report** for which $\Sigma.\text{Ver}(\text{spk}, m, \sigma) = 1$ and $(m, \sigma, \cdot, \cdot, \cdot) \notin X$, $\mathcal{B}$ outputs (m, σ) as a forgery to the challenger. Clearly, $\mathcal{B}$ perfectly simulates $\mathbf{H}_{\mathcal{F}^7, S^7}^7$ for $\mathcal{A}$, therefore, the distinguishing advantage of $\mathcal{A}$ is bounded by $\text{Adv}_{\Sigma}^{\text{EU-CMA}}(\mathcal{B})$.

With regards to the output of **Report**, S^7 returns e_J to $\mathcal{F}^7$, and since $b = 0$, by the description of the functionality, there should exist (m, mid) such that $(m, e_J, \text{mid}) \in \text{BM}$, and the functionality returns m , matching the output of the previous hybrid.

$\mathbf{H}_{\mathcal{F}^8, S^8}^8$: If $A \notin \mathcal{C}$, $\mathcal{F}^8$ diverges from $\mathcal{F}^7$ as follows: during **Deliver**, $\mathcal{F}^8$ does not forward anything to the simulator, and simply outputs the message that

has been recorded w.r.t. the input ciphertext during the send operation. The current hybrid is indistinguishable from the previous one assuming correctness of SE, and we formalize this idea in the below lemma.

Lemma 25. *Assuming $\varepsilon_{\text{se-corr}}$ -correctness of SE, then $\mathbf{H}_{\mathcal{F}^8, \mathcal{S}^8}^8$ cannot be distinguished from $\mathbf{H}_{\mathcal{F}^7, \mathcal{S}^7}^7$ with probability more than $\varepsilon_{\text{se-corr}}$, where the probability runs over the randomness used by the two experiments.*

Proof. Since $A \notin \mathcal{C}$, during the **Send** operation, $\mathcal{S}^8$ returns to $\mathcal{F}^8$ the real symmetric ciphertext over the real message m (here the message is still provided to the simulator as input by $\mathcal{F}^8$). $\mathcal{F}^8$ records c and m and since $A \notin \mathcal{C}$, the ideal functionality would output a non- $\perp$ message only if c has been already recorded, i.e., $(m, c, \cdot) \in \text{GM}$, therefore the current hybrid diverges from the previous one only if $\mathcal{F}^8$ returns the recorded m still in the previous hybrid $\text{SE.Dec}(k, c) \neq m$, which clearly violates the correctness property of SE. Recall here that we don't allow injections from A to R if $A \notin \mathcal{C}$, therefore we avoid the case in which in the real world the adversary outputs a new (not recorded) ciphertext and decryption could possibly output a non- $\perp$ value, diverging from our ideal-world execution. This would require the additional authenticity property for SE (and one more reduction), which we omit on purpose.

$\mathbf{H}_{\mathcal{F}^9, \mathcal{S}^{9,i}}^{9,i}$, for $0 \leq i \leq q$: We define $\mathbf{H}_{\mathcal{F}^9, \mathcal{S}^{9,0}}^{9,0} := \mathbf{H}_{\mathcal{F}^8, \mathcal{S}^8}^8$ and if $S \notin \mathcal{C}$, $\mathcal{S}^{9,i+1}$ diverges from $\mathcal{S}^{9,i}$ as follows: for the first i **Send** operations, $\mathcal{S}^{9,i}$ encrypts for SE the zero message. The indistinguishability between the current hybrid and the previous one follows from the IND-CCA security of SE and is formalized the following lemma.

Lemma 26. *For any adversary $\mathcal{A}$, there exists an adversary $\mathcal{B}$ such that*

$$\Pr \left[\mathbf{H}_{\mathcal{F}^9, \mathcal{S}^{9,q}}^{9,q}(\mathcal{A}) \Rightarrow 1 \right] - \Pr \left[\mathbf{H}_{\mathcal{F}^8, \mathcal{S}^8}^8(\mathcal{A}) \Rightarrow 1 \right] \leq q \cdot \text{Adv}_{\text{SE}}^{\text{IND-CCA}}(\mathcal{B}).$$

The probability runs over the randomness used by the experiments.

Proof. We consider intermediate hybrids and for $i \in \{0, \dots, q-1\}$, we define an adversary $\mathcal{B}_i$ as follows: $\mathcal{B}_i$ fully simulates $\mathbf{H}_{\mathcal{F}^{\text{PCSM}}, \mathcal{S}^{9,i}}^{9,i}$ up to the point where it needs to execute the i -th **Send** operation, where instead of encrypting the actual message m w.r.t. SE, it sends to the IND-CCA challenger of SE $M_0 := m$, $M_1 := 0$, receives back c , and uses it for the rest of the execution, and outputs to the IND-CCA challenger the bit that is output by $\mathcal{A}$ when the execution halts. In addition, if $A \in \mathcal{C}$, for all the delivery operations for $c' \neq c$, $\mathcal{B}_i$ simulates delivery by querying the decryption oracle of SE. Observe that if the IND-CCA challenger encrypts M_0 , $\mathcal{B}$ simulates perfectly $\mathbf{H}_{\mathcal{F}^9, \mathcal{S}^{9,i}}^{9,i}$, otherwise it perfectly simulates $\mathbf{H}_{\mathcal{F}^9, \mathcal{S}^{9,i+1}}^{9,i+1}$, and indistinguishability between adjacent hybrids follows from the IND-CCA security of SE. The proof of the lemma is concluded via a standard probabilistic argument.

The proof is concluded.